100 COSAS QUE TODO PROGRAMADOR DEBERÍA SABER

```
function resolverProblema() {
  let enfoque = analizar();
  let solucion = pensar();
  probar(solucion);
  aprender();
  mejorar();
}

if (persistencia) {
  exito();
} else {
  intentarDeNuevo();
}
```

```
// Código es pensamiento

const programador = {
  curioso: true,
  disciplinado: true,
  paciente: true,
  resolutivo: true
};

while (vida) {
  aprender();
  construir();
  inspirar();
}
```

POR
Miguel Ángel Durán

Introduction

Why this book?

This book was born out of the need to collect and modernize fundamental advice for programmers. It is inspired by the collective work **"97 Things Every Programmer Should Know"**, updating its spirit with the most relevant lessons from modern development.

The goal is to offer a roadmap of knowledge that transcends specific programming languages and frameworks. Tips that will be just as valid today as they will be in ten years.

What will you find here?

This is not just a list of advice; it is a tool to:

- **Reflect** on the daily practice of programming.
- **Discover** design and architecture principles that are often overlooked.
- **Internalize** concepts that will make you a better professional, one tip at a time.

Programming in the AI era

We live in an era where Artificial Intelligence can generate code in seconds. However, this makes **having solid foundations more important than ever**.

> *Why? Because AI is a tool, not a substitute for professional judgment.*

Understanding the "why" behind the code, knowing when a solution is scalable, detecting code smells, and applying robust design principles is what allows a programmer to guide the AI, rather than being guided by it.

Foundations are the basis that allow you to discern if what the AI generates is really the best solution for your problem.

Code is read much more often than it is written

Por Guido van Rossum - Fundamentals

It is one of the most uncomfortable truths in our profession: **we spend 90% of our time trying to understand code and only 10% actually writing it.**

This maxim, popularized by **Guido van Rossum**, isn't just an opinion—it's a statistical reality. Robert C. Martin (Uncle Bob) took it even further in his book *Clean Code*, stating that the ratio is **more than 10 to 1**. Every time you sit down to program, you are reading what you did yesterday, what your teammate did last month, or what was left behind by a developer who isn't even at the company anymore.

Cognitive Load: Your Scarcest Resource

Every time you open a file with variable names like `data`, `process()`, or `temp`, you are forcing your brain to perform extra work. You have to "download" the context, decipher the intent, and reconstruct the logic.

If that code is cryptic, your productivity doesn't just slow down; it stops. **Hard-to-read code generates mental fatigue**, and mental fatigue is the mother of all bugs.

The Danger of Being "Too Clever"

Many junior developers (and some not-so-junior ones) fall into the trap of **unnecessary cleverness**. They write impossible "one-liners" or use obscure language tricks to prove their mastery.

But true mastery isn't about writing code that nobody understands—it's about **writing code that feels obvious**. As Brian Kernighan said: *"Debugging is twice as hard as writing the code in the first place. If you write the code as cleverly as possible, you are not smart enough to debug it."*

Evolution of a Craft Developer

See how readability changes when we stop writing for the machine and start writing for other humans:

```python
# ❌ LEVEL 1: Cryptic
# What is 'l'? What does 'p' do? A total mystery.
def p(l):
    r = []
    for i in l:
        if i % 2 == 0:
            r.append(i * 2)
    return r

# ⚠️ LEVEL 2: "The Smartest in the Room"
# Very concise, but does it really help understand the business logic?
def f(l):
    return [x * 2 for x in l if x % 2 == 0]

# ✅ LEVEL 3: Professional and Readable
# Now we know we are doubling even numbers.
def get_doubled_evens(numbers):
    return [n * 2 for n in numbers if n % 2 == 0]

# 🚀 LEVEL 4: Code that reads like a book
# We extract the intention. No thinking, just reading.
def is_even(number):
    return number % 2 == 0

def get_doubled_evens(numbers):
    return [number * 2 for number in numbers if is_even(number)]
```

The next time you're about to hit `Enter`, pause. Don't think about whether the compiler will understand it (it doesn't have feelings). Think about the person who will have to read that code six months from now on a Friday at 5 PM. It will probably be you.

Optimize for understanding, not for character saving. Your success as a developer isn't measured by the complexity of your algorithms, but by how simple it is for others—and for your "future self"—to work with them.

The right type eliminates bugs before they exist

Por Anders Hejlsberg - Typing

For decades, the software industry debated between the agility of dynamic languages and the safety of static languages. However, thanks to the work of engineers like **Anders Hejlsberg** (architect of C# and TypeScript), today we understand that a modern type system is not a chain that holds us back, but a co-pilot that guides us.

The premise is radical but effective: **if the type system is rich enough, many of the errors we would normally encounter at runtime become physically impossible to write.**

Executable documentation

The biggest problem with traditional documentation (comments, diagrams, README files) is that **it lies**. Over time, code evolves and documentation stays behind. Types, however, are documentation that the compiler verifies with every keystroke.

When you define a type, you are not just saying "this is a number"; you are communicating to your peers (and your future self) what the limits of that data are and what operations are valid on it.

Make impossible states unrepresentable

This is the mindset that separates a junior developer from a senior one. Instead of filling your code with `if (data === null)` or `try-catch` validations, you should design your data structures so that invalid states cannot exist.

If your application has a user who can be "Anonymous" or "Registered", don't use a boolean `isRegistered` and optional fields. Use a **Discriminated Union**. In this way, the compiler will force you to handle each case and prevent you from accessing the email of a user who hasn't registered yet.

The power of advanced typing

Notice how we move from code that we "hope works" to code that we "guarantee works".

```typescript
// ❌ LEVEL 1: The danger of ambiguity
// What if the status is 'WAITING'? What if we send an empty id?
interface Order {
  id: string;
  status: string; // 'pending', 'shipped', 'delivered'...
}

// ✅ LEVEL 2: Nominal typing and Unions
// Now the compiler knows exactly which values are valid.
type OrderStatus = 'pending' | 'shipped' | 'delivered';

// 🔥 LEVEL 3: Making the impossible unrepresentable
// We design states that only contain the data they need.
interface PendingOrder {
  status: 'pending';
  createdAt: Date;
}

interface ShippedOrder {
  status: 'shipped';
  shippedAt: Date;
  trackingId: string;
}

type Order = PendingOrder | ShippedOrder;

function processOrder(order: Order) {
  if (order.status === 'shipped') {
    // Here the compiler KNOWS order.trackingId exists
    console.log(order.trackingId);
  } else {
    // Here order.trackingId would give a compilation error ❌
    // We prevent an "undefined" bug before it happens.
    console.log(order.createdAt);
  }
}
```

Typing is not bureaucracy. It is a conversation with the compiler where you explain your business rules and it makes sure no one breaks them by accident. Investing time in designing the right types at the beginning of a feature is probably the cheapest and most effective form of quality control in the history of computing.

As Anders Hejlsberg says: *"Types give you the confidence to refactor without fear"*. And a developer without fear is a much more creative and productive developer.

Choose boring technology

Por Margaret Hamilton - Strategy

Margaret Hamilton led the software team for NASA's Apollo program at the MIT Instrumentation Laboratory. When astronauts' lives depended on every line of code, there was no room to experiment with unproven technology. The Apollo Guidance Computer (AGC) was built on well-understood principles, verifiable architecture, and an operating system grounded in ideas that had already proven themselves. **Choosing "boring" technology wasn't a limitation — it was the engineering decision that saved the Apollo 11 mission.**

This philosophy has a modern name: *Choose Boring Technology*. The idea is that every team has a limited capacity to absorb novelty — roughly **three "innovation tokens"** you can spend however you like. Every unfamiliar technology you introduce consumes attention, creates uncertainty, and multiplies failure modes nobody knows how to diagnose. If you spend your tokens replacing PostgreSQL with the trendy new database, you have none left to innovate where it truly matters: your product.

What does "boring" mean?

"Boring" doesn't mean bad or obsolete. It means **well understood**. A boring technology is one whose failure modes are documented, whose limits are known, and whose solutions to common problems have been refined over years.

Hamilton's team at MIT understood this instinctively. The AGC had just 74 KB of ROM and 4 KB of RAM. They worked within those constraints using proven memory management and task-scheduling techniques. The "boring" parts of their approach — exhaustive verification, manually reviewed code, defensive architecture — are precisely what allowed the software to recover on its own when the computer overloaded during the lunar landing.

The Apollo 11 lesson

Three minutes before the Eagle touched down on the lunar surface, alarms 1202 and 1203 fired. The computer was overloaded because the rendezvous radar was sending unnecessary data. In that critical moment, Hamilton's software didn't crash. Thanks to its priority-based scheduling — built with well-understood process scheduling techniques — the AGC dropped low-priority tasks and kept the essentials: navigation and descent control.

Had the team experimented with an untested OS or unproven memory architecture, that automatic recovery might never have worked. **Reliability came from choosing the known and executing it flawlessly.**

Innovation tokens in practice

Imagine you're starting a new project. You have decisions to make:

```
❌ Spending all your innovation tokens:

- Database: SurrealDB (new, promising, small community)
- Backend: Bun + Hono (fast, but young ecosystem)
- Frontend: Qwik (innovative, but few developers know it)
- Infrastructure: Fly.io with Edge Computing
- Auth: Custom solution with WebAuthn

→ Result: 5 unknowns at once. Any bug could be in any layer
  and nobody has the experience to diagnose it.
```

```
✅ Spending tokens strategically (the Hamilton approach):

- Database: PostgreSQL (boring, reliable, exhaustively documented)
- Backend: Node.js + Express (boring, millions of answers online)
- Frontend: React (boring by now — and that's a good thing)
- Infrastructure: AWS/Vercel (predictable, well-documented)
- Auth: Auth0/Clerk (solved problem, don't reinvent it)

→ Innovation token spent on: your core differentiating feature

→ Result: 1 controlled unknown. The rest of the stack is predictable.
```

When to innovate

Hamilton didn't avoid innovation — she concentrated it. Her team invented revolutionary concepts like asynchronous error recovery and fault-tolerant software. But they built those innovations on a foundation of known techniques. Apply the same principle:

1. **Innovate on your differentiator**: If you're a real-time streaming company, that's where you spend your token — not on replacing your relational database.
2. **Innovate when the pain is real**: If PostgreSQL genuinely can't handle your load after optimization, then explore alternatives. But not before.
3. **Innovate one thing at a time**: Never change two things at once. If something breaks, you need to know exactly what caused it — just like in Apollo flight tests.

The hidden cost of new tech

Every new technology carries costs that don't show up in any benchmark: ramp-up time for the whole team, rare problems not yet on Google, an immature ecosystem with fewer libraries and integrations.

The most dangerous cost is the **bus factor**: only one or two people truly understand the new tool. If one leaves, the knowledge leaves with them. Hamilton addressed this in Apollo by insisting that code be understandable by the entire team, not just the author.

Next time you're tempted to adopt the latest technology, remember Margaret Hamilton's philosophy: focus your innovation where it truly matters and build everything else on solid ground. When lives depend on your software — or when your business does — **boring is your superpower.**

Good data structures simplify code

Por Linus Torvalds - Architecture

Linus Torvalds shared one of his most famous insights in a 2006 mailing list: **"Bad programmers worry about the code. Good programmers worry about data structures and their relationships"**.

This philosophy is at the core of Git's design. Instead of focusing on complex algorithms for file diffing, Linus focused on how data is stored (objects, commits, trees). If the data structure is right, the code that manages it becomes almost trivial.

The Problem: Logic-Heavy vs. Structure-First

Imagine you need to manage order statuses and which transitions are allowed.

The "Bad" Approach: Scattered Control Logic

Here, the programmer focuses on the "code" (the conditions). Every time a new state is added, the complexity grows exponentially.

```
type Status = 'pending' | 'paid' | 'shipped' | 'delivered' | 'cancelled';

function canChangeStatus(current: Status, next: Status): boolean {
  if (current === 'pending' && (next === 'paid' || next === 'cancelled')) {
    return true;
  }
  if (current === 'paid' && (next === 'shipped' || next === 'cancelled')) {
    return true;
  }
  if (current === 'shipped' && next === 'delivered') {
    return true;
  }
  // ... this becomes a mess that is hard to read and maintain
  return false;
}
```

The "Good" Approach: Data Structure as the Engine

Here, we define the "relationship" between data first. The code simply queries that structure.

```typescript
type Status = 'pending' | 'paid' | 'shipped' | 'delivered' | 'cancelled';

// The data structure defines the business rules
const ALLOWED_TRANSITIONS: Record<Status, Status[]> = {
  pending: ['paid', 'cancelled'],
  paid: ['shipped', 'cancelled'],
  shipped: ['delivered'],
  delivered: [],
  cancelled: []
};

function canChangeStatus(current: Status, next: Status): boolean {
  return ALLOWED_TRANSITIONS[current].includes(next);
}
```

Why is this better?

1. **Clarity:** Business rules are in one place and easy to read at a glance.
2. **Maintainability:** If you want to add a rule (e.g., from 'pending' to 'processing'), you just update an array in an object without touching the function's logic.
3. **Extensibility:** It's easy to add additional validations without breaking the main structure.

When faced with a complex logic problem, stop and ask: **"Is there a way to organize my data that makes this code unnecessary?"**

The best JavaScript is no JavaScript

Por Una Kravets - Performance

Una Kravets, Developer Advocate at Google Chrome, often says a phrase that might sound contradictory for a web developer: **"The best JavaScript is no JavaScript"**.

It's not that Una hates JavaScript; she understands that JavaScript is the most expensive resource we ship to the browser. It has to be downloaded, parsed, compiled, and executed. Every time we can replace JS logic with a native HTML or CSS capability, we win in performance, accessibility, and robustness.

The Declarative Revolution: Invoker Commands API

Historically, for a button to open a modal or a popover, we needed JavaScript to listen for the `click` event and call the corresponding method.

The new **Invoker Commands API** changes the game by allowing us to define this behavior declaratively directly in HTML.

The "Legacy" Approach: JavaScript as Glue

```
// We need to wait for JS to load for the button to work
const btn = document.querySelector('#open-dialog');
const dialog = document.querySelector('#my-dialog');

btn.addEventListener('click', () => {
  dialog.showModal();
});
```

The "Native" Approach: Declarative and JS-free

With `commandfor` and `command` attributes, the browser handles everything. It works instantly, even before any scripts load.

```
<!-- Without a single line of JavaScript -->
<button commandfor="my-dialog" command="show-modal">
  Open Modal
</button>

<dialog id="my-dialog">
  <p>Modal Content</p>
```

```
  <button commandfor="my-dialog" command="close">Close</button>
</dialog>
```

Popover: Goodbye to Heavy Dependencies

How many times have we installed a 20kb library just to show a tooltip or a dropdown menu? The native `popover` attribute solves this brilliantly.

```
<button popovertarget="my-popover">Show menu</button>

<div id="my-popover" popover>
  <p>This is a native popover that handles:</p>
  <ul>
    <li>Light dismiss (closing on outside click)</li>
    <li>Closing with the Escape key</li>
    <li>Automatic layering (infinite z-index management)</li>
  </ul>
</div>
```

Why Choose Native?

1. **Performance:** The browser executes native code (C++) much faster than any JS engine.
2. **Resilience:** If your page's JavaScript fails or is slow to load (due to a poor connection), native HTML still works.
3. **Accessibility (A11y):** Native elements like `<dialog>` or `popover` come with correct keyboard behavior and ARIA roles "out of the box."
4. **Less Code is Less Liability:** As Rich Harris would say, if you don't write the JS, you don't have to maintain or test it.

The next time you're about to write an `addEventListener`, stop for a second. Look around. It's very likely that the web standard has already evolved so that you don't need that code.

Embrace the platform. Use native. Write less JavaScript.

Code is a liability, not an asset

Por Rich Harris - Strategy

Rich Harris, the creator of Svelte, has a very clear vision of development: **"Code is a liability, not an asset"**.

We often think that the more code we write, the more value we are creating. Rich argues the opposite: every line of code you write is something that must be tested, maintained, documented, and inevitably contains bugs. Code is "weight" that you carry throughout the life of the project.

Less code, fewer problems

The best way to reduce this liability is to use the tools you already have at your disposal (like native browser APIs) instead of building complex abstractions.

The "Heavy" Approach: Reinventing the wheel

Sometimes we create complex logic for things the browser already knows how to do, adding code that we now have to maintain.

```
// Manual implementation of "scroll to top" behavior
function scrollToTopManual() {
  const duration = 500;
  const start = window.scrollY;
  const startTime = performance.now();

  function animate(currentTime: number) {
    const elapsed = currentTime - startTime;
    const progress = Math.min(elapsed / duration, 1);

    window.scrollTo(0, start * (1 - progress));

    if (progress < 1) {
      requestAnimationFrame(animate);
    }
  }

  requestAnimationFrame(animate);
}
```

The "Light" Approach: Use the platform

We take advantage of what already exists. Zero lines of our own logic, zero bugs to maintain.

```
// The browser already has an API for this
function scrollToTopNative() {
  window.scrollTo({
    top: 0,
    behavior: 'smooth'
  });
}
```

Svelte vs. React: A Matter of Liability

This philosophy is what gave birth to **Svelte**. Rich Harris questioned why modern frameworks forced users to download a heavy library (the "runtime") to manage the DOM.

- **React:** Ships a significant amount of code to the browser (the Virtual DOM, the event system, etc.). That code is a liability: it consumes memory, takes time to download, and must be executed on the user's device.
- **Svelte:** Moves that liability to **build time**. Svelte disappears in production, leaving only the minimal code necessary to update the DOM.

By reducing the amount of code that travels over the wire and that the browser must process, Svelte applies the mantra that the best code is the one that doesn't exist (on the client).

The Dilemma: Build vs. Buy (or Use a Library)

Sometimes, the temptation to "avoid dependencies" leads us to write hundreds of lines of "custom" code for common problems. This is where **opportunity cost** comes in.

Every hour you spend debugging your own validation system is an hour you're not spending on the features that actually differentiate your product.

The Value of Community and Tests

A small, well-maintained library with tests (like `date-fns` or `zod`) is often a **lower liability** than 500 lines of untested custom code.

- **Security:** Hundreds of eyes have reviewed that code.
- **Edge Cases:** Libraries often handle edge cases you haven't even considered.
- **Documentation:** It already exists, you don't have to write it yourself.

What Really Adds Value?

Your success is not measured by how many lines of "original" code you've written, but by how much value you've delivered with the **minimum maintenance cost** possible.

1. **Maintenance:** The code you don't write never breaks.
2. **Cognitive Load:** Less code means the team understands the project faster.
3. **Error Surface:** Every line of your own code is an opportunity for a bug that only you can fix.

The next time you're about to write a new feature, ask yourself: **"Is this code an asset for my company, or a burden for my future self?"**.

Optimize when it hurts, not before

Por Kent C. Dodds - Practices

Donald Knuth, one of the fathers of modern computing, once wrote: *"Premature optimization is the root of all evil"*. Decades later, in the era of JavaScript frameworks and real-time applications, this warning by **Kent C. Dodds** remains more relevant than ever.

The seduction of writing the fastest and most efficient code possible from the first minute is a trap for the ego. It makes us feel like elite engineers, but it often turns us into architects of unnecessary complexity.

The cycle of mastery: 3 Stages

To avoid this trap, Kent C. Dodds and other community leaders propose a sacred order that every developer should tattoo in their memory:

1. **Make it work:** Solve the problem. Validate your idea. Make sure the tests pass.
2. **Make it right:** Refactor. Improve variable names. Eliminate duplication. Make it readable for humans.
3. **Make it fast:** Only if you have evidence (real metrics) that performance is an issue.

The high cost of premature optimization

Why is it so dangerous to try to be fast before it's time?

- **Free complexity:** Optimizations usually require more complex algorithms and less intuitive structures. This drastically increases maintenance costs.
- **Edge bugs:** By pushing the machine, you are more likely to introduce subtle errors that only occur in extreme conditions and are very difficult to debug.
- **Wasted time:** We often spend hours optimizing a function that only runs once a day or represents 0.01% of the application's loading time.

A real example in Web development

In the React ecosystem, it is very common to see developers using `useMemo` or `useCallback` in absolutely every component "just in case".

```
// ❌ LEVEL 1: Optimization as a sport
// We are adding cognitive load and wasting memory to save
// a function that is extremely cheap to recreate.
```

```
const handleClick = useCallback(() => {
  console.log('Action');
}, []);

// ✅ LEVEL 2: Simplicity by default
// Clean code, easy to read and debug.
// We will only add useCallback if this component causes real rendering iss
const handleClick = () => {
  console.log('Action');
};
```

The irony is that sometimes the optimization infrastructure itself (like the comparisons React does in `useMemo`) can be more expensive than the task we are trying to optimize.

Optimizing is a transaction. You are exchanging **clarity** for **speed**. As in any business, you should only make the trade if the benefit outweighs the cost.

If you don't have a metric that says something is slow, don't touch it. Simplicity is the ultimate optimization because it reduces development time, the number of bugs, and team stress. As Kent C. Dodds says: *"Make sure your optimizations are necessary before they are permanent"*.

Clean code is not the end goal

Por Dan Abramov - Philosophy

Dan Abramov, in his famous essay *Goodbye, Clean Code*, reminds us of a truth often forgotten in the pursuit of perfection: **"Clean code is not a goal. It's a tool to help us deal with complexity"**.

Many developers, especially early in their careers, become obsessed with principles like **DRY** (Don't Repeat Yourself) or perfect abstraction. The problem arises when "cleaning" the code makes it so abstract that it becomes impossible to understand or change.

The Premature Abstraction Trap

Imagine you have two components that look somewhat similar. The "clean" impulse is to merge them into one.

The "Obsessive" Approach: Over-abstraction

Trying to make an ultra-generic component so everything is "clean" and there's no repetition.

```
// A component that tries to do too much just to be "DRY"
interface GenericCardProps {
  title: string;
  type: 'user' | 'product';
  onAction: () => void;
  showPrice?: boolean; // Product only
  avatarUrl?: string;  // User only
}

const GenericCard = ({ title, type, ...props }: GenericCardProps) => {
  return (
    <div className="card">
      <h2>{title}</h2>
      {type === 'product' && <span>{props.showPrice}</span>}
      {type === 'user' && <img src={props.avatarUrl} />}
      <button onClick={props.onAction}>Click</button>
    </div>
  );
};
```

Problem: As the requirements for 'user' and 'product' diverge, this component fills up with `ifs` and optional props. It's "clean" because there's no repeated code, but it's a maintenance nightmare.

The Pragmatic Approach: Code as a Tool

Accepting a bit of repetition to gain clarity and flexibility. Use clean code only where it actually reduces mental load.

```
// Simple, specific components
const UserCard = ({ name, avatarUrl, onProfileClick }: UserProps) => (
  <div className="card">
    <h2>{name}</h2>
    <img src={avatarUrl} alt={name} />
    <button onClick={onProfileClick}>View Profile</button>
  </div>
);

const ProductCard = ({ title, price, onBuy }: ProductProps) => (
  <div className="card">
    <h2>{title}</h2>
    <span>${price}</span>
    <button onClick={onBuy}>Buy Now</button>
  </div>
);
```

Key Takeaways

1. **Avoid Dogmatic Clean Code:** If a refactor makes the code harder to follow for a teammate, it's not an improvement.
2. **Accept Duplication:** It is much better to duplicate some code than to create the wrong abstraction.
3. **Measure Value:** Ask yourself: "Does this cleanup help me understand the system better, or does it just make me feel like a better programmer?"

Code is a means to an end (delivering value), not a work of art that must be immutable.

Curiosity is the engine of learning

Por Ada Lovelace - Learning

"The more I study, the more insatiable do I feel my genius for it to be." This statement by **Ada Lovelace**, written in a letter to her mother in 1841, was not a display of arrogance but the recognition of a driving force: an intellectual curiosity that knew no bounds.

In software development, curiosity is not an accessory; it is a fundamental technical competency. It is what allows us to navigate an ecosystem that redefines itself every few years.

The Dunning-Kruger Effect and the Learning Rollercoaster

When we start learning something new, we often experience what is known as the "Peak of Mount Ignorance" within the **Dunning-Kruger Effect**. With just a little knowledge, our confidence is at its peak. It's the moment when we believe that "we already understand everything."

However, as we delve deeper, we fall into the "Valley of Despair." This is where complexity reveals itself, and we realize the immensity of what we don't know. The difference between a developer who plateaus and one who reaches mastery is how they manage this phase. Curiosity is the bridge that allows you to cross that valley.

The Guitar Analogy: From Noise to Harmony

Learning to program has a fascinating parallel with learning a musical instrument, such as the guitar:

1. **The Physical and Cognitive Barrier:** At first, your fingers hurt (or your head hurts trying to understand recursion). The sound is dirty, the strings buzz, and coordinating your hands seems impossible. It is the moment when it's easiest to say: "I have no talent for this."
2. **The Pattern Epiphany:** One day, suddenly, your fingers find their place, and the *C major* chord sounds clean. In programming, it's that moment when you finally understand how a closure or a promise works.
3. **The Expansion of the Horizon:** Once you master the basic chords, you realize that you're not done; you've just begun. Now you understand that music theory, scales, rhythm, and composition exist. Curiosity transforms initial frustration into a desire for infinite exploration.

Insatiable Genius as a Professional Attitude

Lovelace anticipated that the Analytical Engine could process more than just numbers; she predicted it could create music and art if given the right algorithms. This "poetic" vision of science is what we now call *lateral thinking*.

A developer with an "insatiable genius":

- **Is not satisfied with code just working:** They want to understand *why* it works and how it impacts the system.
- **Embraces obsolescence:** They see the arrival of new technology not as a burden, but as an opportunity for rediscovery.
- **Practices intellectual humility:** They know that, in technology, being the one who knows the most in the room is just a sign that you're in the wrong room.

Richard Feynman used to say: *"What I cannot create, I do not understand."* Curiosity pushes us to try to create, to fail, and in that process, to truly understand. Do not fear the feeling that the path is infinite; in programming, if you stop feeling that you have much more to learn, it's because you've stopped moving forward.

Keep your curiosity insatiable. The day you believe you already know everything about software development will be the day your code begins to become obsolete.

Your application needs to work right now just once; it must be easy to change forever

Por Sandi Metz - Design

This quote by **Sandi Metz** summarizes the essence of software engineering: the real cost of an application is not in its creation, but in its maintenance.

Many developers obsess over making the code "work" today. If the requirement is met and the tests pass, they consider the job done. However, code that is not easy to change is, in reality, a technical liability.

The dilemma of Present vs. Future

As Sandi says in her book *Practical Object-Oriented Design in Ruby (POODR)*:

1. **Your application needs to work right now.** (Business value today).
2. **It must be easy to change forever.** (Business value tomorrow).

If you only fulfill the first point, you are building a trap. Successful software is that which survives its initial requirements.

A Ruby Example: From Coupling to Flexibility

Imagine a system that processes orders and sends notifications.

Code that "just works today"

This code meets the requirement, but it is coupled to a specific email implementation. If tomorrow we want to send an SMS or a Slack message, we will have to modify the `Order` class.

```ruby
class Order
  def finalize
    # ... logic to finalize the order ...
    send_confirmation
  end

  def send_confirmation
    # Hard coupling: Order knows about EmailNotifier
    notifier = EmailNotifier.new
    notifier.send("Your order has been finalized")
```

```
  end
end
```

Code "easy to change forever"

By applying **Dependency Injection**, the `Order` class no longer knows *how* the notification is sent, it only knows that something can send it.

```
class Order
  attr_reader :notifier

  def initialize(notifier:)
    @notifier = notifier
  end

  def finalize
    # ... logic to finalize the order ...
    notifier.send("Your order has been finalized")
  end
end

# Now we can change the behavior without touching the Order class
web_order = Order.new(notifier: EmailNotifier.new)
api_order = Order.new(notifier: SMSNotifier.new)
```

The 4 Pillars of Design (TRUE)

Sandi Metz proposes the **TRUE** acronym to evaluate if our code is easy to change:

- **T**ransparent: The consequences of a change should be obvious.
- **R**easonable: The cost of a change should be proportional to the benefit.
- **U**sable: The code should be reusable in new contexts.
- **E**xemplary: The code should invite those who maintain it to follow the same design.

Don't write code for the computer to understand today. Write code so that your colleague (or your "future self") can modify it tomorrow without fear. Architecture is not about predicting the future, but about being ready for when it changes.

The Boy Scout Rule

Por Robert C. Martin - Maintainability

"Always leave the campsite cleaner than you found it." This is the Boy Scouts' ideal, and Robert C. Martin, better known as **Uncle Bob**, adapted it to software development with a simple yet transformative premise: **always leave the code a little better than you found it.**

It's not about doing a massive refactor every time you open a file. It's about small, continuous victories against disorder and software entropy.

Software Entropy

Software, by nature, tends toward disorder. Every quick patch, every variable poorly named due to haste, and every obsolete comment increases "technical debt." If we do nothing, the code ends up becoming a swamp where no one wants to enter.

The Boy Scout Rule is the antidote to this degradation. If every time a developer touches a module they improve it even a little bit, the code doesn't just stop degrading—it **improves over time**.

What does "a little better" mean?

You don't need to rewrite the entire system. "Better" can be something as small as:

1. **Renaming a variable**: Changing `d` to `daysSinceCreation`.
2. **Extracting a function**: If you see a 5-line block inside an `if` that does something specific, extract it into a function with a descriptive name.
3. **Deleting dead code**: If you see a function that is no longer used or a comment that no longer applies, delete it.
4. **Simplifying an expression**: Replacing a complex `if/else` with a ternary operator or a guard clause.

Practical Example in TypeScript

Imagine you have to add a feature to this order service:

```
// ❌ BEFORE: Functional but improvable code
class OrderService {
  process(o: any) {
```

```
    if (o.status === 'pending' && o.items.length > 0) {
      // processing logic
      o.items.forEach((i: any) => {
        console.log('Processing item: ' + i.name);
      });
      o.processedAt = new Date();
      o.status = 'completed';
    }
  }
}
```

By applying the Boy Scout Rule while adding your change, you could leave it like this:

```
// ✅ AFTER: Applying the Boy Scout Rule
// We've typed the object, improved names, and used guard clauses
interface Order {
  id: string;
  status: 'pending' | 'completed';
  items: Array<{ name: string }>;
  processedAt?: Date;
}

class OrderService {
  process(order: Order) {
    const isProcessable = order.status === 'pending' && order.items.length
    if (!isProcessable) return;

    this.processItems(order.items);

    order.processedAt = new Date();
    order.status = 'completed';
  }

  private processItems(items: Order['items']) {
    items.forEach(item => {
      console.log(`Processing item: ${item.name}`);
    });
  }
}
```

The Accumulated Benefit

Imagine a team of 5 people. If each person makes one small improvement a day, by the end of the year the project will have received over 1,000 small improvements. That's the difference between a project that dies from its own complexity and one that remains young and agile.

Hint for the next tip: While cleaning is good, sometimes excessive zeal can be dangerous. In the next tip, we'll see the counterpoint: why trying to fix everything can end up being worse than leaving it as it was.

Security starts in the code

Por Parisa Tabriz - Security

In 2007, a young security engineer was hired by Google as a "hired hacker." Her job: find vulnerabilities in Google's web applications before attackers did. That person was **Parisa Tabriz**, who would go on to become VP of Chrome and lead one of the most significant security transformations on the Internet.

Her business card read **"Security Princess"** — a title she chose herself to demystify security and make it more approachable. Her philosophy is clear: **security isn't bolted on at the end, it's built from the very first line of code.**

The HTTPS lesson: changing the entire web

When Parisa began leading Chrome security, less than half of web pages used HTTPS. Most traffic traveled in plain text — passwords, banking data, private messages, all visible to anyone intercepting the connection.

Instead of waiting for sites to migrate voluntarily, Parisa made a bold call: Chrome would display a "Not Secure" warning next to the address bar on HTTP pages. The industry pushed back, but it worked. Within a few years, HTTPS adoption went from 50% to over 90%.

The deeper lesson: **making the secure path the easy path** is more effective than any audit. If your framework, linter, or CI/CD pipeline catches security issues automatically, your team will avoid them effortlessly.

Think like an attacker: threat modeling

Parisa manages **Project Zero**, Google's offensive security research team that hunts zero-day vulnerabilities across all kinds of software. Their approach isn't "wait for something bad to happen" — it's **attack your own system before someone else does**.

You don't need to be a security expert. Before writing a feature, ask yourself three questions:

1. **What data am I handling?** Personal, financial, or authentication data demands the highest level of protection.
2. **Who might want access?** A curious user, an automated bot, an experienced attacker — each requires a different defense.

3. **What's the worst-case scenario?** If the answer is "all my users' passwords leak," invest more time in that part of the code.

CSRF: the attack that impersonates you

While SQL injection and XSS are the most well-known attacks, **Cross-Site Request Forgery (CSRF)** is equally dangerous and far more subtle. The attacker doesn't need your password — they make your own browser perform actions on your behalf.

```
<!-- ❌ An "innocent" email with a hidden image -->
<img src="https://your-bank.com/api/transfer?to=attacker&amount=5000" />
<!-- Your browser sends the request with your active session cookie -->
```

```
// ✅ CSRF token protection
app.use((req, res, next) => {
  if (['POST', 'PUT', 'DELETE'].includes(req.method)) {
    const token = req.headers['x-csrf-token'];
    if (!token || token !== req.session.csrfToken) {
      return res.status(403).json({ error: 'Invalid CSRF token' });
    }
  }
  next();
});
```

The `SameSite` cookie attribute is another effective barrier that costs a single line:

```
// ✅ Cookies with SameSite to prevent CSRF
res.cookie('session', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
});
```

Make the secure choice the default

Parisa's philosophy boils down to one idea: **if the secure option requires extra effort, nobody will use it**. That's why her greatest impact wasn't finding bugs — it was changing Chrome's defaults for millions of users.

Apply the same principle in your code:

```
// ❌ Security as an optional parameter
function createServer(port, { https = false } = {}) {
```

```
  // Most developers won't even notice this option
}

// ✅ Secure by default, insecure only when explicitly requested
function createServer(port, { disableHTTPS = false } = {}) {
  if (disableHTTPS) {
    console.warn('⚠️ Running without HTTPS. Only use in development.');
  }
}
```

Security is everyone's responsibility

Parisa insists on something many companies overlook: **security isn't just the security team's job**. Every developer writing a form, every person configuring a server, every designer deciding what data to request — they're all part of the security chain.

This translates into concrete habits:

1. Validate on the server, never only on the client. Frontend validation is UX; backend validation is security.
2. Use `Content-Security-Policy` to control which scripts run on your page.
3. Set `Strict-Transport-Security` to enforce HTTPS without exceptions.
4. Review your dependencies regularly: `npm audit` or `pnpm audit` are not optional.
5. Rate-limit login attempts and API endpoints.

You don't need to be Google's "Security Princess" to write secure code. You just need to accept that every line of code is a security decision — and that the best defense starts the moment you open your editor. As Parisa proved by transforming the entire web: **when you make the secure path the easy path, everyone wins.**

Talk is cheap. Show me the code

Por Linus Torvalds - Communication

This phrase from **Linus Torvalds**, dropped in a Linux kernel mailing list thread in 2000, has become an industry mantra: **"Talk is cheap. Show me the code."**

It's a call to action, a brutal reminder that in software development, ideas are worthless until they're materialized. You can spend hours in meetings debating architectures, discussing patterns, or designing diagrams on a whiteboard, but at the end of the day, the only thing that matters is working code.

The original context

The phrase wasn't born as a motivational slogan. It emerged in the middle of a technical discussion where someone was proposing changes to the kernel without providing a concrete implementation. Linus, with his characteristically direct style, cut the debate short:

> *"Talk is cheap. Show me the code."*

It wasn't contempt for theory; it was a demand for **commitment**. If you believe your idea is better, prove it. Code is the great equalizer: your title, seniority, or eloquence don't matter. What matters is whether your solution works.

The trap of analysis paralysis

In development teams, it's easy to fall into infinite planning cycles:

- "We need to research more before starting."
- "First we need to properly define the architecture."
- "Let's do another spike to validate the approach."

While planning has its place, it's often a form of **procrastination disguised as professionalism**. Code, even an imperfect prototype, generates real information: you discover hidden dependencies, performance issues, and edge cases that no diagram will ever show you.

The value of the prototype

A working prototype, no matter how ugly, is worth more than a thousand PowerPoint slides.

```
// ❌ LEVEL 1: The theoretical proposal (weeks of meetings)
// "We should create a distributed caching system
// with eventual invalidation and configurable consistency..."

// ✅ LEVEL 2: The prototype (one afternoon of work)
// A simple Map that demonstrates the concept
const cache = new Map<string, { data: unknown; expiresAt: number }>();

function getCached<T>(key: string, fetcher: () => Promise<T>, ttlMs = 60000
  const cached = cache.get(key);
  if (cached && cached.expiresAt > Date.now()) {
    return Promise.resolve(cached.data as T);
  }

  return fetcher().then(data => {
    cache.set(key, { data, expiresAt: Date.now() + ttlMs });
    return data;
  });
}

// Now we can discuss something concrete:
// Does it work? What's missing? How do we scale this?
```

The "doer" culture

This philosophy doesn't mean you should ignore design or jump into coding without thinking. It means that:

1. **Discussions should be short:** If a technical debate lasts more than 15 minutes without code involved, you're probably going in circles.
2. **Prototypes kill debates:** A 50-line prototype can resolve a discussion that would take weeks of emails.
3. **Code is the contract:** Documentation lies, comments age, but executable code is the absolute truth of the system.

Open Source: The meritocracy of code

The open-source software world lives by this maxim. It doesn't matter if you're a student or a senior engineer at Google; if your Pull Request is good, it gets accepted. If it's not, it gets rejected. Code is the only resume that matters.

The next time you find yourself in an endless meeting about "how we should do X," remember Linus's words. Leave the meeting, open your editor, and prove your point with code. **Words convince, but code transforms.**

Premature optimization is the root of all evil

Por Donald Knuth - Performance

In 1974, **Donald Knuth**, considered the father of algorithm analysis, wrote one of the most quoted phrases in computing history: **"Premature optimization is the root of all evil."**

This quote, from his paper *"Structured Programming with go to Statements"*, is not an invitation to write slow code. It's a warning against one of the most seductive traps for programmers: sacrificing clarity, time, and sanity in pursuit of performance improvements that probably won't ever matter.

The full quote: The context everyone forgets

Almost no one remembers Knuth's complete statement:

> *"Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We should forget about small efficiencies, say about 97% of the time:* ***premature optimization is the root of all evil.*** *Yet we should not pass up our opportunities in that critical 3%."*

The real message is about **proportion**: 97% of your code doesn't need optimization. Only 3% is critical, and you must identify that 3% with **data**, not intuition.

The hidden cost of optimizing "just in case"

When you optimize without measuring, you pay a steep price:

1. **Unnecessary complexity:** Optimized code is usually harder to read and maintain.
2. **Wasted time:** Hours invested in micro-optimizations that don't move the needle.
3. **Subtle bugs:** Performance tricks introduce hard-to-detect errors.
4. **Rigidity:** Prematurely "optimized" code is usually harder to change.

Example: The micro-optimization trap

Imagine you're processing a list of users and decide to "optimize" before having any performance problem.

```
// ❌ LEVEL 1: Premature "optimization"
// We use a classic for loop because "it's faster than forEach"
// and reuse variables to "save memory"
function processUsersOptimized(users: User[]) {
  let i: number, len: number, user: User, result: string[] = [];
  for (i = 0, len = users.length; i < len; i++) {
    user = users[i];
    if (user.active) {
      result[result.length] = user.name.toUpperCase();
    }
  }
  return result;
}

// ✅ LEVEL 2: Clear and maintainable code
// Readable, declarative, and probably just as fast
// for any real-world use case (< 10,000 users)
function processUsersClear(users: User[]) {
  return users
    .filter(user => user.active)
    .map(user => user.name.toUpperCase());
}
```

The "optimized" version is harder to read, more error-prone, and... is it really faster? In most cases, the difference is imperceptible. Modern JavaScript engines optimize `filter` and `map` in ways that would surprise many.

The correct cycle: Measure, identify, optimize

Knuth left us a clear method:

1. **Write clear code first:** Make it work, make it readable, make it correct.
2. **Measure real performance:** Use profilers, benchmarks, production metrics.
3. **Identify the bottleneck:** Find that 3% that actually matters.
4. **Optimize with data:** Now yes, apply your performance knowledge where it has impact.

```
// The professional approach
async function processLargeDataset(data: Item[]) {
  // 1. First, make it work in a clear way
  const results = data
    .filter(item => item.isValid)
    .map(item => transformItem(item));

  // 2. If (and only if) there's a measured performance problem,
  // consider alternatives like batch processing
```

```
  // or streams, but not before.

  return results;
}
```

The wisdom of patience

As Knuth himself says, there's a time to optimize: when you have data to justify it. But that time is almost never "now" and almost never "here."

Your job as a professional developer is to deliver software that works, is maintainable, and solves real problems. Speed matters, but **development speed and change speed are usually more valuable than a few milliseconds of execution time**.

The next time you feel tempted to "optimize" something you haven't measured, remember Knuth's words. You might be cultivating the root of all evil in your code.

Abstraction is the key to survival

Por Barbara Liskov - Architecture

Computing is not about computers, but about managing complexity. **Barbara Liskov**, one of the brightest minds in software history and a Turing Award winner, left us a lesson that is more relevant today than ever: abstraction is not a luxury, it is our code's defense mechanism against time.

> *"Abstraction is the key to building systems that survive change."*

Change is the only constant

In a software project, the only thing we know for sure is that requirements will change. The database that is perfect today will be replaced tomorrow; the external API we use today will change its terms; or simply the business rules will evolve.

If our code knows too many details about how things work inside, it becomes rigid. When a detail changes, everything that depends on it breaks. Abstraction allows us to create an "interface" or a contract that hides those details, allowing us to change the implementation without affecting the rest of the system.

The Liskov Substitution Principle (LSP)

Barbara Liskov is world-famous for the principle that bears her name (the 'L' in SOLID). This principle states that if you have a base class and a subclass, you should be able to use the subclass anywhere you expect the base class, without anything breaking.

In TypeScript, this means our abstractions must be honest. It's not enough for types to match; **behavior** must also be consistent.

Practical example in TypeScript

Imagine a notification system. If we design the abstraction poorly, we'll end up with code full of conditionals. If we design it following the Liskov principle, the system will be infinitely extensible.

```
// ❌ THE ERROR: An abstraction that breaks the contract
// Some subtypes require extra steps or behave differently
interface Notifier {
  send(message: string): void;
```

```
}

class EmailNotifier implements Notifier {
  send(message: string) {
    console.log(`Sending Email: ${message}`);
  }
}

class SMSNotifier implements Notifier {
  // ⚠️ LISKOV VIOLATION:
  // We force the client to know a detail (the number)
  // that the general interface does not cover.
  sendWithNumber(message: string, phone: string) {
    console.log(`Sending SMS to ${phone}: ${message}`);
  }

  send(message: string) {
    throw new Error("Phone number required");
  }
}

// ✅ THE SOLUTION: Abstracting correctly
// The client doesn't need to know implementation details
interface MessageService {
  send(content: string): Promise<void>;
}

class EmailService implements MessageService {
  constructor(private email: string) {}
  async send(content: string) {
    console.log(`Email sent to ${this.email}: ${content}`);
  }
}

class PushService implements MessageService {
  constructor(private deviceId: string) {}
  async send(content: string) {
    console.log(`Push sent to device ${this.deviceId}: ${content}`);
  }
}

// Now any service can replace MessageService
// without the code using it having to change.
async function notifyUser(service: MessageService, message: string) {
  await service.send(message); // Works for ANY service
}
```

Why this saves your system

When you apply the correct abstraction:

1. **Decouple the "what" from the "how"**: Your business logic says "notify the user", not "connect to Amazon's SMTP server and send a packet".
2. **Facilitate testing**: You can replace real implementations with "mocks" that follow the same interface.
3. **Reduce fear of change**: If tomorrow you decide to use WhatsApp instead of SMS, you only have to create a new class that implements the interface. The rest of your application won't even notice the change.

Abstraction is not about making code harder to read by adding unnecessary layers; it's about creating the right boundaries so your system doesn't collapse under its own weight when the outside world changes.

Any fool can write code that a computer can understand

Por Martin Fowler - Readability

Martin Fowler, author of *Refactoring* and one of the most influential voices in software architecture, left us this devastating reflection: **"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."**

This phrase is a direct blow to the ego of programmers who take pride in their "clever" or "compact" code. The computer doesn't judge your code; it executes it blindly. It's your teammates, your future self, and the poor developers who will inherit your project who will suffer if you prioritize brevity over clarity.

The myth of "elegant" code

There's a dangerous confusion between *concise* code and *readable* code. A 200-character one-liner might be technically brilliant, but if your colleague needs 20 minutes to decipher it, it's not elegant; it's an obstacle.

```
// ❌ LEVEL 1: "Clever" (the computer understands it, you don't)
const r = d.filter(x => x.a && x.b > 5).reduce((p, c) => ({...p, [c.id]: c.

// ✅ LEVEL 2: Readable (humans understand it)
const TAX_MULTIPLIER = 1.1;

function calculateProductPrices(products: Product[]): Record<string, number
  const activeProducts = products.filter(product => product.isActive);
  const expensiveProducts = activeProducts.filter(product => product.price

  const priceMap: Record<string, number> = {};

  for (const product of expensiveProducts) {
    const basePrice = product.value;
    const finalPrice = product.hasTax ? basePrice * TAX_MULTIPLIER : basePr
    priceMap[product.id] = finalPrice;
  }

  return priceMap;
}
```

Does the second example have more lines? Yes. Does it take longer to write? A bit. Does it take less time to understand, modify, and debug? **Infinitely less.** And that's what matters.

Refactoring as an act of communication

Fowler's book *Refactoring* isn't just about improving code; it's about improving how we communicate through code. Each refactoring (extract method, rename variable, introduce explaining object) is a way to make the code's intention clearer to the next reader.

Fundamental clarity techniques

1. **Names that reveal intention:**

```
// ❌ Cryptic
const d = new Date().getTime() - u.c;

// ✅ Clear
const millisecondsSinceUserCreation = Date.now() - user.createdAt;
```

2. **Small functions with a single purpose:**

```
// ❌ Function that does "everything"
function processOrder(order) {
  // 50 lines of validation, tax calculation,
  // sending emails, inventory updates...
}

// ✅ Functions that read like a table of contents
function processOrder(order: Order) {
  validateOrder(order);
  const total = calculateTotalWithTax(order);
  updateInventory(order.items);
  sendConfirmationEmail(order.customer, total);
}
```

3. **Eliminate unnecessary comments by writing self-explanatory code:**

```
// ❌ The comment compensates for bad code
// Increment the failed attempts counter
cnt++;

// ✅ The code explains itself
failedLoginAttempts++;
```

The economic cost of unreadable code

Fowler understands that software is a business asset. Every minute a developer spends deciphering cryptic code is money lost. Every bug introduced because someone didn't understand a "clever" function is money lost. Every developer who abandons a project out of frustration is money lost.

Writing readable code is not a luxury or an aesthetic whim; it's an **economic investment** with guaranteed returns.

The readable code test

Before committing, ask yourself:

1. Would a junior developer on my team understand this without help?
2. Will I myself understand this in 6 months?
3. Can I read the code out loud and have it sound like a coherent sentence?

If the answer to any of these questions is "no," refactor. Not for the computer. **For the humans who will come after.**

Vibe Coding: Forget that the code even exists

Por Andrej Karpathy - AI

In February 2025, **Andrej Karpathy**, co-founder of OpenAI and former Director of AI at Tesla, coined a term that captures a seismic shift in how we program: **"Vibe Coding"**.

> *"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."*

The paradigm shift

For decades, programming has meant writing code: memorizing syntax, fighting with semicolons, understanding the difference between `==` and `===`. Vibe coding proposes something radical: **you describe what you want in natural language and AI generates the code for you**.

It's not assisted programming. It's not autocomplete. It's a fundamental change in the relationship between human and machine.

How does it work in practice?

```
// BEFORE: You write every line
function getActiveUserEmails(users: User[]): string[] {
  return users
    .filter(user => user.isActive && user.emailVerified)
    .map(user => user.email)
    .filter((email): email is string => email !== null);
}

// NOW: You describe, AI implements
// "Give me emails from active users who have verified email,
// excluding nulls"

// AI generates the code. You review and adjust.
```

The three levels of vibe coding

Level 1: Intelligent autocomplete

AI completes lines based on context. This is what Copilot and Cursor do in their basic mode. You're still writing code, just faster.

Level 2: Instruction-based generation

You describe a complete function or component in natural language. AI generates a code block that you review. It's "pair programming" with an AI.

Level 3: Pure vibe coding

You forget about code. You talk to the AI as if it were a very fast junior developer. You tell it what you want, it implements, you correct course. You iterate until it works.

The rules of vibe coding according to Karpathy

1. **Accept imperfection:** Generated code won't be perfect. Your job is to guide, not dictate every character.
2. **Think high-level:** Instead of "make a for loop," think "process all users and return a summary."
3. **Iterate fast:** The first attempt won't be right. Ask for changes, adjust, refine. The conversation IS the development process.
4. **Read the code (at least once):** It's not magic. It's code that must be correct, secure, and maintainable.

The risks you should know

Vibe coding has a dark side that Karpathy also acknowledges:

```
// ⚠️ DANGER: Code that "works" but you don't understand
// AI generated this. Do you know exactly what it does?
const result = data.reduce((acc, item) =>
  item.type === 'A'
    ? { ...acc, a: [...(acc.a || []), transform(item)] }
    : item.type === 'B'
      ? { ...acc, b: [...(acc.b || []), process(item)] }
      : acc
, {} as Record<string, unknown[]>);

// If you don't understand it, you can't debug it.
// If you can't debug it, you can't maintain it.
```

When to use vibe coding

✅ **Ideal for:**

- Prototypes and MVPs
- Automation scripts
- Idea exploration
- Learning new frameworks
- Boilerplate and repetitive code

⚠️ **With caution for:**

- Security-critical systems
- Production code without review
- Financial or medical algorithms
- Anything you can't verify

The future according to Karpathy

Karpathy predicts that in a few years, most code will be AI-generated. Developers will be "orchestra conductors" who guide and supervise, not "musicians" playing every note.

But that doesn't mean programming is irrelevant. On the contrary: **understanding code becomes more important when you're not the one writing it**. You must be able to read, evaluate, and correct what AI generates.

Vibe coding isn't the end of programming. It's its evolution. And like any powerful tool, its value depends on how you use it.

Embrace the vibes, but don't lose control.

There are only two hard things in Computer Science

Por Phil Karlton - Fundamentals

Phil Karlton, an engineer at Netscape during the golden age of browsers, said a phrase that has become legend: **"There are only two hard things in Computer Science: cache invalidation and naming things."**

This seemingly humorous quote hides a deep truth about the nature of software development. While cache invalidation is a complex technical problem (when does data expire? how do you sync distributed copies?), naming things is a **cognitive** problem that affects every line of code you write.

Why is naming so hard?

Naming variables, functions, classes, and modules seems trivial, but it's actually an exercise in **semantic compression**: you must capture the essence of what something *does* or *represents* in just a few words.

The problem is that:

1. **Language is ambiguous:** `data`, `info`, `item`, `element`... What do they really mean?
2. **Context changes:** A good name today might be confusing tomorrow when the code evolves.
3. **Abstraction is hard:** Naming a function that does multiple things forces you to admit it does too much.

The cost of bad names

Poor names have real consequences:

```
// ❌ LEVEL 1: Names that say nothing
function process(d: any[]) {
  const r = [];
  for (const i of d) {
    if (i.f) r.push(i.v * 2);
  }
  return r;
}
```

```
// What does it process? What is 'd'? What does 'f' mean?
// Every developer who reads this will waste time deciphering it.
```

```
// ✅ LEVEL 2: Names that reveal intention
function getDoubledPricesForActiveProducts(products: Product[]) {
  const doubledPrices: number[] = [];

  for (const product of products) {
    if (product.isActive) {
      doubledPrices.push(product.price * 2);
    }
  }

  return doubledPrices;
}

// Now anyone understands what this function does
// without needing to read its implementation.
```

The golden rule: The name is documentation

A good name eliminates the need for comments. If you need a comment to explain what a variable or function does, the name probably isn't good enough.

```
// ❌ The comment compensates for the bad name
// Time in milliseconds since the user registered
const t = Date.now() - user.createdAt;

// ✅ The name is the comment
const millisecondsSinceRegistration = Date.now() - user.createdAt;
```

Patterns for better naming

1. Functions: Verb + Noun

Functions *do* things. Their names should reflect that.

```
// ❌ Vague nouns
function user(id: string) { ... }
function validation(form: Form) { ... }

// ✅ Clear verbs
```

```
function getUserById(id: string) { ... }
function validateForm(form: Form) { ... }
```

2. Booleans: Yes/no question

Booleans answer questions. Name them as such.

```
// ❌ Ambiguous
const status = true;
const admin = false;

// ✅ Clear question
const isActive = true;
const hasAdminPrivileges = false;
const canEditPost = true;
```

3. Collections: Plural

Lists and arrays contain multiple elements.

```
// ❌ Singular for collection
const user = [user1, user2, user3];

// ✅ Clear plural
const users = [user1, user2, user3];
const activeUserIds = ['1', '2', '3'];
```

The "newcomer" test

Before confirming a name, imagine a developer new to the project reading it for the first time. Will they understand what it is without additional context? If the answer is no, find a better name.

As Phil Karlton said (and as any experienced developer will confirm), naming well is one of the most undervalued and most valuable skills in our profession. **A good name isn't decoration; it's design.**

Algorithms are not neutral: they have the face of who programs them

Por Joy Buolamwini - AI

Joy Buolamwini, MIT researcher and founder of the Algorithmic Justice League, discovered something disturbing while working on her thesis: facial recognition software couldn't detect her face. She was a woman and she was Black. The algorithm, trained mostly on white male faces, simply couldn't see her.

From this experience came her concept of **"The Coded Gaze"**: the idea that AI systems reflect the priorities, biases, and blind spots of those who create them.

The myth of algorithmic objectivity

There's a dangerous belief in the industry: that algorithms are "objective" because they're mathematical. Buolamwini dismantled this myth:

> *"Algorithms are opinions embedded in code."*

An algorithm doesn't decide anything by itself. It learns from the data we give it. And that data comes from a world full of prejudices, inequalities, and history.

The case that changed everything

In her research *Gender Shades*, Buolamwini analyzed facial recognition systems from Microsoft, IBM, and Face++. The results were devastating:

Demographic group	Error rate
White men	0.8%
White women	6.9%
Black men	12.0%
Black women	**34.7%**

The system worked almost perfectly for white men and failed one in three times for Black women. It wasn't a technical bug; it was a reflection of who was (and wasn't) in the training data.

What this means for developers

Every line of code you write has consequences in the real world. And when you train AI models, those consequences are amplified at massive scale.

```
// ❌ DANGER: Training with biased data
// If your "normal faces" dataset only has certain groups,
// your model will learn that those are the "normal" ones
const trainingData = await fetchFaces({
  sources: ['stock-photos-generic'],  // Biased toward certain groups
  limit: 10000
});

// ✅ BETTER: Audit and diversify the data
const trainingData = await fetchFaces({
  sources: ['diverse-faces-in-the-wild'],
  requirements: {
    genderBalance: true,
    ethnicityDistribution: 'representative',
    ageRange: [18, 80]
  }
});

// And still, audit the resulting model to detect biases
await auditModelForBias(trainedModel, testDatasets);
```

The questions you should ask yourself

Before deploying any system that makes decisions about people, ask yourself:

1. **Who is in the data?** If you train with internet data, you're training with the biases of the internet.
2. **Who is missing from the data?** Underrepresented groups will have worse outcomes.
3. **Who suffers the consequences of errors?** If your hiring algorithm fails, who doesn't get the job? If your credit algorithm fails, who doesn't get the loan?
4. **Can you explain why the system made a decision?** If the answer is "the model decided," you have a problem.

Joy's work continues

Thanks to Buolamwini's work, companies like IBM and Microsoft improved their systems. Amazon paused selling Rekognition to police. And cities like San Francisco banned government facial recognition.

But the battle isn't over. Every day, new AI systems are deployed that make decisions about who gets a loan, who gets hired, who gets surveilled.

As developers, we have a responsibility. It's not enough for the code to work. It must work **for everyone**.

The algorithm has your face. Make sure it reflects values you're proud of.

Simplicity is a prerequisite for reliability

Por Edsger W. Dijkstra - Architecture

Edsger W. Dijkstra, Turing Award winner and one of the founding fathers of computer science, once wrote: **"Simplicity is a prerequisite for reliability."**

It's not a "nice to have," it's not an aesthetic preference: it's a **necessary condition**. If your system isn't simple, it cannot be reliable. Period.

Dijkstra dedicated his career to demonstrating that complexity is the mortal enemy of correct software. His handwritten manuscripts (known as EWDs) are a goldmine of wisdom where he argues again and again that the only way to create systems that work is by reducing their complexity to the bare minimum.

Complexity: The silent enemy

Complexity doesn't arrive all at once. It accumulates line by line, feature by feature, patch by patch. One day you look at your code and realize that:

- No one fully understands the system.
- Small changes cause bugs in unexpected places.
- Onboarding time for new developers is measured in months, not days.
- There are parts of the code that are "better not to touch."

That's not a system; it's a **ticking time bomb**.

Simple vs. Easy: The crucial distinction

Dijkstra would warn us not to confuse "simple" with "easy." They're different concepts:

- **Easy:** Requires little effort now.
- **Simple:** Has few interconnected parts.

Sometimes the easy solution is complex (add another if, create another exception), while the simple solution requires more initial effort (redesign, refactor).

```
// ❌ LEVEL 1: "Easy" but complex
// Each special case adds complexity to the system
function calculateDiscount(user: User, product: Product, date: Date) {
  let discount = 0;

  if (user.isPremium) discount += 10;
```

```
  if (user.isPremium && product.category === 'electronics') discount += 5;
  if (date.getMonth() === 11) discount += 15; // December
  if (user.isPremium && date.getMonth() === 11) discount += 5; // Premium i
  if (product.price > 100) discount = Math.min(discount, 20); // Cap for ex
  if (user.isEmployee) discount = 30; // Employees always 30%

  // What happens if they're a premium employee in December with an expensi
  // Nobody knows for sure.

  return discount;
}
```

```
// ✅ LEVEL 2: Simple and predictable
// We separate the rules and make them composable
interface DiscountRule {
  name: string;
  calculate: (context: DiscountContext) => number;
}

const discountRules: DiscountRule[] = [
  { name: 'premium', calculate: (ctx) => ctx.user.isPremium ? 10 : 0 },
  { name: 'december', calculate: (ctx) => ctx.date.getMonth() === 11 ? 15 :
  { name: 'employee', calculate: (ctx) => ctx.user.isEmployee ? 30 : 0 },
];

function calculateDiscount(context: DiscountContext): number {
  // If employee, always 30% (priority rule, clear)
  if (context.user.isEmployee) return 30;

  // For everyone else, we sum applicable rules (max 25%)
  const total = discountRules
    .filter(rule => rule.name !== 'employee')
    .reduce((sum, rule) => sum + rule.calculate(context), 0);

  return Math.min(total, 25);
}
```

Dijkstra's principles

1. **"The competent programmer is fully aware of the strictly limited size of his own skull."** This is why simplicity matters: our brains can only hold so much complexity at once.

2. **"Testing can show the presence of bugs, but not their absence."** That's why simplicity is so important: in a simple system, you can **reason** about its correctness, not just test it.

3. **"The question of whether computers can think is like asking whether submarines can swim."** Dijkstra reminds us that analogies deceive. Simplicity forces us to be precise.

How to cultivate simplicity

1. **Say "no" more often:** Every feature adds complexity. Do you really need it?
2. **Delete before adding:** Before writing new code, ask if you can solve the problem by removing existing code.
3. **Look for invariants:** Truths that always hold in your system simplify reasoning about it.
4. **Distrust "flexibility":** "Flexible" code is often complex code in disguise. Build for the real use case, not for every possible case.

Dijkstra left us a profession and a legacy of rigor. Honoring that legacy means resisting the temptation of complexity and seeking, in every decision, **the simplest path that works**. Because if it's not simple, it won't be reliable. And if it's not reliable, it's useless.

AI should augment human intelligence, not replace it

Por Fei-Fei Li - AI

Fei-Fei Li, professor at Stanford and co-creator of ImageNet (the dataset that triggered the deep learning revolution), has defended for years a vision of AI that contrasts with the dominant apocalyptic narrative:

> *"I believe AI should augment human intelligence, not replace it."*

For Fei-Fei, the goal of artificial intelligence is not to create machines that think like humans, but tools that **amplify our capabilities** in areas where we're limited.

The ImageNet story: A humble beginning

In 2009, Fei-Fei Li and her team published ImageNet: a dataset with 14 million hand-labeled images across more than 20,000 categories. Nobody paid attention.

Three years later, in 2012, a model trained on ImageNet (AlexNet) won the image recognition competition by a margin nobody had seen before. That moment marked the beginning of the modern AI era.

But what many people don't know is that ImageNet was created to **study how humans learn**, not to create superintelligences. Fei-Fei wanted to understand human vision in order to teach machines.

Human-Centered AI (HAI)

At Stanford, Fei-Fei co-founded the Institute for Human-Centered Artificial Intelligence (HAI). Its premise is simple but radical:

AI should be designed, developed, and deployed with human wellbeing as its primary objective.

It's not AI for AI's sake. It's AI for people.

What this means in practice

```
// ❌ AI that replaces: The human disappears
class AutomatedHiringSystem {
  async processApplications(applications: Application[]): Promise<Decision[
    // The algorithm makes all decisions
    // The human never sees the applications
    return applications.map(app => this.model.predict(app));
  }
}

// ✅ AI that augments: The human is central
class AugmentedHiringAssistant {
  async analyzeApplications(applications: Application[]): Promise<Analysis[
    const analyses = await Promise.all(
      applications.map(async app => ({
        application: app,
        // AI identifies patterns and suggests
        skillsMatch: await this.analyzeSkillsMatch(app),
        potentialConcerns: await this.flagPotentialIssues(app),
        suggestedQuestions: await this.generateInterviewQuestions(app),
        // But the decision is human
        humanDecisionRequired: true
      }))
    );

    return analyses;
  }
}
```

The three pillars of human-centered AI

1. Augment, don't automate

The goal isn't to eliminate human jobs, but to enhance what humans do well:

- A radiologist with AI detects tumors better than a radiologist alone OR than an AI alone.
- A programmer with Copilot is more productive, but still makes the architecture decisions.

2. Keep humans in the loop

For important decisions, humans must be involved:

```
// The "Human in the Loop" pattern
interface AIRecommendation {
  suggestion: string;
```

```
  confidence: number;
  reasoning: string[];
  // The final decision is always human
  requiresHumanApproval: true;
}
```

3. Design for collaboration

The best AI tools aren't those that do everything alone, but those that work **with** people:

- They show their reasoning
- They accept corrections
- They learn from human decisions
- They explain their limitations

The future Fei-Fei imagines

Fei-Fei Li doesn't fear AI. She fears poorly designed AI. AI that ignores human needs. AI that concentrates power in few hands.

That's why she founded AI4ALL, an organization working to ensure AI is created by people from all backgrounds, not just by a homogeneous group of engineers.

Diversity in who creates AI determines who benefits from it.

The next time you work on an AI system, ask yourself: Am I building a tool that augments human capabilities? Or am I building something that makes humans irrelevant?

The answer to that question defines the future we're creating.

Adding manpower to a late project makes it later

Por Fred Brooks - Management

In 1975, **Fred Brooks** published *The Mythical Man-Month*, a book that forever changed how we understand software project management. His most devastating observation is known as **Brooks's Law**: **"Adding manpower to a late software project makes it later."**

This counterintuitive statement for any manager thinking in terms of "resources" has proven true again and again for over 50 years. Brooks won the Turing Award in 1999, partly for this contribution that remains painfully relevant.

Why more people = more delay?

1. The cost of communication

When you add a person to a team, you don't just add work capacity; you add **communication channels**. The formula is brutal:

```
Channels = n × (n − 1) / 2
```

- 3 people = 3 channels
- 5 people = 10 channels
- 10 people = 45 channels
- 20 people = 190 channels

Each channel is an opportunity for misunderstandings, sync meetings, and decisions requiring consensus.

2. Ramp-up time

New team members aren't productive from day one. They need to:

- Understand the business domain
- Learn the system architecture
- Get familiar with code conventions
- Integrate into the team dynamics

During this period, **they consume time from existing developers** who could be working on the project.

3. Division of labor has limits

Not all tasks can be parallelized. As Brooks says with a memorable analogy:

> *"Nine women can't have a baby in one month."*

Some parts of software have sequential dependencies. Adding more people doesn't speed up those parts; it only adds overhead.

The antipattern in action

```
// ❌ Traditional manager thinking
interface Project {
  deadline: Date;
  estimatedEffort: number; // in person-days
  teamSize: number;
}

function calculateNewDeadline(project: Project, additionalPeople: number):
  // "If we add 3 more people, we'll finish sooner"
  const newTeamSize = project.teamSize + additionalPeople;
  const daysRemaining = project.estimatedEffort / newTeamSize;

  // WRONG: This ignores Brooks's Law
  return addDays(new Date(), daysRemaining);
}
```

```
// ✅ Reality according to Brooks
function calculateRealDeadline(project: Project, additionalPeople: number):
  const newTeamSize = project.teamSize + additionalPeople;

  // Communication cost grows quadratically
  const communicationOverhead = (newTeamSize * (newTeamSize - 1)) / 2;

  // Time lost in onboarding (typically 2-4 weeks)
  const onboardingCost = additionalPeople * 15; // days

  // Effective work is reduced by coordination
  const effectiveCapacity = newTeamSize * 0.7; // 30% goes to coordination

  const adjustedDaysRemaining =
    (project.estimatedEffort + onboardingCost) / effectiveCapacity;

  // Probably worse than before
```

```
  return addDays(new Date(), adjustedDaysRemaining);
}
```

What to do when a project is running late?

Brooks doesn't say you can't do anything. He proposes alternatives:

1. **Reduce scope:** Which features are truly essential for launch?
2. **Move the date:** Sometimes honesty is the best policy. An admitted delay is better than a denied disaster.
3. **Improve tools and processes:** Are there bottlenecks that can be eliminated without adding people?
4. **Protect the team:** Remove interruptions, unnecessary meetings, and constant priority changes.
5. **If you must add people, do it early:** Brooks's Law is crueler the further along the project is. If you're going to grow the team, do it at the beginning, not the end.

The "mythical man-month"

The book's title refers to the fallacy of measuring work in "person-months" or "person-days," as if human labor were fungible and interchangeable. Brooks demonstrated that this metric is **a myth**.

One month of work from an experienced developer doesn't equal one month from a newcomer. Two developers working one month doesn't equal one developer working two months. **People are not interchangeable units.**

The next time someone suggests "throwing more people" at a project to save it, remember Fred Brooks's words. The solution is rarely in adding resources; it's in **better managing what you have and being realistic about what's possible**.

Technology should empower citizens, not control them

Por Audrey Tang - Civic Tech

Audrey Tang is not a typical digital minister. A hacker since age 8, Perl 6 contributor, and Taiwan's first Digital Minister, Audrey has demonstrated that technology can be a tool to strengthen democracy rather than erode it.

Her philosophy is summarized in one phrase:

> *"When we see the internet of things, let's make it an internet of beings."*

The Taiwanese model: Technology for participation

While other countries use technology to surveil, Taiwan uses it to **listen**. Audrey implemented platforms like vTaiwan, where citizens debate public policies and their opinions actually influence legislation.

The system works like this:

1. **A topic is proposed** (e.g., Uber regulation)
2. **Citizens share opinions** on an open platform
3. **AI clusters opinions** and finds consensus points
4. **The government acts** based on that consensus

It's not democracy by polls. It's **deliberative democracy scaled with technology**.

Radical Transparency: Everything public

Audrey practices what she calls "radical transparency." Her meetings with lobbyists and interest groups are recorded and published. Her logic is simple:

```
// The traditional model: Asymmetric information
interface TraditionalGov {
  publicMeetings: 'opaque';
  lobbyistInfluence: 'hidden';
  decisionProcess: 'black-box';
}

// Audrey's model: Radical transparency
interface RadicalTransparency {
```

```
  allMeetings: 'recorded-and-published';
  lobbyistInfluence: 'documented-publicly';
  decisionProcess: 'open-source';
}

// Result: Citizens can verify who influences what
```

Lessons for developers

1. Design for empowerment, not addiction

```
// ❌ Design for extraction (engagement at all costs)
const feedAlgorithm = {
  goal: 'maximize_time_on_site',
  methods: ['outrage_amplification', 'infinite_scroll', 'notification_spam'
};

// ✅ Design for empowerment
const feedAlgorithm = {
  goal: 'provide_value_to_user',
  methods: ['relevance_filtering', 'time_well_spent_metrics', 'clear_stoppi
};
```

2. Make technology appropriable

Audrey believes citizens should be able to modify the technological tools they use, not just consume them.

```
// Civic code should be open
interface CivicTool {
  sourceCode: 'open-source';
  dataFormat: 'open-standards';
  governance: 'community-driven';
}
```

3. Inclusion is not optional

During the pandemic, Taiwan distributed masks with a digital system that included elderly people without smartphones and people with disabilities. Technology that excludes is not civic technology; it's elitist technology.

The future Audrey imagines

Audrey sees a future where technology amplifies citizens' voices instead of silencing them. Where governments use AI to better understand their population, not to control it. Where open source is the norm, not the exception.

For developers, the message is clear: **the tools you build have political consequences**. A recommendation algorithm can polarize a society. A debate platform can heal it.

The question isn't whether your code will have political impact. It will. The question is: **what kind of impact will you choose to create?**

Make it work, make it right, make it fast

Por Kent Beck - Methodology

Kent Beck, creator of Test-Driven Development (TDD) and Extreme Programming, left us a mantra that should guide every line of code we write: **"Make it work, make it right, make it fast"** — in that exact order.

This order is not arbitrary; it's a deliberate strategy to avoid the most common traps in software development: analysis paralysis, premature optimization, and over-engineering.

The three phases of professional code

Phase 1: Make it work

The first objective is for the code to **do what it needs to do**. Nothing more. It doesn't matter if it's ugly, if it has duplication, or if it's not efficient. The only thing that matters is that the tests pass (or if you don't have tests, that the functionality is verified).

```
// PHASE 1: It works, but it's ugly
function calculateTotal(items: Item[]) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    if (items[i].type === 'product') {
      total = total + items[i].price;
      if (items[i].taxable) {
        total = total + (items[i].price * 0.21);
      }
    }
    if (items[i].type === 'service') {
      total = total + items[i].price;
      total = total + (items[i].price * 0.21); // Services always have VAT
    }
  }
  return total;
}

// ✅ It works. The test passes. Next phase.
```

Phase 2: Make it right

Now that it works, it's time to **refactor**. Remove duplication, improve names, extract functions, apply patterns where they make sense. The goal is for the code to be

readable, maintainable, and clearly express its intention.

```
// PHASE 2: Refactored and clear
const TAX_RATE = 0.21;

interface Item {
  type: 'product' | 'service';
  price: number;
  taxable?: boolean;
}

function calculateItemTax(item: Item): number {
  const isTaxable = item.type === 'service' || item.taxable;
  return isTaxable ? item.price * TAX_RATE : 0;
}

function calculateTotal(items: Item[]): number {
  return items.reduce((total, item) => {
    return total + item.price + calculateItemTax(item);
  }, 0);
}

// ✅ Readable, maintainable, testable. Next phase (if needed).
```

Phase 3: Make it fast

Only when you have **evidence** that performance is a problem, you optimize. This phase may never come for many functions, and that's fine. Most code doesn't need to be "fast"; it needs to be correct and maintainable.

```
// PHASE 3: Optimized (only if there's evidence it's necessary)
// For example, if we process millions of items and the profiler
// shows this function is a bottleneck

function calculateTotalOptimized(items: Item[]): number {
  let total = 0;
  const len = items.length;

  for (let i = 0; i < len; i++) {
    const item = items[i];
    const price = item.price;
    total += price;

    if (item.type === 'service' || item.taxable) {
      total += price * 0.21; // Inlined to avoid function call
    }
  }
```

```
  return total;
}

// ⚠️ Faster, but less readable.
// Only justified with real performance data.
```

Why the order matters

❌ If you try to make it fast first

You optimize code that might not even work correctly. You introduce unnecessary complexity. You waste time on micro-optimizations that don't matter.

❌ If you try to make it right first

You fall into analysis paralysis. You design abstractions for cases that don't exist. You never get to validate if your solution solves the real problem.

✅ If you follow the correct order

1. **You validate quickly:** You discover if your approach is viable before investing in polishing it.
2. **You refactor safely:** The tests from phase 1 protect you while improving the code.
3. **You optimize with data:** You only touch performance when you have metrics that justify it.

TDD: The guardian of the process

Kent Beck created TDD precisely to enforce this order:

1. **Red:** Write a failing test. Define what "working" means.
2. **Green:** Write the minimum code to make the test pass. "Make it work".
3. **Refactor:** Improve the code without breaking the test. "Make it right".

The Red-Green-Refactor cycle is the practical implementation of "Make it work, make it right, make it fast" (with the "fast" phase reserved for when there's evidence).

The perfectionism trap

Many developers skip phase 1 because they're embarrassed to write "ugly" code. But ugly code that works is infinitely more valuable than beautiful code that doesn't exist.

As Kent Beck says: *"I'm not a great programmer; I'm just a good programmer with great habits."*

The habit of following this order — work, improve, optimize — is what separates professionals from amateurs. **First solve the problem. Then make it pretty. And only at the end, if necessary, make it fast.**

Accessibility is not a feature, it's a right

Por Sara Soueidan - Accessibility

Sara Soueidan, UI/UX engineer and consultant specializing in web accessibility, has dedicated her career to an uncomfortable truth for many development teams: **accessibility is not a "nice to have"**. It must be in the project's DNA from the start.

> *"Accessibility is not about disabilities. It's about usability for all."*

Every time you say "we'll make it accessible later," you're saying "some people don't matter now." And that "later" almost never comes.

The 1% myth

Many teams justify ignoring accessibility with statistics: "Only 1% of our users have disabilities." This figure is wrong for two reasons:

1. **Disabilities are more common than you think:** 15-20% of the world's population has some type of disability (WHO). Many are invisible: color blindness, dyslexia, anxiety, ADHD.

2. **Accessibility benefits everyone:** Have you ever used subtitles in a noisy bar? Navigated with one hand while holding something? Squinted because of sun glare? That's situational disability.

3. **Almost everyone experiences temporary disability at some point:** A cast on your arm or leg? Reduced mobility. An ear infection? Temporary hearing loss. A migraine or stress? Harder to focus. The list goes on.

Accessibility from design

Sara insists that accessibility cannot be a patch. It must be in the project's DNA from the first wireframe.

```
<!-- ❌ HTML that excludes -->
<div onclick="handleClick()" class="button">
  <span class="icon-cart"></span>
</div>
<!--
  Problems:
  - Not keyboard navigable
  - Screen readers don't know it's a button
```

```
  - No alternative text
-->

<!-- ✅ HTML that includes -->
<button
  type="button"
  onclick="handleClick()"
  aria-label="Add to cart"
>
  <span class="icon-cart" aria-hidden="true"></span>
  <span class="visually-hidden">Add to cart</span>
</button>
<!--
  - Navigable with Tab
  - Activatable with Enter and Space
  - Correctly announced by screen readers
-->
```

The four principles of accessibility (POUR)

According to WCAG (Web Content Accessibility Guidelines), web content must be:

1. Perceivable

Information can't rely on a single sense, like color.

❌ Wrong - color only

```
.error {
  color: red;
}
```

If the user can't perceive red (or can't see the screen well), the message may be lost.

⚠️ Partially better - visual redundancy only

```
.error {
  color: red;
  border-left: 4px solid red;
}
```

This helps visually, but CSS-generated text is not reliable for assistive technologies.

✅ Correct - real, perceivable information

```html
<p class="error">
  <span aria-hidden="true">⚠️ </span>
  <strong>Error:</strong> Email is not valid
</p>
```

```css
.error {
  color: red;
  border-left: 4px solid red;
  padding-left: 0.5rem;
}
```

The information is in the HTML, doesn't rely on color alone, and is perceivable through multiple channels.

2. Operable

```html
<!-- ❌ Looks like a button, but it's not operable with keyboard (no focus,
<div class="close" onclick="closeModal()">×</div>

<!-- ✅ Real button: tabbable, focusable and works with keyboard -->
<button type="button" class="close" aria-label="Close" onclick="closeModal(
  ×
</button>
```

3. Understandable

```html
<!-- ❌ Placeholder as label (and wrong type) -->
<input type="text" placeholder="Email" />

<!-- ✅ Clear form -->
<label for="email">Email address</label>
<p id="email-hint">We'll use this email to send you the confirmation.</p>
<input
  type="email"
  id="email"
  name="email"
  autocomplete="email"
  required
  aria-describedby="email-hint"
/>
```

4. Robust

Use semantic, valid HTML. Native elements (`<button>`, `<a>`, `<form>`, `<label>`, `<nav>`, `<main>`, `<article>`) already come with keyboard support, focus behavior, and correct roles; they work better across browsers and assistive technologies. Use ARIA to complement, not to "turn" a `<div>` into an interactive component.

The cost of exclusion

Ignoring accessibility has real consequences:

- **Legal:** In many countries, web inaccessibility can have legal implications.
- **Economic:** 71% of users with disabilities abandon inaccessible sites (Click-Away Pound Survey).
- **Reputational:** Web accessibility lawsuits are public and damage brands.
- **User experience:** Inaccessible sites usually provide a worse experience for everyone.

Change starts with you

You don't need to be an accessibility expert to start:

1. **Navigate your app with keyboard only** (Tab, Enter, Escape). Can you do everything?
2. **Use a screen reader** (VoiceOver on Mac, NVDA on Windows). Does what you hear make sense?
3. **Check color contrast.** Tools like Lighthouse help.

The next time someone suggests "leaving accessibility for later," remember Sara's words: **accessibility is not an extra you add at the end. It's the difference between building a ramp and forcing someone to stay outside.**

Debugging is twice as hard as writing the code

Por Brian Kernighan - Debugging

Brian Kernighan, co-author of the legendary book *"The C Programming Language"* and co-creator of fundamental tools like AWK, wrote one of the most important warnings in our profession:

> ***"Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."***

This seemingly humorous observation hides a brutal mathematical truth about the limits of our cognitive capacity.

The arithmetic of cleverness

Let's assume your level of "programming intelligence" is X. When you write code, you use a portion of X. When you debug that code, you need **twice** the mental capacity because you must:

1. **Understand what the code does** (same as writing it).
2. **Understand what it should do** (the original requirement).
3. **Find the discrepancy** between both.

If you write code that uses 100% of your capacity (the most "clever" code you can create), you would need 200% of your capacity to debug it. But you only have 100%. **You're mathematically doomed.**

"Clever" code in practice

```
// ❌ LEVEL 1: The "clever" programmer
// This works, but can you find the bug?
const r = (a: number[]) => a.reduce((p, c, i) =>
  i % 2 ? p : [...p, a.slice(i, i + 2).reduce((x, y) => x + y)], [] as numb

// When it fails in production at 3 AM, good luck debugging it.
```

```
// ✅ LEVEL 2: The professional programmer
// The same algorithm, but debuggable
function sumPairs(numbers: number[]): number[] {
  const result: number[] = [];

  for (let i = 0; i < numbers.length; i += 2) {
    const first = numbers[i];
    const second = numbers[i + 1] ?? 0; // Handle odd-length arrays
    const pairSum = first + second;
    result.push(pairSum);
  }

  return result;
}

// When it fails, I can put a breakpoint on any line
// and understand exactly what's happening.
```

The three laws of debuggable code

1. Write code for your 3 AM self

Your cognitive capacity at 3 in the morning, with a production incident and the phone ringing, is approximately 30% of your normal capacity. Write code you can understand in that state.

2. Each line should have a single purpose

If a line does multiple things, finding which one fails is exponentially harder.

```
// ❌ One line, multiple operations
const result = users.filter(u => u.active).map(u => u.email).join(', ');

// ✅ Separate operations, easy to inspect
const activeUsers = users.filter(user => user.active);
const emails = activeUsers.map(user => user.email);
const result = emails.join(', ');

// If the result is wrong, is it the filter? The map? The join?
// With the second version, you can inspect each step.
```

3. Names are your first line of defense

When something fails, the first thing you look at are the names. Good names guide you toward the problem; bad names confuse you.

```
// ❌ What does 'd' mean? Why is it negative?
if (d < 0) throw new Error('Invalid');

// ✅ The name explains the context
if (daysUntilExpiration < 0) throw new Error('Subscription has expired');
```

The real cost of "elegant" code

Every time you choose the "clever" solution over the "obvious" solution, you're taking out a loan against your future debugging time. And that loan has compound interest:

- **Comprehension time:** Every developer who touches that code will lose minutes (or hours) deciphering it.
- **Latent bugs:** Complex code has more places for errors to hide.
- **Blocked refactoring:** Nobody wants to touch code they don't understand.

Humility as a strategy

Kernighan's quote isn't just technical; it's philosophical. It invites us to **humility**. No matter how good you are, there's a limit to what you can mentally handle. The best programmers aren't those who write the most sophisticated code, but those who write code that **anyone** (including their exhausted future self) can understand and fix.

The next time you're tempted to write an "elegant" one-liner solution, remember Kernighan's words. If you use all your intelligence to write the code, you'll have nothing left to fix it when it fails. **And it always fails.**

Your community is your superpower

Por Cassidy Williams - Community

Cassidy Williams has worked at Amazon, CodePen, Netlify, and GitHub. But if you ask her what's been key to her career, she won't mention any company. She'll talk about her **community**.

Her newsletter "rendezvous with cassidoo" reaches thousands of developers every week. Her programming memes are legendary. And her message is constant:

> *"The tech industry can be lonely. Building a community around you is the best career investment you can make."*

The myth of the lone genius

Hollywood has sold us the image of the solitary programmer in a dark basement, solving impossible problems at 3 AM with only coffee and determination.

Reality is very different. The best developers we know:

- Share what they learn
- Ask for help when they need it
- Celebrate others' successes
- Build in public

Nobody gets far alone.

How to build your community

1. Share what you learn (even as a beginner)

```
// You don't need to be an expert to add value
// The best time to document something is when you just learned it

// ❌ "I'll wait until I know enough"
// You'll never reach that point. Impostor syndrome doesn't disappear.

// ✅ "I just learned X. Here's how I'd explain it to myself a week ago"
// Your beginner perspective is valuable to other beginners.
```

2. Participate before expecting reciprocity

- Answer questions on Discord/Stack Overflow
- Comment on others' posts
- Share work from people you admire
- Attend meetups (in-person or virtual)

Community isn't transactional, but generosity generates generosity.

3. Be yourself (seriously)

Cassidy is known for her memes, her humor, and her personal style. She doesn't try to be "professional" in the corporate sense. And that's precisely what makes people connect with her.

```
// Your "personal brand" isn't a logo or a slogan
// It's how you treat people and what you share consistently

interface AuthenticPersonalBrand {
  interests: string[];        // What you're really passionate about
  voice: 'your-own-voice';    // Don't imitate others
  consistency: 'weekly' | 'when-you-have-something';
  perfectionism: false;       // Publishing > perfecting forever
}
```

The real benefits

Community isn't just "networking" in the superficial sense. It's:

1. **Your safety net:** When you lose a job or need help, your community responds.
2. **Your source of opportunities:** Most of the best jobs come through referrals, not cold applications.
3. **Your learning system:** You learn faster when you have someone to ask.
4. **Your mental health:** Knowing you're not alone in impostor syndrome, burnout, or technical frustration helps more than you imagine.

Start small

You don't need 100,000 followers. Cassidy started by answering questions in forums. Community is built one conversation at a time.

- Join a Discord for your favorite stack
- Comment on a DEV.to post
- Answer a question on Stack Overflow
- Attend a local meetup

- Tweet about what you learned today

The best investment in your career isn't another course. It's not another certification. It's the people around you and how you choose to engage with them.

Your community is your superpower. Building it is your responsibility.

Question everything you take for granted

Por Lynn Conway - Mindset

In 1964, a young engineer at IBM invented **dynamic instruction scheduling** — a technique that allows processors to execute instructions out of order for maximum performance. It was revolutionary. Then IBM fired her, erased her name from the records, and took credit for her work for decades.

Lynn Conway lost everything. She had to rebuild her career from scratch, unable to mention her most important contribution. Most people would have given up. She chose to question again.

At Xerox PARC in the 1970s, Conway asked a question nobody in the industry dared to ask: **"Why does designing a chip have to be so hard?"** At the time, creating an integrated circuit required years of artisanal work by highly specialized engineers. Conway, together with Carver Mead, radically simplified the process. Their book *Introduction to VLSI Systems* democratized chip design, and her **MOSIS** system let universities worldwide prototype chips over the internet. What once cost millions and took years could now be done by a student in weeks.

Her philosophy: **"Why not question everything?"**

The danger of "that's how it's always been done"

In programming, past decisions solidify into absolute truths. Nobody remembers why that database, that architecture pattern, or that naming convention was chosen. They only know "that's how it's always been done."

```
// ❌ Code nobody questions because "it's always worked"
const config = {
  maxRetries: 3,        // Why 3? Who decided this?
  timeout: 5000,        // Is 5 seconds optimal or just made up?
  batchSize: 100,       // Was 50 or 200 ever tested?
};

// ✅ Decisions backed by evidence and open to challenge
const config = {
  maxRetries: 3,        // Measured: 97% of transient errors resolve in 3 re
  timeout: 2000,        // Reduced from 5s after measuring real p95 latency:
  batchSize: 250,       // Benchmarked: 250 optimal for our payload size
};
```

Every "magic number" in your code is an assumption someone made once. Conway teaches us that questioning those assumptions is where innovation begins.

Failure as creative fuel

Conway believed deeply that **failure is a necessary part of creativity**. Not an obstacle to avoid, but an essential learning mechanism. Her own life is proof: she lost her career, her professional identity, and her work — and from those ashes built something that transformed an entire industry.

In software development, fear of failure produces conservative code, rigid architectures, and teams that never experiment. Conway proposed the opposite: **fail fast, fail cheap, fail often**.

```
// A search system evolving through failure

// Attempt 1: Exact match → fails (nobody types exact names)
// Attempt 2: includes() → fails (too many irrelevant results)
// Attempt 3: Relevance scoring ✅
function search(query, items) {
  const normalize = str => str.toLowerCase()
    .normalize('NFD').replace(/[\u0300-\u036f]/g, '');
  const q = normalize(query);

  return items
    .map(item => {
      const name = normalize(item.name);
      let score = 0;
      if (name === q) score = 100;
      else if (name.startsWith(q)) score = 75;
      else if (name.includes(q)) score = 50;
      return { ...item, score };
    })
    .filter(item => item.score > 0)
    .sort((a, b) => b.score - a.score);
}
// Each previous failure made this code better
```

None of the intermediate versions were "mistakes." Each was a necessary step. As Conway said: failure isn't the opposite of success — it's the path to it.

"If you want to change the future, start living as if you're already there"

This quote from Conway is more than motivational. It's an engineering strategy. When she designed MOSIS so any university could fabricate chips, she wasn't solving a present-day problem — she was building the infrastructure for the future she imagined.

Apply this to your code:

```
// ❌ Code for today: "we only have one language right now"
function getGreeting() {
  return "Hello, welcome";
}

// ✅ Code for the future: "let's live as if it's already international"
function getGreeting(locale: string = 'en') {
  const greetings: Record<string, string> = {
    en: "Hello, welcome",
    es: "Hola, bienvenido",
    pt: "Olá, bem-vindo",
  };
  return greetings[locale] ?? greetings.en;
}
```

This isn't over-engineering. It's **designing in the right direction** from the start, leaving room to grow without rewriting everything.

The courage to ask "why?"

Questioning everything takes courage. In a team, asking "why do we do it this way?" can be read as ignorance or attack. But Conway proved that uncomfortable questions generate the most important breakthroughs.

Next time you open a code file and see something that's "always been this way," stop for a moment. Ask why. There might be a good reason. That reason might have ceased to exist years ago. And your question might be, like Lynn Conway's, the beginning of something that changes the rules of the game.

The greatest enemy of innovation isn't lack of talent or resources. It's lack of questions. **Question everything. Fail with intention. Build the future you imagine.**

The web platform is your best framework

Por Lea Verou - Web Standards

Lea Verou, researcher at MIT and member of the W3C CSS Working Group, has spent years designing the future of CSS. Her perspective is refreshing in an industry obsessed with frameworks:

> *"The web platform is the most powerful framework we have. Learn it first."*

Before adding another npm dependency, ask yourself: can the browser do this natively?

The debt of frameworks

Every framework you add to your project is:

- **Weight:** KB your users download.
- **Complexity:** Documentation to maintain, versions to update.
- **Risk:** Will this framework exist in 5 years?

The web platform, on the other hand, is **forever**. A page created in 1995 with basic HTML still works today. Can you say the same about your React app from last year?

Modern CSS: More powerful than you think

```
/* ❌ Before: You needed JavaScript or SASS */
/* Now: Native CSS */

/* CSS Variables (Custom Properties) */
:root {
  --color-primary: #3b82f6;
  --spacing-md: 1rem;
}

.button {
  background: var(--color-primary);
  padding: var(--spacing-md);
}

/* Container Queries: Responsive components */
.card-container {
  container-type: inline-size;
}
```

```
@container (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}

/* Native Nesting */
.nav {
  background: white;

  & a {
    color: var(--color-primary);

    &:hover {
      text-decoration: underline;
    }
  }
}

/* Subgrid: Total layout control */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

.grid-item {
  display: grid;
  grid-template-rows: subgrid;
  grid-row: span 3;
}
```

Modern HTML: Interactivity without JavaScript

```
<!-- Native modals -->
<dialog id="my-dialog">
  <h2>I'm a native modal!</h2>
  <p>No JavaScript for open/close. No z-index hell.</p>
  <form method="dialog">
    <button>Close</button>
  </form>
</dialog>
<button onclick="document.getElementById('my-dialog').showModal()">
  Open modal
</button>

<!-- Native popovers -->
```

```html
<button popovertarget="menu">Open menu</button>
<div id="menu" popover>
  <ul>
    <li><a href="/profile">Profile</a></li>
    <li><a href="/settings">Settings</a></li>
  </ul>
</div>

<!-- Native accordion -->
<details>
  <summary>What is the web platform?</summary>
  <p>Standard HTML, CSS, and JavaScript, without frameworks.</p>
</details>

<!-- Native lazy loading -->
<img src="photo.jpg" loading="lazy" alt="Photo">
```

Lea's principle

Lea proposes a layered approach:

1. **HTML first:** Can I solve this with semantic HTML?
2. **CSS second:** Can I add style and basic behavior with CSS?
3. **JavaScript last:** Only when I really need complex interactivity.

```jsx
// ❌ Common overengineering
import { useState } from 'react';
import { motion } from 'framer-motion';
import styled from 'styled-components';

const Button = styled(motion.button)`...`;

function AccordionItem({ title, content }) {
  const [isOpen, setIsOpen] = useState(false);
  // 50 more lines...
}

// ✅ Native HTML + CSS
// <details><summary>Title</summary>Content</details>
// Zero JavaScript. Zero dependencies. Works forever.
```

The future Lea helps build

As a member of the CSS Working Group, Lea doesn't just preach; she builds. Features like `color-mix()`, the new `:has()` selectors, and Custom Properties improvements

exist partly thanks to her work.

Every time you learn a native feature of the web platform, you're:

- Reducing dependencies
- Improving performance
- Building for longevity
- Becoming a better developer

Frameworks come and go. The web platform remains.

Code never lies, comments sometimes do

Por Ron Jeffries - Documentation

Ron Jeffries, one of the three founders of Extreme Programming along with Kent Beck and Ward Cunningham, warns us with this uncomfortable truth: **"Code never lies, comments sometimes do."**

This phrase isn't an attack on documentation; it's a reminder that code is the **only source of truth** about what a program does. Comments are opinions, interpretations, or worse, outdated memories of what the code *used to* do.

The lifecycle of a lying comment

1. **Day 1:** A developer writes a function and adds an explanatory comment.
2. **Day 30:** Another developer modifies the function for a new requirement.
3. **Day 30 (minute 5):** The developer forgets to update the comment.
4. **Day 90:** A third developer reads the comment, trusts it, and writes code based on false information.
5. **Day 91:** Bug in production.

```
// ❌ The comment lies
// Calculates total with 21% VAT
function calculateTotal(price: number): number {
  return price * 1.10; // Someone changed VAT to 10% but not the comment
}

// ✅ The code speaks for itself
const VAT_RATE = 0.10;

function calculateTotalWithVat(price: number): number {
  return price * (1 + VAT_RATE);
}
```

When are comments useful?

Ron Jeffries doesn't say comments are always bad. There are cases where they're valuable:

1. Explain the "why", not the "what"

Code shows *what* it does. The comment can explain *why* that decision was made.

```
// ✅ The comment provides context the code cannot
function processPayment(amount: number): void {
  // Stripe requires amounts in cents, not dollars
  // See: https://stripe.com/docs/currencies#zero-decimal
  const amountInCents = Math.round(amount * 100);
  stripe.charge(amountInCents);
}
```

2. Warnings about non-obvious consequences

```
// ✅ Legitimate warning
// CAUTION: This function clears the global cache.
// Only call during the deploy process.
function clearGlobalCache(): void {
  globalCache.flush();
}
```

3. References to external documentation

```
// ✅ Useful reference
// Implementation of the Luhn algorithm for card validation
// RFC: https://en.wikipedia.org/wiki/Luhn_algorithm
function isValidCardNumber(cardNumber: string): boolean {
  // ...
}
```

Comments that should be code

Most comments exist because the code isn't expressive enough. Instead of commenting, **refactor**.

```
// ❌ BEFORE: Comment compensating for cryptic code
function calc(a: number, b: number, t: string): number {
  // If type is 'p', apply premium discount of 20%
  // Otherwise, apply standard discount of 10%
  if (t === 'p') {
    return a * b * 0.8;
  }
  return a * b * 0.9;
}
```

```
// ✅ AFTER: The code explains itself
const PREMIUM_DISCOUNT = 0.20;
const STANDARD_DISCOUNT = 0.10;

type CustomerType = 'premium' | 'standard';

function calculateDiscountedPrice(
  quantity: number,
  unitPrice: number,
  customerType: CustomerType
): number {
  const subtotal = quantity * unitPrice;
  const discountRate = customerType === 'premium'
    ? PREMIUM_DISCOUNT
    : STANDARD_DISCOUNT;

  return subtotal * (1 - discountRate);
}
```

The redundant comment test

Before writing a comment, ask yourself:

1. Can I improve the variable/function name so the comment is unnecessary?
2. Can I extract a function with a descriptive name?
3. Can I use a named constant instead of a magic value?

If the answer to any of these is "yes," refactor instead of comment.

Code as living documentation

The Extreme Programming philosophy, which Ron Jeffries pioneered, promotes the idea that well-written code is its own documentation. Tests are executable documentation. Names are inline documentation. Structure is architectural documentation.

```
// The code tells a clear story
const activeUsers = users.filter(user => user.isActive);
const premiumUsers = activeUsers.filter(user => user.subscription === 'prem
const eligibleForPromotion = premiumUsers.filter(user => user.purchaseCount

sendPromotionalEmail(eligibleForPromotion);
```

There are no comments, but anyone understands exactly what's happening.

The best comment is the one you don't need to write because your code already says it all.

With large models come large responsibilities

Por Timnit Gebru - AI

Timnit Gebru, founder of the Distributed AI Research Institute (DAIR) and one of the most influential researchers in AI ethics, was fired from Google in 2020 for a paper warning about the dangers of large language models (LLMs). That paper has become prophetic.

Her central message is clear:

> *"We need to ask: Who benefits from this AI? Who is harmed? Who decides?"*

The hidden costs of LLMs

When you use ChatGPT, Claude, or any LLM, there are costs you don't see:

1. Environmental cost

Training GPT-4 consumed energy equivalent to hundreds of homes for a year. Every query you make has a carbon footprint.

```
// What you see
const response = await openai.chat.completions.create({
  model: "gpt-4",
  messages: [{ role: "user", content: "Hello" }]
});

// What you don't see
// - Servers consuming megawatts
// - Data center cooling
// - CO2 emissions
// - Water usage for cooling
```

2. Invisible labor cost

Models are "aligned" by workers in developing countries who review traumatic content for minimum wages. The "clean" AI we use is built on invisible human labor.

3. Data cost

LLMs are trained on internet data, including:

- Your code on GitHub (without your explicit consent)
- Articles from journalists and writers (without compensation)
- Art from creators (without attribution)

The questions Timnit forces us to ask

Before deploying any AI system, ask yourself:

```
interface AIEthicsChecklist {
  // Who benefits?
  beneficiaries: {
    primary: string[];      // The company? Users? Society?
    secondary: string[];
  };

  // Who might be harmed?
  potentialHarms: {
    directHarms: string[];   // Discrimination, job loss
    indirectHarms: string[]; // Environmental impact, power concentration
    invisibleLabor: string[]; // Who keeps the system running?
  };

  // Who decides?
  governance: {
    whoDecides: string[];    // Only engineers? Affected parties?
    appealProcess: boolean;  // Can affected people appeal decisions?
    transparency: 'full' | 'partial' | 'none';
  };
}
```

The problem of scale

Timnit warns that AI errors multiply by millions:

```
// A bug in a traditional app
// Affects: users of that app
// Solution: deploy a fix

// A bias in a globally used LLM
// Affects: millions of people
// Perpetuates: stereotypes at massive scale
// Solution: ¿? (much more complex)
```

When GPT generates biased content about a group, it doesn't affect one person; **it affects the global perception of that group**.

What you can do as a developer

1. **Audit before integrating:** Don't assume LLMs are neutral. Test with edge cases and marginalized groups.
2. **Document limitations:** When using AI in production, be transparent about what it can and cannot do.
3. **Keep humans in the loop:** For important decisions, AI suggests, humans decide.
4. **Question the need:** Do you really need an LLM for this? Sometimes a database query is better (and more ethical).

```
// ❌ Using LLM by default
const response = await llm.generate("What are the store hours?");

// ✅ Using the appropriate tool
const hours = await database.getStoreHours(storeId);
// Faster, more accurate, cheaper, more ethical
```

Timnit's legacy

Timnit paid a high price for telling the truth. But her work has changed how the industry talks about AI. Thanks to her, questions that were once ignored are now central to the conversation.

With large models come large responsibilities. And those responsibilities are yours too, developer.

If you can't explain it simply, you don't understand it

Por Richard Feynman - Simplicity

Richard Feynman, Nobel Prize winner in Physics and one of the most brilliant scientists of the 20th century, had a principle he applied to all his work: **"If you can't explain it simply, you don't understand it well enough."**

Although Feynman wasn't a programmer, this maxim has become a pillar of software development. The ability to reduce complex problems to simple explanations isn't just a communication skill; it's a **sign of deep mastery**.

The Feynman Technique applied to code

Feynman developed a learning method that bears his name. Applied to software development, it works like this:

1. **Choose a concept:** For example, "How does JavaScript's event loop work?"
2. **Explain it as if talking to a 12-year-old:** No jargon, no assumed prior knowledge.
3. **Identify the gaps:** Where you stumble or use technical jargon, that's where you don't really understand.
4. **Go back to the source:** Study until you can explain it simply.

Code as explanation

If your code needs extensive comments to be understood, you probably don't understand it enough. Truly understood code is written in a way that **explains itself**.

```
// ❌ LEVEL 1: The author doesn't understand their own code
// This function processes user data applying the necessary transformations
// according to the system state and current business rules
// to determine if the user meets the eligibility criteria
// based on multiple interrelated factors
function proc(u: any, s: any, r: any[]): boolean {
  return r.some(x => x.t === 'e' && x.v > s.th && u.a > x.m);
}

// ✅ LEVEL 2: Deep understanding = simple code
interface User {
  age: number;
```

```
}

interface EligibilityRule {
  type: 'age' | 'membership' | 'purchase';
  minimumAge: number;
  threshold: number;
}

function isUserEligible(
  user: User,
  systemThreshold: number,
  rules: EligibilityRule[]
): boolean {
  const ageRules = rules.filter(rule => rule.type === 'age');

  return ageRules.some(rule =>
    rule.threshold > systemThreshold &&
    user.age > rule.minimumAge
  );
}

// No comment needed because the code IS the explanation
```

The "explain it to a junior" test

Feynman said that if you couldn't explain something to a first-year student, you didn't understand it. In programming, the equivalent is:

Can a junior developer on your team understand your code without your help?

If the answer is no, you have two options:

1. The code is too complex (refactor).
2. You don't understand what it does well enough (study before modifying).

Simple architecture = Understood architecture

Simplicity isn't just for individual functions. It applies to the entire architecture.

```
// ❌ Architecture nobody can explain in one sentence
// "Well, data comes in through the API Gateway, passes through the Event B
// gets processed in the Lambda Orchestrator, cached in Redis,
// persisted in DynamoDB except metadata which goes to S3,
// and then CDC replicates it to the Data Lake for the ML pipeline..."

// ✅ Architecture explainable in one sentence
// "Users make requests to the API, which reads/writes to the database."
```

```
// If you need more than that, ask yourself: is it essential or accidental
```

Symptoms of not understanding

Signs that you don't understand something well enough:

1. **You use many conditionals:** You probably don't understand the problem well enough to model it properly.
2. **You copy and paste code:** You don't understand the underlying pattern.
3. **Your explanation has lots of "basically" and "kind of":** You're camouflaging gaps.
4. **You need documentation open to make changes:** You haven't internalized the mental model.

The humility of the expert

Feynman was known for admitting what he didn't know. The best developers do the same. They say "I don't understand this" and then do the work to understand it before writing code.

```
// The Feynman approach in a code review:

// ❌ "This works but I'm not really sure why"
// 🚨 ALERT: Don't deploy this code to production

// ✅ "This works because X causes Y, which results in Z"
// ✅ Now you can maintain it, debug it, and improve it
```

The end result

When you truly understand something:

- Your code is shorter.
- Your functions have fewer parameters.
- Your names are more precise.
- Your architectures have fewer layers.
- Your explanations take less time.

As Feynman said, nature isn't complicated; we're the ones who complicate things when we don't understand them. **Simplicity isn't the starting point; it's the reward of deep understanding.**

Developer experience matters as much as user experience

Por Sarah Drasner - Developer Experience

Sarah Drasner, Director of Engineering at Google and formerly VP of Developer Experience at Netlify, has been a pioneer in treating developer experience (DX) as a serious discipline:

> *"Developer Experience is User Experience for developers. And developers are users too."*

Bad DX doesn't just frustrate your team; it kills productivity, increases turnover, and slows down all development.

What is DX?

DX encompasses everything a developer experiences when working with a tool, API, or codebase:

- Is it easy to get started? (onboarding)
- Are errors clear? (debugging)
- Is documentation helpful? (learning)
- Is feedback fast? (iteration)
- Is the process predictable? (reliability)

```
// ❌ BAD DX: Cryptic error
Error: ENOENT at Object.fs.openSync

// ✅ GOOD DX: Actionable error
Error: File not found: './config/database.yml'

  Possible solutions:
  1. Create the file: touch config/database.yml
  2. Copy from example: cp config/database.example.yml config/database.yml

  Documentation: https://docs.example.com/config
```

The pillars of good DX

1. Time to "Hello World"

How long does it take someone to see something working?

```
# ❌ BAD DX: 47 steps before starting
1. Install Docker
2. Configure database
3. Create environment variables
4. Generate SSL certificates
5. Compile assets
# ... 42 more steps

# ✅ GOOD DX: One command
git clone project && cd project && npm run dev
# "Your app is running at http://localhost:3000"
```

2. Fast feedback loops

```
// ❌ BAD DX: Wait 5 minutes to see a change
// Full build → Deploy → Manual test

// ✅ GOOD DX: Instant feedback
// Hot Module Replacement
// Tests in watch mode
// Automatic previews
```

3. Errors that help

```
// ❌ BAD DX: "Something went wrong"
throw new Error("Invalid input");

// ✅ GOOD DX: Context + Solution
class ValidationError extends Error {
  constructor(field: string, expected: string, received: unknown) {
    super(
      `Invalid value for "${field}"\n` +
      `  Expected: ${expected}\n` +
      `  Received: ${JSON.stringify(received)}\n` +
      `  Tip: Check the API documentation at /docs/api#${field}`
    );
    this.name = 'ValidationError';
  }
}
```

4. Documentation that respects your time

```
// ❌ BAD DX: Documentation that only explains "what"
/**
 * Sets the value.
 * @param value - The value to set.
 */

// ✅ GOOD DX: Documentation that explains "why" and "how"
/**
 * Sets the cache TTL in seconds.
 *
 * @param ttl - Time to live. Default: 3600 (1 hour)
 *
 * @example
 * // Cache for 5 minutes
 * cache.setTTL(300);
 *
 * @example
 * // Disable caching
 * cache.setTTL(0);
 *
 * @see https://docs.example.com/caching for performance implications
 */
```

Internal DX: Your own team is your user

Sarah insists that DX isn't just for public tools. Your internal team also deserves good DX:

```
// Signs of bad internal DX:
const redFlags = [
  "Only Juan knows how to deploy",
  "Don't touch that file, it's fragile",
  "Read the wiki (which hasn't been updated in 2 years)",
  "Ask María, she knows how to configure it",
];

// Signs of good internal DX:
const greenFlags = [
  "The README has everything you need to get started",
  "Tests run in less than 30 seconds",
  "PRs have automatic previews",
  "Errors tell you exactly what's wrong",
];
```

The ROI of DX

Investing in DX isn't a luxury; it's a productivity multiplier:

- **Less time in setup** = more time on features
- **Better errors** = faster debugging
- **Instant feedback** = more iterations per day
- **Good documentation** = fewer team interruptions

Sarah sums it up: "Every minute you save a developer multiplies by every developer, every day, forever."

Developer experience is not a detail. It's infrastructure.

Code as if the maintainer is a violent psychopath who knows where you live

Por John Woods - Maintainability

This phrase, attributed to **John Woods** and popularized on Usenet forums in the 90s, is perhaps the most visceral warning about the importance of maintainable code: **"Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live."**

It's dark humor, yes. But beneath the surface lies a deep truth: **the code you write today will be someone's problem tomorrow**. And that "someone" has every right to be furious if you leave them a disaster.

The maintainer is you (probably)

The irony is that, statistically, the "violent psychopath" who will maintain your code will be yourself in six months. It'll be you at 2 AM with a production incident. It'll be you trying to remember what the hell that function you wrote "temporarily" a year ago does.

```
// ❌ Code that will make you want to commit a crime
function x(a: any[], b: number, c?: boolean) {
  return c ? a.filter((_, i) => i % b === 0).map(x => x * 2)
           : a.reduce((p, c, i) => i % b ? p : [...p, c], []);
}

// In 6 months: "What is 'a'? What does 'b' do? Why does 'c' exist?"
// Your future self wants words with you.
```

```
// ✅ Code that preserves your physical integrity
type FilterStrategy = 'doubled' | 'raw';

function getEveryNthElement<T extends number>(
  items: T[],
  interval: number,
  strategy: FilterStrategy = 'raw'
): T[] {
  const filteredItems = items.filter((_, index) => index % interval === 0);

  if (strategy === 'doubled') {
    return filteredItems.map(item => (item * 2) as T);
```

```
  }

  return filteredItems;
}

// Your future self will thank you. And won't know your address.
```

Signs of "killer" code

Code that invites violence has recognizable characteristics:

1. Names that require a decoder

```
// ❌ Crimes against humanity
const d = new Date();
const x = u.p - d.getTime();
const r = x > 0 ? 'v' : 'e';

// ✅ Peace and harmony
const now = new Date();
const timeUntilExpiration = user.premiumExpiresAt - now.getTime();
const subscriptionStatus = timeUntilExpiration > 0 ? 'valid' : 'expired';
```

2. Functions that do 47 things

```
// ❌ An attack in function form
function processUserData(user: User) {
  // 200 lines that validate, transform, save to DB,
  // send emails, update cache, call 3 external APIs,
  // generate reports, and make coffee
}

// ✅ Functions that don't provoke rage
function validateUser(user: User): ValidationResult { /* ... */ }
function saveUser(user: User): Promise<void> { /* ... */ }
function notifyUser(user: User): Promise<void> { /* ... */ }
```

3. Hidden side effects

```
// ❌ Surprise... I've mutated everything
function getFormattedName(user: User): string {
  user.lastAccessed = new Date(); // WHAT?!
  globalCache.invalidate(user.id); // WHY HERE?!
  return `${user.firstName} ${user.lastName}`;
```

```
}

// ✅ No unpleasant surprises
function getFormattedName(user: User): string {
  return `${user.firstName} ${user.lastName}`;
}

function recordUserAccess(userId: string): void {
  // Clearly separated, clearly intentional
}
```

Empathy as professional practice

Writing maintainable code isn't just a technical skill; it's an act of **empathy**. It's thinking about the person who will come after (who might be you) and asking:

1. Will they understand what this does?
2. Will they be able to modify it without breaking something?
3. Will they know why I made this decision?
4. Will they want to hurt me after reading it?

If the answer to the last question is "yes," refactor.

Code as legacy

Every line you write is a message to the future. It can be a gift or it can be a curse. Professional developers consciously choose to leave a legacy that doesn't embarrass.

```
// Code tells a story
// Which one do you want yours to be?

// Story A: "The previous developer was a misunderstood genius
// whose code is so cryptic no one can touch it"

// Story B: "The previous developer was a professional
// whose code is so clear anyone can improve it"
```

The maintainer's golden rule

Before committing, read your code as if you were someone else. Better yet: imagine you're someone who just joined the team, it's Friday at 5 PM, there's a critical bug in production, and this code is all you have to fix it.

If in that situation you'd want to punch the author, **you're the one who needs to change the code now**.

Because remember: **the violent psychopath who knows where you live might be yourself in six months**. Treat yourself well.

AI-generated code needs human review

Por Andrej Karpathy - AI

Vibe coding is liberating. You write a description, AI generates code, it works on the first try. But **Andrej Karpathy**, the one who coined the term, constantly warns about its dark side:

> *"Vibe coding is amazing for prototypes, but for production you need to actually understand and review what the AI generates."*

The problem: Code that "works"

AI is very good at generating code that compiles, passes basic tests, and seems to work. The problem is that "works" doesn't mean "is secure" or "is correct".

```
// AI-generated code that "works"
async function getUserData(userId: string) {
  const query = `SELECT * FROM users WHERE id = '${userId}'`;
  return await db.query(query);
}

// ⚠️ Problem 1: SQL Injection
// An attacker can send: userId = "'; DROP TABLE users; --"

// ⚠️ Problem 2: SELECT * exposes sensitive data
// Do you really want to return the password hash?

// ⚠️ Problem 3: No input validation
// What happens if userId is undefined?
```

The studies are alarming

Recent research shows that a significant percentage of AI-generated code contains vulnerabilities:

- **SQL Injection:** AI concatenates strings in queries
- **XSS:** Generates unsanitized HTML
- **Hardcoded secrets:** Sometimes includes example API keys that look real
- **Insecure dependencies:** Suggests libraries with known vulnerabilities

The AI code review checklist

Before merging AI-generated code, ask yourself:

```
interface AICodeReviewChecklist {
  // Security
  "Are there unvalidated inputs?": boolean;
  "Are there queries built with concatenation?": boolean;
  "Are there hardcoded secrets or API keys?": boolean;
  "Is HTML output sanitized?": boolean;

  // Correctness
  "Do I understand what each line does?": boolean;
  "Does it handle edge cases?": boolean;
  "What happens with null/undefined?": boolean;
  "Are there tests for edge cases?": boolean;

  // Maintainability
  "Are names descriptive?": boolean;
  "Can I debug this at 3 AM?": boolean;
  "Does it follow project conventions?": boolean;
}
```

AI code patterns you should always review

1. Error handling

```
// ❌ AI often ignores errors
const data = await fetch(url).then(r => r.json());

// ✅ Explicit handling
const response = await fetch(url);
if (!response.ok) {
  throw new FetchError(`HTTP ${response.status}: ${response.statusText}`);
}
const data = await response.json();
```

2. Input validation

```
// ❌ AI trusts everything
function processOrder(order: Order) {
  return order.items.reduce((sum, item) => sum + item.price, 0);
}

// ✅ Validate before processing
```

```
function processOrder(order: unknown) {
  const validated = orderSchema.parse(order);
  return validated.items.reduce((sum, item) => sum + item.price, 0);
}
```

3. Data access

```
// ❌ AI doesn't think about authorization
async function getDocument(docId: string) {
  return await db.documents.findById(docId);
}

// ✅ Verify permissions
async function getDocument(docId: string, userId: string) {
  const doc = await db.documents.findById(docId);
  if (doc.ownerId !== userId && !doc.isPublic) {
    throw new ForbiddenError('No access to this document');
  }
  return doc;
}
```

The golden rule

If you don't understand the code, don't merge it.

It doesn't matter how much of a hurry you're in. It doesn't matter if it "works". Code you don't understand is code you can't:

- Debug when it fails
- Modify when requirements change
- Protect when there's a vulnerability

Vibe coding is a powerful tool. But like any powerful tool, it can cause harm if used carelessly. **AI generates the code; the responsibility is yours.**

The best request is the one not made

Por Addy Osmani - Performance

Addy Osmani, engineering leader at Google Chrome and one of the minds behind tools like Lighthouse, summarizes decades of web optimization in a simple truth: **the fastest way to load something is not to load it**.

The hidden cost of every request

Each HTTP request has hidden costs:

- **Network latency**: Even on fast connections, each round trip to the server adds up
- **Connection overhead**: TLS handshakes, DNS lookups, TCP connections
- **Parsing and processing**: The browser must interpret each resource
- **Memory**: Each asset consumes device RAM

On mobile networks or unstable connections, these costs multiply dramatically.

Practical strategies

1. Audit before adding

Before installing that library that's "only 2KB gzipped":

```
// Do you really need moment.js to format a date?
const date = new Date().toLocaleDateString('en-US', {
  year: 'numeric',
  month: 'long',
  day: 'numeric'
});
// "January 6, 2026"
```

2. Native lazy loading

```
<!-- Images outside the viewport don't load initially -->
<img src="photo.webp" loading="lazy" alt="...">

<!-- Iframes too -->
<iframe src="video.html" loading="lazy"></iframe>
```

3. Prioritize what's critical

```
<!-- Tell the browser what's important -->
<img src="hero.webp" fetchpriority="high" alt="...">
<link rel="preload" href="main-font.woff2" as="font" crossorigin>
```

4. Remove dead code

```
# Analyze what JavaScript is actually used
npx lighthouse --only-categories=performance
npx webpack-bundle-analyzer
```

Core Web Vitals: measure to improve

Osmani has been instrumental in defining the metrics that matter:

- **LCP** (Largest Contentful Paint): When does the main content appear?
- **INP** (Interaction to Next Paint): Does the page respond to interactions?
- **CLS** (Cumulative Layout Shift): Does content move unexpectedly?

Each additional request can negatively affect these metrics.

The 20% rule

Osmani suggests: *"Be at least 20% faster than your fastest competitor"*. This means:

1. **Measure first**: Don't optimize blindly
2. **Prioritize impact**: 80% of the benefit comes from 20% of the optimizations
3. **Automate**: Integrate performance budgets into your CI/CD

Final reflection

In a world where it's easy to add dependencies with an `npm install`, the discipline of asking *"do I really need this?"* has become a differentiating skill. Every kilobyte you don't send is time your users don't wait.

The code that doesn't exist works best

Por Carlos Blé - Craftsmanship

Carlos Blé, author of the first book in Spanish about TDD and founder of Lean Mind, has spent over two decades helping teams write better software. His philosophy is summarized in an apparent paradox: **the best code is the code you don't need to write**.

The productivity trap

We measure programmers by lines of code written, features delivered, commits per day. But these metrics ignore an uncomfortable truth: **every line of code is a liability, not an asset**.

Code that doesn't exist:

- Has no bugs
- Needs no tests
- Requires no documentation
- Needs no maintenance
- Can't become legacy

TDD: the art of writing just enough

Test-Driven Development isn't just a technique to avoid bugs. It's a discipline for **writing exactly the necessary code and not a line more**:

```
// 1. Write a failing test
test('calculates VAT correctly', () => {
  expect(calculateVAT(100)).toBe(21);
});

// 2. Write the minimum code to make it pass
function calculateVAT(base) {
  return base * 0.21;
}

// 3. Refactor if necessary (not if possible)
```

The Red-Green-Refactor cycle forces you to constantly ask: is this necessary now?

The "You Aren't Gonna Need It" rule (YAGNI)

Carlos insists: *"Don't build for an imaginary future. Build for the known present"*.

```
// ❌ Anticipated over-engineering
class UserRepositoryFactoryAbstractSingletonProxy {
  // 500 lines for "when we scale"
}

// ✅ What you actually need today
async function getUser(id) {
  return await db.users.findById(id);
}
```

Agile Design: emerges from tests

Good design isn't planned in UML diagrams for weeks. It **emerges organically** from writing tests first:

1. Tests force you to think about the API before the implementation
2. Tests reveal unnecessary dependencies
3. Tests document expected behavior

Legacy code starts on day one

> *"Legacy code isn't old code. It's code without tests"*

This definition by Michael Feathers, which Carlos has adopted and taught, changes the perspective: you can write legacy code on your first day if it's not testable.

Refactoring: the art of deleting

The best refactors don't add code, **they remove it**:

```
// Before: 47 lines
function processOrder(order, user, config, options, flags) {
  // Tangled logic...
}

// After: the same behavior in 12 lines
function processOrder(order) {
  // Simplicity emerges by eliminating the unnecessary
}
```

Final reflection

In an industry obsessed with building more, Carlos reminds us that the true craftsman knows when **not** to write code. Technical excellence isn't in the complexity you can create, but in the simplicity you can maintain.

Before learning a framework, learn the problem it solves

Por Dan Abramov - Learning

Dan Abramov, co-creator of Redux and former member of the React team at Meta, is known for his ability to explain complex ideas simply. One of his most valuable pieces of advice: **don't learn frameworks, learn the problems they solve**.

The tutorial trap

It's tempting to jump straight into the "How to build X with [Framework of the moment]" tutorial. But there's a problem: **you learn to copy patterns without understanding why they exist**.

When problems come (and they always do), you find yourself:

- Searching for solutions you don't understand
- Copying code from Stack Overflow without knowing what it does
- Frustrated because "it should work"

Dan's approach

In his blog overreacted.io, Dan shares a different approach:

1. Understand the problem first

Before learning React, ask yourself:

- Why is it hard to keep the DOM synchronized with state?
- What problems does direct DOM manipulation cause?
- Why do frameworks use a virtual DOM?

2. Build a simplified version

```
// Try to build a mini-React
function render(element, container) {
  // Your implementation here
  // The process will teach you more than any tutorial
}
```

3. Now learn the actual framework

With the problem context, the framework makes sense:

```
// Now you understand why this is powerful
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Mental models matter more than syntax

Dan constantly talks about "mental models" - the way we conceptualize how something works.

Correct mental model for useEffect:

> *"useEffect lets you synchronize your component with something external"*

It's not "the hook for doing things after render". With the right model, you write better code:

```
// With the correct mental model
useEffect(() => {
  const subscription = subscribe(roomId);
  return () => subscription.unsubscribe();
}, [roomId]);
```

Documentation is your friend

Dan spent years writing documentation because he understands something: **the best way to learn is through explanations that address the "why"**, not just the "how".

The new React documentation (react.dev) is an example: every concept explains the problem before the solution.

Final reflection

Frameworks come and go. Vanilla JavaScript has survived jQuery, Backbone, Angular 1, and will survive whatever comes next. When you understand the fundamental problems, learning new tools becomes trivial - because they all solve the same things in slightly different ways.

Content should guide design, not the other way around

Por Jen Simmons - CSS

Jen Simmons, now Apple Evangelist for Web Developer Experience on Safari and WebKit, coined the term **"Intrinsic Web Design"**. Her philosophy revolutionizes how we think about layouts: **let the content determine the design, not the arbitrary constraints of a grid**.

The era of hacks is over

For years, we built web layouts with tools that weren't designed for layout:

- **Tables**: Semantics destroyed
- **Floats**: Clearfix, overflow hidden, negative margin
- **CSS Frameworks**: 12 columns because reasons

CSS Grid and Flexbox change the rules of the game.

Intrinsic Web Design: the 6 principles

1. Combine flexible and fixed

```
.layout {
  /* Columns that adapt AND fixed columns */
  grid-template-columns: auto 1fr 200px;
}
```

2. Let content determine the size

```
.card-grid {
  /* As many columns as fit, minimum 250px */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
}
```

3. Use space wisely

```
.container {
  /* gap replaces margin hacks */
  display: grid;
  gap: 2rem;
}
```

4. Media queries aren't the only answer

```
/* Container queries: respond to the container, not the viewport */
@container (min-width: 400px) {
  .card {
    flex-direction: row;
  }
}
```

5. Images should be fluid AND have aspect-ratio

```
img {
  max-width: 100%;
  height: auto;
  aspect-ratio: 16/9;
  object-fit: cover;
}
```

6. Two-dimensional layouts with Grid

```
.magazine-layout {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: auto 1fr auto;
  /* Content flows naturally */
}
```

Subgrid: the power of inheritance

One of the features Jen has fought for years:

```
.card-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

.card {
```

```
  display: grid;
  /* The card inherits the parent's lines */
  grid-template-columns: subgrid;
  grid-template-rows: subgrid;
}
```

Now elements inside cards align perfectly with each other.

The web is not print

> *"We stopped trying to make the web look like a printed magazine. We started designing for the medium it actually is: fluid, flexible, and inherently responsive"*

Jen reminds us that the web is its own medium, with its own rules. We shouldn't fight against its fluid nature - we should embrace it.

Final reflection

The tools exist. Grid, Flexbox, Container Queries, Subgrid. The question is no longer "can we do it?" but "do we understand the medium enough to let content guide design?". Jen Simmons has shown us the way.

CSS is a programming language, treat it as such

Por Una Kravets - CSS

Una Kravets, Developer Advocate at Google Chrome, has spent years demonstrating that **CSS is not "just styles"**. It's a complete declarative programming language, and treating it with the respect it deserves transforms how we build interfaces.

The anti-CSS bias

There's a stigma in the industry: "CSS is easy", "CSS isn't real programming", "anyone can do CSS". This bias has consequences:

- Unmaintainable CSS code in large projects
- Over-dependence on libraries like Tailwind or styled-components
- Developers who avoid learning CSS deeply

Modern CSS is programming

Variables (Custom Properties)

```
:root {
  --color-primary: oklch(65% 0.24 255);
  --spacing-unit: 8px;
  --font-size-base: clamp(1rem, 0.5vw + 0.875rem, 1.25rem);
}

.button {
  background: var(--color-primary);
  padding: calc(var(--spacing-unit) * 2);
  font-size: var(--font-size-base);
}
```

Mathematical functions

```
.responsive-grid {
  --columns: 3;
  grid-template-columns: repeat(var(--columns), 1fr);
  gap: calc(100vw / 50);
```

```
}

.fluid-typography {
  /* Linear interpolation between two sizes */
  font-size: clamp(1rem, 1rem + 2vw, 3rem);
}
```

Conditional logic (without JavaScript)

```
/* Container Queries: if/else based on the container */
@container (min-width: 400px) {
  .card { flex-direction: row; }
}

/* Logical selectors */
.card:has(.featured-image) {
  grid-template-rows: 200px 1fr;
}

.form:has(:invalid) .submit-button {
  opacity: 0.5;
  pointer-events: none;
}
```

Declarative animations

```
@keyframes slide-in {
  from { transform: translateX(-100%); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}

.element {
  /* Scroll-based animation, no JavaScript */
  animation: slide-in linear;
  animation-timeline: view();
  animation-range: entry 0% entry 100%;
}
```

The power of the platform

Una insists on a message: **"Less clutter, more power"**. Many libraries solve problems that CSS already solved:

Instead of...	Use...
JavaScript scroll spy	`animation-timeline`

Instead of...	Use...
Tooltip library	CSS Anchor Positioning
Modal with JS	`<dialog>` + `:modal`
Overflow detection	`@container style()`

CSS Architecture

Like any language, CSS requires architecture:

```
/* Layers to control specificity */
@layer reset, base, components, utilities;

@layer base {
  body { font-family: system-ui; }
}

@layer components {
  .button { /* component styles */ }
}

@layer utilities {
  .hidden { display: none; }
}
```

Final reflection

Treating CSS as a second-class language is a costly mistake. The best frontend developers understand that CSS is as deep and requires as much expertise as JavaScript. Una Kravets shows us that mastering CSS is not only possible - it's the key to building faster, more accessible, and maintainable interfaces.

Technical debt is like financial debt: you have to pay it

Por Ward Cunningham - Architecture

Ward Cunningham coined the term **"technical debt"** in 1992, and it has been constantly misinterpreted ever since. The original metaphor was much more subtle than most people assume.

What Ward really meant

Technical debt **is not bad code**. Ward defined it as:

> *"Shipping code that reflects your current understanding of the problem, knowing that understanding will improve over time"*

It's like taking out a loan: you get value now (ship fast) in exchange for future interest (refactoring effort).

Good debt vs. bad debt

Intentional debt (good)

```
// "I know this design doesn't scale, but I need to validate
// the idea with real users before investing more"
function processPayment(amount) {
  // Simple implementation that works for 100 users
  // TODO: Refactor when we have traction
  return stripe.charge(amount);
}
```

Accidental debt (bad)

```
// "I don't know what I'm doing but it works, don't touch it"
function processPayment(a, b, c, flag1, flag2) {
  if (flag1 && !flag2 || (b > 0 && c !== undefined)) {
    // 500 lines of spaghetti code
  }
}
```

The interest on debt

Every day you don't pay the debt, interest accumulates:

1. **Reduced velocity**: Every new feature takes longer
2. **Mysterious bugs**: Fragile code breaks in unexpected places
3. **Slow onboarding**: New developers take months to understand the system
4. **Fear of change**: Nobody wants to touch "that part of the code"

When to take on debt

Technical debt is a **strategic tool**, not an accident:

```
✅ Take on debt when:
- You need to validate a business hypothesis quickly
- The cost of not shipping exceeds the cost of refactoring
- You have a clear plan to pay off the debt

❌ Don't take on debt when:
- It's the path of least resistance
- You don't understand the consequences
- You have no intention of paying it back
```

How to pay off debt

Ward suggests a continuous approach, not big-bang:

1. **Make debt visible**: Document each conscious decision
2. **Pay small interest constantly**: Refactor while you work
3. **Don't let it accumulate**: A little debt is manageable, a lot is paralyzing

Final reflection

Ward's metaphor is powerful precisely because it's familiar. We all understand that financial debt can be useful (mortgage for a house) or destructive (maxed-out credit cards). The same applies to code: conscious, planned debt is a tool; ignored debt is a problem.

All abstractions are leaky

Por Joel Spolsky - Architecture

Joel Spolsky, co-founder of Stack Overflow, formulated a law that every developer should know: **"All non-trivial abstractions, to some degree, are leaky"**.

What is a leaky abstraction?

An abstraction is a simplification that hides complexity. A "leak" occurs when that hidden complexity escapes and forces you to understand the details the abstraction was supposed to hide.

Classic example: SQL

SQL abstracts how data is physically stored. But when your query is slow:

```sql
-- This should "just work"
SELECT * FROM users WHERE email = 'user@example.com';

-- But you need to know about indexes, which are an implementation detail
CREATE INDEX idx_users_email ON users(email);
```

Suddenly, you need to understand B-trees, disk pages, and execution plans. The abstraction leaks.

Modern example: ORMs

```
// Looks simple and clean
const users = await User.findAll({
  include: [{ model: Post }, { model: Comment }]
});

// But it generates the N+1 problem, and now you need to understand SQL
// The abstraction that promised you could ignore SQL... forces you to know
```

Leaks are inevitable

Joel argues that **you can't design a perfect abstraction**:

- **TCP** abstracts the network, but when there's latency, you notice

- **File systems** abstract the disk, but when the disk is full, it fails
- **Garbage collectors** abstract memory, but when there's a GC pause, your app freezes
- **Async/await** abstracts callbacks, but deadlocks still exist

The "you don't need to know" trap

The danger of good abstractions:

```
// A junior can write this without understanding HTTP
const response = await fetch('/api/users');
const data = await response.json();

// But when it fails, they need to understand:
// - HTTP status codes
// - CORS
// - Headers
// - Timeouts
// - Retries
// - Cache
```

The abstraction lets you start without knowledge, but **demands that knowledge when it fails**.

How to survive leaks

1. Learn one layer below

You don't need to know assembly, but you should understand:

- JavaScript → How the event loop works
- React → How the DOM works
- Kubernetes → How containers work

2. Read error messages carefully

Cryptic errors often come from lower layers leaking through:

```
ECONNREFUSED 127.0.0.1:5432
```

Your ORM won't tell you "PostgreSQL isn't running". It just leaks the TCP error.

3. Have debugging tools for each layer

```
# Network
curl, wget, tcpdump

# Database
EXPLAIN ANALYZE

# Containers
docker logs, kubectl describe
```

Final reflection

Abstractions are essential - we couldn't build modern software without them. But the illusion that you can ignore what's underneath is exactly that: an illusion. The best developers understand multiple layers of the stack, not because they want to, but because leaks forced them to learn.

Convention over configuration: fewer decisions, more productivity

Por David Heinemeier Hansson - Productivity

David Heinemeier Hansson (DHH), creator of Ruby on Rails, popularized a principle that changed the way we think about frameworks: **"Convention over configuration"**.

The problem of infinite configuration

Before Rails, configuring a web project was a nightmare:

```
<!-- struts-config.xml in Java circa 2004 -->
<action-mappings>
  <action path="/users/show"
          type="com.app.actions.users.ShowAction"
          name="userForm"
          scope="request"
          validate="false">
    <forward name="success" path="/WEB-INF/views/users/show.jsp"/>
  </action>
</action-mappings>
```

Each endpoint required explicit configuration. Each table needed manual mapping. Every decision fell on the developer.

The Rails revolution

Rails proposed something radical: **if you follow the conventions, you don't need to configure anything**:

```
# This is ALL you need
class UsersController < ApplicationController
  def show
    @user = User.find(params[:id])
  end
end
```

Rails assumes:

- The table is called `users` (plural of the model)
- The view is at `app/views/users/show.html.erb`
- The route is `GET /users/:id`

Zero configuration. If you don't like the convention, you can change it. But 90% of the time, the convention is exactly what you need.

Applying the principle today

Next.js: file system based routing

```
pages/
├── index.js        → /
├── about.js        → /about
└── users/
    ├── index.js    → /users
    └── [id].js     → /users/:id
```

No route configuration. The folder structure **is** the configuration.

Vite: zero config to start

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

It works. No 500-line webpack.config.js.

Tailwind: classes with predictable names

```
<!-- The convention is obvious: {property}-{value} -->
<div class="mt-4 p-6 bg-blue-500 text-white rounded-lg">
```

You don't need to memorize anything because the pattern is consistent.

When to break convention

DHH doesn't say "never configure". He says "**configure only when you need to**":

```
# The convention doesn't apply here: my legacy table is called "tbl_usr_dat
class User < ApplicationRecord
```

```
  self.table_name = "tbl_usr_data"
end
```

Convention is the happy path. Configuration is the escape hatch.

The hidden cost of "maximum flexibility"

Some frameworks boast about being "flexible" and "unopinionated":

```
// "You can organize your code however you want"
// 6 months later: every file in a different place
// Every developer with their own "architecture"
```

Flexibility without guidance becomes chaos. Strong conventions create productive teams.

Final reflection

DHH understood something fundamental: **decisions have a cognitive cost**. Every configuration you have to write is time not spent solving real problems. The best frameworks reduce decisions without eliminating flexibility. They give you a default path that works, and let you deviate when you really need to.

Don't design for the worst case, design for the real case

Por Radia Perlman - Architecture

Radia Perlman, inventor of the Spanning Tree protocol that makes Ethernet networks work, has a unique perspective on system design: **solve the problem you have, not the one you imagine you might have**.

The "What if...?" syndrome

Engineers love hypothetical scenarios:

- "What if we have a million users?"
- "What if we need to support 50 languages?"
- "What if the client wants to change the database?"

The result: overcomplicated systems for problems that never arrive.

```
// What we build "just in case"
class AbstractFactoryProviderManagerSingletonProxy {
  // 2000 lines of "flexible" code
}

// What we actually needed
function getUser(id) {
  return db.users.findById(id);
}
```

Radia's approach

When designing Spanning Tree, Radia didn't try to solve all possible network problems. She focused on **one specific problem**: preventing loops in Ethernet networks.

The protocol:

- Is simple (fits on a napkin)
- Solves exactly what it needs to solve
- Has worked for 40 years

Pragmatic design in practice

Appropriate scale

```
// You don't need Redis when you have 100 users
const cache = new Map();

function getUser(id) {
  if (cache.has(id)) return cache.get(id);

  const user = await db.users.findById(id);
  cache.set(id, user);
  return user;
}

// When you really need Redis, migration is trivial
```

Appropriate database

```
-- PostgreSQL does all this well for 99% of cases:
-- - Relational data ✓
-- - JSON ✓
-- - Full text search ✓
-- - Geolocation ✓

-- You don't need 5 specialized databases
```

Appropriate architecture

```
# For most projects:
One server → One process → One database

# Premature "microservices":
12 services → 12 deploys → 12 failure points → 1 developer crying
```

The cost of unnecessary flexibility

Every "just in case" abstraction has costs:

1. **Complexity**: More code to maintain
2. **Performance**: More layers to traverse
3. **Understanding**: More concepts to learn
4. **Bugs**: More places where things can fail

When TO design for the future

Radia doesn't say "never plan". She says "**plan for probable changes, not possible ones**":

```
// Probable change: adding fields to user
// Design: flexible schema, automatic migrations
const userSchema = {
  name: String,
  email: String,
  // Easy to add more fields
};

// Possible but unlikely change: switch from PostgreSQL to Oracle
// Design: DON'T abstract the entire database
// If that moment comes, you refactor
```

Final reflection

Radia's protocol has survived decades not because it anticipated all changes, but because it solved one specific problem well. The best systems aren't the most flexible - they're the ones that do one thing well. You can always add complexity later; removing it is much harder.

Teach bravery, not perfection

Por Reshma Saujani - Mindset

Reshma Saujani, founder of Girls Who Code, has observed a pattern in thousands of students learning to code. Her conclusion changed how we think about tech education: **"We teach girls to be perfect, and boys to be brave"**.

The blank screen syndrome

In Girls Who Code courses, Reshma noticed something:

> *"A teacher told me that girls came to her desk saying 'I don't know what code to write'. But when she looked at their screens, they weren't blank. They had written code, but deleted it because it wasn't perfect the first time."*

Boys, on the other hand, showed their incomplete code without problem. They were used to trying, failing, and continuing.

Perfection paralyzes

```
// The code we delete for "not being perfect"
function calculateTotal(items) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    total += items[i].price;
  }
  return total;
}

// But this WORKS. It's a starting point.
// Later you can refactor:
const calculateTotal = items => items.reduce((sum, item) => sum + item.pric
```

The first code isn't elegant. But it exists. And code that exists can be improved. Code you delete because "it's not good enough" can't be improved.

Bravery in programming

Being brave in code means:

1. Showing work in progress

```
// TODO: This is incomplete but I want feedback
function processPayment(amount) {
  // Basic implementation
  console.log(`Processing ${amount}...`);
  // Missing: validation, errors, Stripe integration
}
```

2. Asking "obvious" questions

In code reviews, in meetings, in Slack:

- "What does this function do?"
- "Why do we use this pattern?"
- "I don't understand this part"

"Dumb" questions often reveal real problems.

3. Publishing imperfect code

Your first GitHub project doesn't have to be perfect. Your first technical article doesn't have to be exhaustive. **Perfection is the enemy of progress**.

The cost of waiting for perfection

```
Scenario A: You wait until your app is "ready"
- Months of development without feedback
- Discover late that nobody needs it
- Burnout from working in a vacuum

Scenario B: You launch something basic
- Real users from day one
- Feedback that guides development
- Motivation from seeing impact
```

Beyond gender

Although Reshma speaks specifically about girls, the advice applies to everyone:

- Impostor syndrome affects all developers
- The "10x developer" culture intimidates
- Perfect tutorial code makes us feel inadequate

How to practice bravery

1. **Share before you're ready**: Small, frequent pull requests
2. **Fail in public**: Talk about your mistakes on your blog or social media
3. **Celebrate attempts**: Not just successes
4. **Mentor**: Helping others normalizes not knowing everything

Final reflection

Programming isn't about writing perfect code the first time. It's about iterating, failing, learning, and improving. The best programmers I know aren't the ones who never make mistakes - they're the ones who aren't afraid to make them.

The profiler is your best friend, not your intuition

Por John Carmack - Performance

John Carmack, the legend who programmed Doom, Quake, and revolutionized 3D graphics, has a golden rule for optimization: **"Never trust your intuition. Measure first"**.

The "this must be slow" trap

Programmers have a sixth sense for detecting "slow" code. The problem is that sense constantly lies.

```
// "Obviously" this is the slow part
const result = items
  .filter(item => item.active)
  .map(item => expensiveTransform(item))
  .reduce((acc, item) => acc + item.value, 0);

// But the profiler reveals: 95% of the time is in
// an HTTP call we didn't even suspect
```

Carmack and the obsession with measuring

When Carmack was optimizing Doom in 1993, every frame counted. His approach:

1. **Measure current state** (don't guess)
2. **Identify the real bottleneck**
3. **Optimize only that**
4. **Measure again**

```
// Carmack didn't assume — he measured
// He discovered wall rendering was 60% of frame time
// He focused only on that, ignoring the "obvious"
```

Modern tools

For JavaScript/Web

```
// Chrome DevTools Performance
// 1. Open DevTools → Performance
// 2. Record an action
// 3. Analyze the flame chart

// Or programmatically:
console.time('operation');
await yourOperation();
console.timeEnd('operation');

// For more detail:
performance.mark('start');
await yourOperation();
performance.mark('end');
performance.measure('operation', 'start', 'end');
```

For Node.js

```
# Profiling with clinic.js
npx clinic doctor -- node server.js

# Or with native profiler
node --prof app.js
node --prof-process isolate-*.log > processed.txt
```

For React

```
// React DevTools Profiler
// Or programmatically:
import { Profiler } from 'react';

<Profiler id="List" onRender={(id, phase, actualDuration) => {
  console.log(`${id} rendered in ${actualDuration}ms`);
}}>
  <ExpensiveList items={items} />
</Profiler>
```

The real 90/10

The 90/10 rule says 90% of time is spent in 10% of the code. But **you don't know which 10% until you measure**.

```
// What we assumed was slow:
function processData(data) {
  return data.map(item => complexCalculation(item)); // ← "Here's the probl
}

// What the profiler revealed:
function processData(data) {
  const result = data.map(item => complexCalculation(item));
  logToRemoteServer(result); // ← The real bottleneck
  return result;
}
```

Blind vs. informed optimization

❌ Blind optimization

```
// "This should be faster"
const result = [];
for (let i = 0; i < items.length; i++) {
  result.push(items[i] * 2);
}

// vs map (which was equally fast or faster)
```

✅ Informed optimization

```
// The profiler showed: 500ms in JSON.parse of a 50MB file
// Real solution: streaming parsing
const stream = require('stream-json');
```

Final reflection

Carmack could have assumed where the performance problems in Doom were. He had the experience to do it. But he chose to measure. And that made Doom run on hardware nobody thought possible. Your intuition is valuable for generating hypotheses. The profiler is what validates them.

The best documentation is code that doesn't need it

Por Kelsey Hightower - Readability

Kelsey Hightower, principal engineer at Google Cloud and one of the most respected voices in DevOps, is famous for his minimalist demos and direct philosophy: **if your code needs a lot of documentation to be understood, the problem is probably the code**.

The problem with separate documentation

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
# See docs/deployment-guide.md to understand these values
# Last updated: 6 months ago (probably outdated)
metadata:
  name: my-app
  labels:
    # Don't change this label, breaks the service (see JIRA-1234)
    app: legacy-name-dont-change
```

Separate documentation has a fundamental problem: **it gets out of sync with the code**.

Kelsey's approach

In his legendary demos, Kelsey shows complex Kubernetes systems without slides. The code speaks for itself:

```yaml
# Self-explanatory
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-frontend
  labels:
    app: web-frontend
    team: platform
    environment: production
spec:
  replicas: 3  # Same number as availability zones
  selector:
```

```
    matchLabels:
      app: web-frontend
```

The names, the structure, the values - everything tells a clear story.

Self-documenting code

Meaningful variables

```
// ❌ Needs comment
const d = 7; // days until expiration

// ✅ Self-explanatory
const daysUntilExpiration = 7;
```

Functions that describe their purpose

```
// ❌ What does this do?
function proc(u, f) {
  if (f) return u.filter(x => x.a);
  return u;
}

// ✅ The name is the documentation
function getActiveUsers(users, includeInactiveUsers = false) {
  if (includeInactiveUsers) return users;
  return users.filter(user => user.isActive);
}
```

Declarative configuration

```
// ❌ Cryptic config
const config = {
  rt: 3,
  to: 5000,
  mx: 100
};

// ✅ Self-explanatory config
const config = {
  maxRetries: 3,
  timeoutMs: 5000,
  maxConcurrentRequests: 100
};
```

When TO document

Kelsey doesn't say "never document". Document:

1. The "why", not the "what"

```
// The "what" is obvious from code
// The "why" needs explanation
const TIMEOUT_MS = 30000; // Stripe webhook can take up to 25s in extreme c

// Workaround for bug in library X (issue #1234)
// TODO: Remove when we upgrade to v3.0
```

2. Architecture decisions

```
# ADR-001: Using PostgreSQL over MongoDB

## Context
We need persistence for user data...

## Decision
PostgreSQL because...

## Consequences
- Positive: ACID, mature tooling...
- Negative: Less flexible for changing schemas...
```

3. Public APIs

```
/**
 * Sends a welcome email to the newly registered user.
 *
 * @param userId - User ID (must exist in database)
 * @throws {UserNotFoundError} If user doesn't exist
 * @throws {EmailServiceError} If sending fails
 */
async function sendWelcomeEmail(userId: string): Promise<void>
```

Final reflection

Documentation is a patch. Sometimes necessary, but a patch. Clear code, descriptive names, and obvious structure are the real cure. Kelsey demonstrates in every demo that the most complex systems can be understandable if they're well designed.

Ask what you don't know, even if it seems obvious

Por Julia Evans - Learning

Julia Evans, creator of Wizard Zines and engineer at Stripe, has built a career doing something many avoid: **asking "dumb" questions in public and documenting what she learns**.

The fear of appearing incompetent

In tech there's enormous pressure to "already know". Admitting you don't understand something feels dangerous:

- "How can a senior not know what a B-tree is?"
- "You should know this after 5 years of experience"
- "It's basic, just Google it"

Julia does the opposite: she publishes her questions and her learning process. And millions of people discover they had the same doubts.

Learning in public

Julia's approach:

1. Admit what you don't know

```
# A real Julia post:
"I always get confused about how DNS works. Let me finally learn it properl

# 50 pages of zine later: one of the most popular resources on DNS
```

2. Ask concrete questions

```
❌ "How does Linux work?"
✅ "What exactly happens when I type 'ls' and press Enter?"
```

The second question leads to useful answers and deep learning.

3. Document while learning

```
# My current understanding of containers:
- They're like isolated processes? ✓
- They use Linux namespaces? ✓
- They share kernel with host? ✓ (this surprised me)
- They're VMs? ✗ (now I understand the difference)
```

Why it works

When you ask in public:

1. **Others confirm or correct**: "Actually, it's not exactly like that..."
2. **You discover you weren't alone**: "I had that same doubt!"
3. **You create resources for others**: Your question becomes documentation
4. **Experts get involved**: They love explaining things well

Examples of "dumb" questions that revealed insights

```
❓ "Why does git have a staging area? Couldn't it just commit directly?"
💡 Insight: Allows partial commits and review before committing

❓ "Why isn't setTimeout(fn, 0) immediate?"
💡 Insight: The event loop, task queue, how JavaScript handles async

❓ "Why does HTTPS need certificates?"
💡 Insight: Asymmetric cryptography, chain of trust, CAs
```

How to start

In your team

```
In code review:
"I don't understand this part. Can you explain why we use this pattern?"

In meetings:
"Sorry, can you explain that acronym? I don't know it."
```

In public

- Write a blog post about something you just learned
- Tweet your technical questions

- Ask questions in community Discord/Slack channels

The secret superpower

Julia has turned "not knowing" into a strength. Her zines are popular precisely because they address the things everyone wanted to ask but didn't dare. By normalizing questions, she created an invaluable resource for the community.

Final reflection

The next time you don't understand something, resist the temptation to pretend you do. Ask. Document. Share. You're probably helping more people than you imagine, starting with your future self.

Data matters, not opinions

Por Tracy Chou - Diversity

Tracy Chou, an engineer who worked at Pinterest and Quora, did something that changed the conversation about diversity in tech: **she asked for data, not opinions**.

The moment that changed everything

In 2013, Tracy published a simple post: "Where is the data on gender diversity in tech?" The industry talked a lot about diversity, but nobody showed numbers.

She created a public spreadsheet asking companies to share how many women engineers they had. The response was revealing:

- Many companies **didn't know** their own numbers
- Those who knew **didn't want to share** them
- The numbers that emerged were **worse than expected**

Why data matters

```
// Opinion without data
"I think we're pretty diverse"

// Real data
const stats = {
  totalEngineers: 500,
  women: 45,           // 9%
  underrepresentedMinorities: 23,  // 4.6%
  latinx: 8,           // 1.6%
  black: 5             // 1%
};

// Data forces honest conversations
```

Applying the principle beyond diversity

Tracy's approach applies to any problem:

1. "Our code is good quality"

```
# Opinion vs. data
eslint . --format json | jq '.length'  # 847 errors
npm run test:coverage                   # 23% coverage
```

2. "Users love our app"

```
const realMetrics = {
  nps: 23,              // "Love" = -100 to +100
  churnRate: '8%/month', // You lose 8% users each month
  avgSessionTime: '47s'  // Less than 1 minute
};
```

3. "Our team is productive"

```
| Metric | Value |
|--------|-------|
| Lead time (idea → production) | 6 weeks |
| Deploy frequency | 1/month |
| Mean time to recovery | 4 hours |
| Change failure rate | 30% |
```

How to collect data without being invasive

For diversity

```
// Voluntary and anonymous surveys
// Aggregates, never individual
// Trends over time, not snapshots

const survey = {
  voluntary: true,
  anonymous: true,
  reportThreshold: 5,  // Don't report groups < 5 people
  categories: ['Prefer not to say', ...options]
};
```

For technical performance

```
# Automate collection
# DORA metrics
name: Collect Metrics
on:
```

```yaml
  schedule:
    - cron: '0 0 * * 1'  # Every Monday
jobs:
  metrics:
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/collect-dora-metrics.sh
```

The change Tracy generated

After her initiative:

1. **Pinterest** published their diversity numbers (first companies to do so)
2. **Other companies followed**: Google, Facebook, etc.
3. **The conversation changed**: From "we think we're fine" to "these are the data, this is what we're going to improve"

Final reflection

Tracy demonstrated that opinions without data are smoke. When you have numbers, you have accountability. When you have accountability, you have real progress. Apply this principle to any area: if you can't measure it, you're probably not improving it.

If your system is good, deploy on Friday

Por Guillermo Rauch - DevOps

Guillermo Rauch, CEO of Vercel and creator of Next.js, challenges one of the most accepted "truths" in the industry: **"Don't deploy on Friday"**. His argument: if your system needs that rule, your system has problems.

Institutionalized fear

```
# Typical development calendar

| Day       | Deploy allowed? |
|-----------|-----------------|
| Monday    | Yes             |
| Tuesday   | Yes             |
| Wednesday | Yes             |
| Thursday  | ⚠️ Careful     |
| Friday    | ❌ NEVER        |
| Weekends  | 🔒 Forbidden    |

# Why? "Because if something fails, no one's there to fix it"
```

But this reveals a deeper problem: **your deploy process is dangerous**.

Vercel's philosophy

Rauch built Vercel with a principle: **deploys should be so safe that the day doesn't matter**.

```
# At Vercel, every push is a deploy
git push origin main

# And every PR has a preview URL
# You can test before merging
# Rollback is one click
```

If Friday is dangerous, every day is dangerous. You just accept the risk Monday through Thursday.

What you need to deploy any day

1. Atomic and instant deploys

```
// Bad: 20-minute deploys where something can fail
// Good: immutable deploys that activate in milliseconds

// Vercel/Netlify/etc. build the new version
// And do atomic switch when ready
```

2. Immediate rollback

```
# If something fails, going back should be trivial
vercel rollback

# Or automatic
if (errorRate > threshold) {
  autoRollback();
}
```

3. Feature flags

```
// Separate deploy from release
if (featureFlags.newCheckout) {
  return <NewCheckout />;
} else {
  return <OldCheckout />;
}

// You can deploy new code
// And activate it gradually (canary release)
```

4. Observability

```
// Know immediately if something's wrong
Sentry.init({ dsn: '...' });

// Real-time business metrics
analytics.track('checkout_completed', { value });

// Alerts that wake the right person
```

5. Automated testing

```yaml
# CI that prevents broken deploys
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npm test
      - run: npm run e2e
      # Only deploy if everything passes
```

The hidden benefit

When you can deploy any day:

- **Fast fixes**: A Friday bug gets fixed on Friday
- **Less stress**: No "deploy windows"
- **More iterations**: More deploys = more feedback = better product

The counterargument

"But if it fails on Friday, no one's there to fix it"

Rauch's response:

1. If your system requires frequent human intervention, it's fragile
2. Emergencies don't respect calendars - they can happen any day
3. Automation should handle common cases

Final reflection

"No Friday deploys" is a symptom, not a solution. It's admitting that your deploys are dangerous and your system is fragile. The real solution is building systems where the day is irrelevant. If Vercel, serving millions of sites, can do it, so can you.

Make it work, make it right, make it fast (in that order)

Por Kent Beck - Process

Kent Beck, creator of Extreme Programming and TDD pioneer, has a mantra that defines how to approach any programming problem: **"Make it work, make it right, make it fast"** - and the order matters.

Why the order is crucial

The natural temptation is to try to do all three at once. We want code that works, is elegant, and is fast from the first moment. But this leads to:

- **Analysis paralysis**: We think too much before writing
- **Premature optimization**: We optimize code we might not need
- **Speculative architecture**: We design for cases that never arrive

Phase 1: Make it work

```
// The ugliest code that solves the problem
function calculateTotal(items) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    if (items[i].type === 'product') {
      total = total + items[i].price;
      if (items[i].discount) {
        total = total - items[i].discount;
      }
    } else if (items[i].type === 'service') {
      total = total + items[i].hourlyRate * items[i].hours;
    }
  }
  if (total > 100) {
    total = total * 0.9; // 10% discount
  }
  return total;
}

// Ugly? Yes. Works? YES.
// We have a passing test. We can continue.
```

This phase is about **validating that you understand the problem**. Ugly code that works is worth more than elegant code that doesn't exist.

Phase 2: Make it right

```
// Now we refactor with confidence (we have tests)
function calculateTotal(items) {
  const subtotal = items.reduce((sum, item) => {
    return sum + calculateItemPrice(item);
  }, 0);

  return applyBulkDiscount(subtotal);
}

function calculateItemPrice(item) {
  switch (item.type) {
    case 'product':
      return item.price - (item.discount || 0);
    case 'service':
      return item.hourlyRate * item.hours;
    default:
      return 0;
  }
}

function applyBulkDiscount(total) {
  const BULK_THRESHOLD = 100;
  const BULK_DISCOUNT = 0.1;

  return total > BULK_THRESHOLD
    ? total * (1 - BULK_DISCOUNT)
    : total;
}
```

The code is more readable, more testable, more maintainable. And **we know it still works** because the tests pass.

Phase 3: Make it fast (only if necessary)

```
// The profiler shows this is called 1 million times
// NOW we optimize, not before

function calculateTotal(items) {
  let sum = 0;
  const len = items.length;
```

```
  // Avoid property access in the loop
  for (let i = 0; i < len; i++) {
    const item = items[i];
    sum += item.type === 'product'
      ? item.price - (item.discount || 0)
      : item.hourlyRate * item.hours;
  }

  return sum > 100 ? sum * 0.9 : sum;
}

// Less readable, but 3x faster
// And we only got here when IT WAS NECESSARY
```

Why most people skip to phase 3

Premature optimization is seductive because:

1. **It feels productive**: "I'm writing efficient code"
2. **It's technically interesting**: Performance tricks are fun
3. **It avoids the hard part**: Understanding the real problem

The reality

90% of code never reaches phase 3. It's fast enough as is. Kent Beck knows this, and that's why he insists on the order.

Final reflection

Next time you find yourself thinking about performance before having something working, remember: the fastest code is the code you don't need to write because the feature was discarded. First make it work.

Less JavaScript is better JavaScript

Por Rich Harris - Performance

Rich Harris, creator of Svelte and Rollup, has dedicated his career to a counterintuitive idea: **the best JavaScript framework is the one that ships less JavaScript**.

The problem with modern JavaScript

```
// Your "simple" React app
import React from 'react';           // 42KB
import ReactDOM from 'react-dom';    // 35KB
import { Router } from 'wouter';     // 8KB
import { QueryClient } from 'tanstack/react-query'; // 39KB
// ... and 200KB more of dependencies

// To show: "Hello, World"
```

The user downloads, parses, compiles, and executes **hundreds of KB** before seeing anything useful.

Svelte's philosophy

Rich created Svelte with a radical idea: **the framework disappears at compile time**.

```
<!-- App.svelte -->
<script>
  let count = 0;
</script>

<button on:click={() => count++}>
  Clicks: {count}
</button>

<!-- This compiles to optimized vanilla JS, not a library -->
```

The result: **less than 2KB** for an interactive app.

Why every kilobyte matters

1. Download time

```
| Bundle | 3G (1.6 Mbps) | 4G (10 Mbps) |
|--------|---------------|--------------|
| 50KB   | 0.25s         | 0.04s        |
| 200KB  | 1.0s          | 0.16s        |
| 500KB  | 2.5s          | 0.4s         |
```

2. Parse time

JavaScript must be parsed before executing. On slow devices:

```
// 1MB of JavaScript ≈ 1 second of parsing on an average mobile
// That's BEFORE executing anything
```

3. Memory

Every byte of JavaScript consumes memory. On devices with limited RAM, this means:

- Tabs that reload
- Apps that crash
- Frustrated users

Strategies for less JavaScript

Use the platform

```
<!-- ❌ JavaScript for a modal -->
<div id="modal" class="hidden">...</div>
<script>/* 50 lines of code */</script>

<!-- ✅ Native HTML -->
<dialog id="modal">...</dialog>
<script>
  modal.showModal(); // 1 line
</script>
```

Islands Architecture

```
---
// Astro: only hydrate what's interactive
---
<html>
```

```
  <Header />           <!-- Static HTML, 0KB JS -->
  <MainContent />      <!-- Static HTML, 0KB JS -->
  <ShoppingCart client:visible />  <!-- JS only here -->
  <Footer />           <!-- Static HTML, 0KB JS -->
</html>
```

Aggressive lazy loading

```
// Don't load what the user doesn't see
const HeavyComponent = lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback={<Loading />}>
      {showHeavy && <HeavyComponent />}
    </Suspense>
  );
}
```

The Rich Harris paradox

Rich created JavaScript tools so we use **less JavaScript**:

- **Rollup**: Bundler with aggressive tree-shaking
- **Svelte**: Framework that disappears
- **SvelteKit**: Meta-framework with SSR by default

Final reflection

Every line of JavaScript has a cost. Not on your MacBook Pro with fiber optic - on your real user's mid-range phone with 3G. Rich Harris reminds us that the best code is code that doesn't exist, and the second best is code that runs on the server.

Data structures dominate. If you choose them right, algorithms are obvious

Por Linus Torvalds - Fundamentals

Linus Torvalds, creator of Linux and Git, has a perspective that goes against the obsession with algorithms: **"Bad programmers worry about the code. Good programmers worry about data structures"**.

The problem with thinking algorithms first

```
// Problem: find duplicates in a list
// "Algorithm first" approach: nested loops
function findDuplicates(arr) {
  const duplicates = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] === arr[j] && !duplicates.includes(arr[i])) {
        duplicates.push(arr[i]);
      }
    }
  }
  return duplicates;
}
// O(n³) - a disaster
```

Linus's approach: structure first

```
// Same problem, but choosing the right structure
function findDuplicates(arr) {
  const seen = new Set();
  const duplicates = new Set();

  for (const item of arr) {
    if (seen.has(item)) {
      duplicates.add(item);
    }
    seen.add(item);
  }

  return [...duplicates];
```

```
}
// O(n) - the algorithm "writes itself"
```

The choice to use `Set` makes the algorithm obvious.

Real-world examples

User cache

```
// ❌ Array - O(n) search
const usersCache = [];
const findUser = id => usersCache.find(u => u.id === id);

// ✅ Map - O(1) search
const usersCache = new Map();
const findUser = id => usersCache.get(id);
```

History with undo/redo

```
// The right structure (stack) makes the code trivial
class History {
  constructor() {
    this.undoStack = [];
    this.redoStack = [];
  }

  push(state) {
    this.undoStack.push(state);
    this.redoStack = []; // Clear redo on new action
  }

  undo() {
    if (this.undoStack.length === 0) return null;
    const state = this.undoStack.pop();
    this.redoStack.push(state);
    return this.undoStack[this.undoStack.length - 1];
  }

  redo() {
    if (this.redoStack.length === 0) return null;
    const state = this.redoStack.pop();
    this.undoStack.push(state);
    return state;
  }
}
```

Frequency counter

```
// ❌ Complex imperative approach
function countFrequencies(items) {
  const result = {};
  for (const item of items) {
    if (result[item] === undefined) {
      result[item] = 0;
    }
    result[item] += 1;
  }
  return result;
}

// ✅ Map with default value
function countFrequencies(items) {
  const counts = new Map();
  for (const item of items) {
    counts.set(item, (counts.get(item) || 0) + 1);
  }
  return counts;
}
```

Git: an example of design by structures

Git is brilliant because Linus chose the structures well:

```
Blob     → File content (hash → content)
Tree     → Directory (list of blobs and trees)
Commit   → Snapshot (points to tree + metadata)
Branch   → Pointer to commit (just that!)
```

With these simple structures:

- Diff is comparing trees
- Merge is combining trees
- Checkout is changing a pointer

The "algorithms" are trivial because the structures are correct.

How to think in structures

1. **What operations do I need?**
    - Frequent search → Hash Map

    - Order matters → Array or Linked List
    - Last in, first out → Stack
    - First in, first out → Queue

2. **What's the common case?**

    - Do I read more than write? → Optimize reads
    - Sorted data? → Search tree
    - Uniqueness matters? → Set

Final reflection

Linus teaches us that programming isn't about being clever with algorithms. It's about modeling data in a way that makes the code that manipulates it obvious. Choose your structures well and the algorithms will write themselves.

A function should do one thing and do it well

Por Robert C. Martin - Clean Code

Robert C. Martin, known as "Uncle Bob", has spent decades teaching a seemingly simple principle: **functions should do one thing**. Sounds obvious, but the reality is we violate it constantly.

What does "one thing" mean?

It's not the number of lines. It's the level of abstraction.

```
// ❌ This function does "one thing" but it's really several
function processUserRegistration(userData) {
  // Validation
  if (!userData.email || !userData.password) {
    throw new Error('Missing fields');
  }
  if (userData.password.length < 8) {
    throw new Error('Password too short');
  }

  // Hashing
  const salt = crypto.randomBytes(16);
  const hash = crypto.pbkdf2Sync(userData.password, salt, 1000, 64, 'sha512

  // Persistence
  const user = {
    email: userData.email,
    passwordHash: hash.toString('hex'),
    salt: salt.toString('hex'),
    createdAt: new Date()
  };
  database.users.insert(user);

  // Notification
  emailService.send({
    to: userData.email,
    subject: 'Welcome!',
    body: 'Thanks for registering...'
  });
```

```
  return user;
}
```

The same code, refactored

```
// ✅ Each function does ONE thing
function registerUser(userData) {
  validateUserData(userData);
  const user = createUser(userData);
  saveUser(user);
  sendWelcomeEmail(user);
  return user;
}

function validateUserData(data) {
  if (!data.email || !data.password) {
    throw new ValidationError('Missing required fields');
  }
  if (data.password.length < 8) {
    throw new ValidationError('Password must be at least 8 characters');
  }
}

function createUser(data) {
  return {
    email: data.email,
    ...hashPassword(data.password),
    createdAt: new Date()
  };
}

function hashPassword(password) {
  const salt = crypto.randomBytes(16);
  const hash = crypto.pbkdf2Sync(password, salt, 1000, 64, 'sha512');
  return {
    passwordHash: hash.toString('hex'),
    salt: salt.toString('hex')
  };
}

function saveUser(user) {
  database.users.insert(user);
}

function sendWelcomeEmail(user) {
  emailService.send({
    to: user.email,
    subject: 'Welcome!',
```

```
    body: 'Thanks for registering...'
  });
}
```

Why does it matter?

1. Testability

```
// Now you can test each part independently
describe('validateUserData', () => {
  it('throws on missing email', () => {
    expect(() => validateUserData({ password: '12345678' }))
      .toThrow('Missing required fields');
  });
});

describe('hashPassword', () => {
  it('returns hash and salt', () => {
    const result = hashPassword('mypassword');
    expect(result).toHaveProperty('passwordHash');
    expect(result).toHaveProperty('salt');
  });
});
```

2. Reusability

```
// hashPassword can now be used in:
// - Registration
// - Password change
// - Password reset
```

3. Readability

```
// The main function is now a "table of contents"
function registerUser(userData) {
  validateUserData(userData);  // Step 1
  const user = createUser(userData);  // Step 2
  saveUser(user);  // Step 3
  sendWelcomeEmail(user);  // Step 4
  return user;
}
```

The "AND" rule

If you can describe what your function does using "AND", it does too much:

```
❌ "This function validates the data AND hashes the password AND saves the

✅ "This function validates user data"
✅ "This function hashes a password"
✅ "This function saves a user"
✅ "This function sends the welcome email"
```

The counterargument

"But I have 20 small functions instead of 1 large one"

Yes, and each one:

- Has a descriptive name
- Does exactly what it says
- Is easy to test
- Is easy to modify
- Is easy to reuse

Final reflection

Uncle Bob says "the first draft is always bad". It's normal to write large functions first. The discipline is in refactoring them afterwards. Every time you use "and" to describe what a function does, consider splitting it.

Write tests that fail, not tests that pass

Por Kent C. Dodds - Testing

Kent C. Dodds, creator of Testing Library and one of the most influential voices in JavaScript testing, has advice that seems contradictory: **"The value of a test is in its ability to fail"**.

The problem with tests that always pass

```
// ❌ This test ALWAYS passes, even if the code is broken
test('user can login', () => {
  expect(true).toBe(true);
});

// ❌ This test looks real but doesn't test anything useful
test('renders component', () => {
  render(<LoginForm />);
  expect(document.body).toBeTruthy();
});
```

Useless tests give false confidence. It's worse than having no tests.

Kent's approach

A good test should:

1. **Fail when the code is broken**
2. **Pass when the code works**
3. **Test behavior, not implementation**

```
// ✅ Test that tests real behavior
test('user can fill and submit login form', async () => {
  const handleSubmit = jest.fn();
  render(<LoginForm onSubmit={handleSubmit} />);

  // Interact like a user would
  await userEvent.type(screen.getByLabelText(/email/i), 'user@example.com')
  await userEvent.type(screen.getByLabelText(/password/i), 'password123');
  await userEvent.click(screen.getByRole('button', { name: /log in/i }));

  // Verify expected behavior
```

```
  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'user@example.com',
    password: 'password123'
  });
});
```

If the button stops working, the test fails. If the email isn't sent, the test fails. **The test fails when it should fail**.

The snapshot testing trap

```
// ❌ Snapshot without context
test('renders correctly', () => {
  const { container } = render(<UserProfile user={mockUser} />);
  expect(container).toMatchSnapshot();
});

// The problem: when something changes (even a space),
// the dev hits "u" to update without thinking
```

Snapshots don't tell you **what** should happen. Only that something changed.

Implementation vs. behavior testing

```
// ❌ Implementation testing (fragile)
test('calls fetchUsers with correct params', () => {
  const spy = jest.spyOn(api, 'fetchUsers');
  renderHook(() => useUsers());
  expect(spy).toHaveBeenCalledWith({ page: 1, limit: 10 });
});
// If you refactor the implementation, the test breaks

// ✅ Behavior testing (robust)
test('displays users list', async () => {
  server.use(
    rest.get('/api/users', (req, res, ctx) => {
      return res(ctx.json([{ id: 1, name: 'Alice' }]));
    })
  );

  render(<UsersList />);

  expect(await screen.findByText('Alice')).toBeInTheDocument();
});
// The test passes if users are displayed, regardless of how
```

The test pyramid that fails well

```
       /\
      /E2E\      ← Few, but fail if the app is broken
     /------\
    /Integra-\   ← Test that parts work together
   /tion      \
  /------------\
 /    Unit      \  ← Many, fast, test isolated logic
/----------------\
```

How to verify your test is useful

Before committing a test, **break it intentionally**:

```
test('calculates total correctly', () => {
  // 1. Write the test
  expect(calculateTotal([10, 20, 30])).toBe(60);

  // 2. Temporarily change the implementation
  // function calculateTotal(items) { return 0; }

  // 3. Verify the test FAILS

  // 4. Restore the implementation
});
```

If the test doesn't fail when you break the code, the test is useless.

Final reflection

Kent reminds us that tests aren't a checkbox to tick. They're guards that protect your code. A guard that lets everyone through is worthless. Write tests that fail when things go wrong - that's their job.

Animations should have a purpose, not be decoration

Por Sarah Drasner - UX

Sarah Drasner, engineering director at Google and author of the most influential book on web animations, has a clear rule: **an animation without purpose is visual noise**.

The trap of animating "because we can"

```
/* ❌ Animation just because */
.card {
  animation: bounce 2s infinite;
}

/* The user: "Why won't this stop moving?" */
```

Gratuitous animations:

- Distract from content
- Tire the eyes
- Consume battery
- Annoy users with motion sensitivity

When animations add value

1. Interaction feedback

```
/* The button responds to click */
.button:active {
  transform: scale(0.95);
  transition: transform 0.1s ease-out;
}
```

The user knows their action was registered.

2. Guide attention

```
// Something new appeared - guide the eye to it
function showNotification(message) {
  const notification = createNotification(message);
  notification.animate([
    { opacity: 0, transform: 'translateY(-20px)' },
    { opacity: 1, transform: 'translateY(0)' }
  ], { duration: 200, easing: 'ease-out' });
}
```

3. Show spatial relationships

```
/* A modal that "comes from" somewhere */
.modal {
  animation: slideFromButton 0.3s ease-out;
}

@keyframes slideFromButton {
  from {
    transform: scale(0);
    transform-origin: var(--button-position);
  }
  to {
    transform: scale(1);
  }
}
```

The user understands where it came from and how to go back.

4. Reduce perception of waiting

```
/* Skeleton loading: better than a frozen screen */
.skeleton {
  background: linear-gradient(90deg, #eee 25%, #ddd 50%, #eee 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
}
```

Animations that respect the user

```
/* Respect prefers-reduced-motion */
.animated-element {
  animation: fadeIn 0.3s ease-out;
}
```

```
@media (prefers-reduced-motion: reduce) {
  .animated-element {
    animation: none;
    opacity: 1;
  }
}
```

Some users have vertigo, migraines, or simply prefer less motion. Respect that.

Sarah's rule: 200-500ms

```
/* Too fast: user doesn't perceive it */
.too-fast { transition: 0.05s; }

/* Too slow: user waits */
.too-slow { transition: 2s; }

/* The sweet spot */
.just-right { transition: 0.2s; } /* Quick UI */
.emphasis { transition: 0.4s; }   /* Important movements */
```

Easing matters

```
/* ❌ Linear feels robotic */
.robotic { transition: transform 0.3s linear; }

/* ✅ ease-out feels natural for entries */
.natural-in { transition: transform 0.3s ease-out; }

/* ✅ ease-in feels natural for exits */
.natural-out { transition: transform 0.3s ease-in; }

/* ✅ custom cubic-bezier for specific movements */
.bouncy { transition: transform 0.4s cubic-bezier(0.34, 1.56, 0.64, 1); }
```

Before adding an animation, ask:

1. What **problem** does this animation solve?
2. Does the user **need** this information?
3. Does the animation **help** or **distract**?
4. Does it work without animation for users who disable it?

Final reflection

Sarah teaches us that the best animations are the ones you don't consciously notice, but that make the interface feel alive and natural. If a user notices your animation, it's probably too much.

Short code is not always better code

Por Sandi Metz - Readability

Sandi Metz, author of fundamental books on object-oriented design, challenges the obsession with "clever" code: **"Code is read many more times than it's written. Optimize for reading"**.

The one-liner trap

```
// ❌ "Look how short it is"
const result = data.filter(x => x.a > 5).reduce((a, b) => ({...a, [b.k]: (a

// Question: What does this do?
// Answer: Nobody knows without stopping 5 minutes to analyze it
```

The same code, readable

```
// ✅ More lines, infinitely clearer
function groupActiveItemsByCategory(items) {
  const activeItems = items.filter(item => item.value > 5);

  const groupedByCategory = {};

  for (const item of activeItems) {
    const category = item.category;

    if (!groupedByCategory[category]) {
      groupedByCategory[category] = [];
    }

    groupedByCategory[category].push(item);
  }

  return groupedByCategory;
}
```

The code is understood in seconds. Not minutes.

Sandi's rules

Sandi has specific rules that seem arbitrary but work:

1. Classes of maximum 100 lines

```
# If your class has 500 lines, it does too many things
# Split it into smaller collaborators
```

2. Methods of maximum 5 lines

```
// ❌ 50-line method
function processOrder(order) {
  // ... 50 lines of mixed logic
}

// ✅ Small methods that call each other
function processOrder(order) {
  validateOrder(order);
  calculateTotals(order);
  applyDiscounts(order);
  saveOrder(order);
  notifyCustomer(order);
}
```

3. Maximum 4 parameters

```
// ❌ Too many parameters
function createUser(name, email, age, address, phone, role, permissions, ac
  // ...
}

// ✅ Options object
function createUser({ name, email, age, address, phone, role, permissions,
  // ...
}
```

"Clever" vs. "Clear"

```
// "Clever" - makes you feel smart writing it
const isEven = n => !(n & 1);

// "Clear" - makes anyone understand it
function isEven(number) {
```

```
  return number % 2 === 0;
}
```

Clever code impresses on Twitter. Clear code keeps teams productive.

The new teammate test

Before committing, ask yourself:

> *"Would someone who just joined the team understand this?"*

If the answer is no, it's probably too clever.

When TO be concise

Brevity has its place:

```
// Established patterns everyone knows
const doubled = numbers.map(n => n * 2);

// Simple operations
const sum = (a, b) => a + b;

// Destructuring that clarifies
const { name, email } = user;
```

The difference: these are **idiomatic**, not **clever**.

Final reflection

Sandi reminds us that we write code for humans, not compilers. Compilers understand anything. Humans need clarity. The time you save writing short code, you pay multiplied when someone (including your future self) has to understand it.

Prefer composition over inheritance

Por Dan Abramov - Architecture

Dan Abramov, co-creator of Redux, has repeatedly explained a principle that React adopted as central philosophy: **"Composition gives you everything inheritance promises, without the problems"**.

The problem with inheritance

```
// The typical class hierarchy that seemed like a good idea
class Animal {
  eat() { /* ... */ }
  sleep() { /* ... */ }
}

class Bird extends Animal {
  fly() { /* ... */ }
}

class Penguin extends Bird {
  // Oops... penguins don't fly
  fly() { throw new Error("I can't fly!"); }
}

// Now you have a method you shouldn't have
// And you break the Liskov Substitution Principle
```

The solution: composition

```
// Instead of "is a", think "has a" or "can do"
const canEat = (state) => ({
  eat: () => { /* ... */ }
});

const canSleep = (state) => ({
  sleep: () => { /* ... */ }
});

const canFly = (state) => ({
  fly: () => { /* ... */ }
});
```

```
const canSwim = (state) => ({
  swim: () => { /* ... */ }
});

// Now you compose what you need
function createPenguin(name) {
  const state = { name };
  return {
    ...canEat(state),
    ...canSleep(state),
    ...canSwim(state)
    // No fly - penguins swim, not fly
  };
}

function createEagle(name) {
  const state = { name };
  return {
    ...canEat(state),
    ...canSleep(state),
    ...canFly(state)
  };
}
```

In React: composition is king

```
// ❌ "Inheritance" in components (antipattern)
class SpecialButton extends Button {
  render() {
    return <button className="special">{this.props.children}</button>;
  }
}

// ✅ Composition
function SpecialButton({ children, ...props }) {
  return <Button className="special" {...props}>{children}</Button>;
}

// Even better: render props or children
function Card({ header, children, footer }) {
  return (
    <div className="card">
      <div className="card-header">{header}</div>
      <div className="card-body">{children}</div>
      <div className="card-footer">{footer}</div>
    </div>
```

```
  );
}
```

Hooks: behavior composition

```
// Each hook is a composable unit of behavior
function useAuth() {
  const [user, setUser] = useState(null);
  // ... auth logic
  return { user, login, logout };
}

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });
  // ... storage logic
  return [value, setValue];
}

// You compose behaviors in your component
function Dashboard() {
  const { user } = useAuth();
  const [preferences] = useLocalStorage('prefs', {});
  const [data] = useFetch(`/api/data/${user.id}`);

  // Your component is a composition of behaviors
}
```

Why it works better

1. Flexibility

```
// With inheritance: you're trapped in the hierarchy
// With composition: you mix what you need

const hybridCreature = {
  ...canFly(state),
  ...canSwim(state),
  ...canBreatheUnderwater(state)
};
```

2. Testing

```
// Each behavior is tested in isolation
test('canSwim', () => {
  const swimmer = canSwim({ energy: 100 });
  swimmer.swim();
  expect(state.energy).toBe(90);
});
```

3. No coupling

Inheritance creates rigid dependencies. Composition creates independent pieces.

Final reflection

Dan and the React team chose composition as the central pattern because it's more flexible, testable, and easier to reason about. Inheritance seems natural at first ("a dog IS an animal"), but quickly becomes restrictive. Composition gives you freedom to combine behaviors like Lego pieces.

Native CSS is more powerful than you think

Por Lea Verou - CSS

Lea Verou, W3C researcher and creator of essential CSS tools like Prism.js, has spent years demonstrating that **native CSS can do more than most people imagine**.

What JavaScript used to do (and CSS now solves)

1. Scroll-driven animations

```
/* Before: IntersectionObserver + JavaScript */
/* Now: pure CSS */
.reveal {
  animation: fadeIn linear;
  animation-timeline: view();
  animation-range: entry 0% cover 40%;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(50px); }
  to { opacity: 1; transform: translateY(0); }
}
```

2. Tooltips with positioning

```
/* Before: positioning library (Popper.js, etc.) */
/* Now: CSS Anchor Positioning */
.tooltip {
  position: absolute;
  anchor-name: --my-anchor;
  position-anchor: --my-anchor;
  top: anchor(bottom);
  left: anchor(center);
  translate: -50% 8px;
}
```

3. Modals

```
<!-- Before: JS for open/close/backdrop/focus trap -->
<!-- Now: HTML + CSS -->
```

```html
<dialog id="modal">
  <form method="dialog">
    <p>Are you sure?</p>
    <button value="cancel">Cancel</button>
    <button value="confirm">Confirm</button>
  </form>
</dialog>

<style>
dialog::backdrop {
  background: rgba(0, 0, 0, 0.5);
  backdrop-filter: blur(4px);
}
</style>
```

4. Dynamic themes

```css
/* Before: CSS-in-JS, dynamic classes */
/* Now: Custom Properties */
:root {
  --bg: #ffffff;
  --text: #1a1a1a;
  --accent: oklch(65% 0.2 250);
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #1a1a1a;
    --text: #ffffff;
    --accent: oklch(75% 0.2 250);
  }
}

body {
  background: var(--bg);
  color: var(--text);
}
```

Features you should use today

Container Queries

```css
/* Responsive to the container, not the viewport */
.card-container {
  container-type: inline-size;
}
```

```
@container (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 200px 1fr;
  }
}
```

:has() - The parent selector

```
/* "If the form has an invalid input, change the border" */
form:has(:invalid) {
  border-color: red;
}

/* "If the card has an image, change the layout" */
.card:has(> img) {
  grid-template-rows: 200px 1fr;
}
```

Native nesting

```
/* You no longer need Sass just for this */
.card {
  padding: 1rem;

  & .title {
    font-size: 1.5rem;
  }

  &:hover {
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  }

  @media (width > 768px) {
    padding: 2rem;
  }
}
```

New color spaces

```
/* oklch: perceptually uniform, ideal for palettes */
:root {
  --primary-100: oklch(95% 0.05 250);
  --primary-200: oklch(85% 0.10 250);
  --primary-300: oklch(75% 0.15 250);
```

```
  --primary-400: oklch(65% 0.20 250);
  --primary-500: oklch(55% 0.20 250);
}
```

Lea's principle

> *"Before adding JavaScript for a visual effect, ask: can CSS do it?"*

The answer is "yes" much more often than you think.

Final reflection

Lea has dedicated her career to expanding what CSS can do. Each year, CSS gains capabilities that previously required JavaScript. The browser optimizes CSS better than any library. Knowing modern CSS makes you a more efficient developer and your apps faster.

Refactor mercilessly, but with tests

Por Martin Fowler - Clean Code

Martin Fowler, author of the book "Refactoring" that defined the term, has a rule that seems contradictory: **refactor constantly and mercilessly, but never without tests**.

What refactoring is (and isn't)

```
// ❌ This is NOT refactoring
// You're changing behavior
function calculateTotal(items) {
  return items.reduce((sum, i) => sum + i.price, 0);
}
// Becomes:
function calculateTotal(items) {
  // Now applies discounts (new behavior)
  return items.reduce((sum, i) => sum + i.price * 0.9, 0);
}

// ✅ This IS refactoring
// Same behavior, better structure
function calculateTotal(items) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    total = total + items[i].price;
  }
  return total;
}
// Becomes:
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}
```

Observable behavior is identical. Only internal structure changes.

Why tests first

Without tests, you don't know if you broke something. With tests, you refactor with confidence. If tests pass, behavior is correct.

The refactoring cycle

```
1. Run tests (must pass)
2. Make ONE small change
3. Run tests (must pass)
4. Repeat
```

Never make multiple refactorings without verifying tests pass between each one.

When to refactor

Fowler suggests the "rule of three":

1. **First time**: Just do it
2. **Second time**: Notice the duplication, but continue
3. **Third time**: Refactor

Final reflection

Fowler transformed refactoring from "something you do when you have time" to "integral part of development". With tests, refactoring stops being risky and becomes code hygiene. Without tests, it's blind surgery.

Learn in public: your best professional investment

Por Cassidy Williams - Career

Cassidy Williams started sharing programming memes on Twitter. Today she's one of the most influential voices in the tech community, has been Director of Developer Experience at several companies, speaks at conferences worldwide, and has a newsletter read by hundreds of thousands of developers. As she says: **"You don't need to be an expert to teach. You just need to know something that someone else doesn't know yet"**.

Traditional method vs. Learning in Public

```
❌ Traditional method:
1. Learn in silence for years
2. Accumulate knowledge
3. Eventually, maybe, share something
4. Constant fear of impostor syndrome

✅ Learning in Public:
1. Learn something new
2. Share it immediately (blog, tweet, video)
3. Receive feedback and corrections
4. Improve your understanding
5. Repeat
```

Why it works

1. The Feynman effect

```
"If you can't explain it simply, you don't understand it well enough"

When writing about something, you discover gaps in your knowledge.
The act of explaining consolidates your learning.
```

2. Your personal documentation

```
Problem 6 months ago: "How do I configure ESLint with TypeScript?"
- Google search: 2 hours
- Trial and error: 3 hours
- Frustration: incalculable

Problem today (if you wrote a post):
- Search your own blog: 30 seconds
- Copy the solution YOU documented
```

3. Involuntary networking

When you share, you attract people with similar interests:

- Recruiters looking for people who communicate well
- Other learners who want to collaborate
- Experts offering corrections (for free!)

Formats to get started

Blog posts

```
# What I learned configuring Docker today

I came from knowing nothing about containers.
These are the mistakes I made and how I solved them.

## Error 1: Port already in use
[problem description and solution]

## Error 2: Container dies immediately
[problem description and solution]
```

Tweets/threads

```
🧵 Today I learned 5 things about CSS Grid I didn't know:

1/ grid-template-areas lets you "draw" your layout with ASCII art

2/ minmax() is magic for responsive without media queries

[continues...]
```

Common fears

"I'm not an expert to talk about this"

That's precisely why you should do it. Experts forget what was hard for beginners. Your novice perspective is valuable.

"There are already 1000 articles about this"

But none written from YOUR perspective, with YOUR mistakes, in YOUR context.

"What if I say something wrong?"

The community will correct you (gently, mostly). You learn faster with public feedback than in silence.

Final reflection

Cassidy transformed her career by applying this principle. Not because she was smarter, but because her learning was visible. Her memes and videos started as fun, but the visibility they generated opened doors she never imagined. In a world where everyone consumes content, being a creator automatically differentiates you.

Duplicate code before creating the wrong abstraction

Por Sandi Metz - Architecture

Sandi Metz has advice that seems to go against everything we're taught: **"Duplication is far cheaper than the wrong abstraction"**.

The premature DRY impulse

```
// You see two similar functions
function formatUserEmail(user) {
  return `${user.firstName} ${user.lastName} <${user.email}>`;
}

function formatAdminEmail(admin) {
  return `[ADMIN] ${admin.firstName} ${admin.lastName} <${admin.email}>`;
}

// Your instinct: "There's duplication! I must abstract"
function formatEmail(person, prefix = '') {
  return `${prefix}${person.firstName} ${person.lastName} <${person.email}>
}
```

Looks better, right? Until...

When the abstraction breaks

```
// New requirements: admins need to show their department
// Users need to show their plan (free/pro)
// Some users are companies, not people

function formatEmail(entity, options = {}) {
  const prefix = options.prefix || '';
  const suffix = options.suffix || '';
  const name = entity.companyName ||
    `${entity.firstName || ''} ${entity.lastName || ''}`.trim();
  const extra = entity.department || entity.plan || '';

  return `${prefix}${name}${extra ? ` (${extra})` : ''} <${entity.email}>${
}
```

```
// 😱 Now we have a function that does too much
// Hard to understand, test, and modify
```

Sandi's rule of three

```
1. First time: just write the code
2. Second time: notice the duplication, but allow it to exist
3. Third time: NOW consider abstracting

Why wait? Because by the third case:
- You know the real patterns better
- You see what's truly common and what's not
- The abstraction emerges naturally
```

Duplicated code vs. forced abstraction

```
// ✅ Honest duplication (easy to change)
function formatUserForEmail(user) {
  return `${user.name} <${user.email}>`;
}

function formatUserForDisplay(user) {
  return `${user.name} (${user.role})`;
}

function formatUserForLog(user) {
  return `[User:${user.id}] ${user.name}`;
}

// Each function is simple and clear
// When one changes, the others aren't affected
```

Signs of premature abstraction

1. **The function has many boolean parameters**
2. **There are many `if/else` based on "type"**
3. **The name is generic:** `handleThing`, `processData`
4. **You need to read the implementation to understand what it does**

Signs it's time to abstract

1. **The pattern has repeated 3+ times**
2. **The cases are genuinely identical** (not just similar)
3. **The abstraction has a clear name**
4. **Tests would be simpler with the abstraction**

Final reflection

Sandi reminds us that the goal isn't eliminating all duplication - it's writing maintainable code. Sometimes, three simple duplicated functions are more maintainable than one "clever" abstract function.

Learn by doing, not by reading

Por Limor Fried - Learning

Limor Fried founded Adafruit Industries with a radical conviction: **the best way to learn technology is by building things with your own hands**. An MIT graduate and the first female engineer on the cover of WIRED, her philosophy is direct: "It's 100 percent teaching. I almost never engineer for the sake of engineering. There's always a project or a goal."

The infinite tutorial trap

```
Monday: "I'm going to learn React"
- Read the complete documentation
- Watch 3 videos of "React in 2025"
- Compare Next.js vs Remix vs Astro

Friday:
- Still hasn't written a line of code
- Feels "not ready yet"
- Finds another "better" tutorial
```

This is called **Tutorial Hell** and it's procrastination disguised as productivity. Limor compares it to learning to solder by reading a book: you can memorize every technique, but until you touch the iron, you haven't learned anything.

The Adafruit maker philosophy

Adafruit started in Limor's MIT dorm room, selling electronics kits anyone could assemble. The idea was simple: give someone a concrete project and the pieces to build it, and they'll learn faster than with any textbook.

The first 15 minutes

Limor says: "Just like DIY electronics, the first 15 minutes are crucial, special and fragile." Those first minutes with a new technology define whether you'll continue or quit.

```
❌ First 15 minutes reading:
- Open official documentation
- Read "Fundamental Concepts"
- Start feeling overwhelmed
```

```
- Close the tab

✅ First 15 minutes building:
- Clone a starter template
- Change some text and see the result
- Add a button that does something
- Feel that you CAN do this
```

The difference isn't technical. It's emotional. One path creates paralysis. The other creates momentum.

Why it works

1. Context-driven learning

```
// You remember this because you BUILT it
async function getComponents(category) {
  const response = await fetch(`https://api.adafruit.com/v2/${category}`);

  if (!response.ok) {
    throw new Error(`Category not found: ${category}`);
  }

  const data = await response.json();
  return data.products;
}
```

You didn't read about error handling. You needed it because your fetch failed.

2. Errors are the best teacher

At Adafruit, every kit comes with a troubleshooting guide. Not because they expect failure, but because **errors are part of the learning process**.

```
// Error: Cannot read property 'map' of undefined

// After hitting this in a REAL project,
// you'll never forget to protect your data:
const items = data?.products ?? [];
const names = items.map(item => item.name);
```

One error you solve yourself is worth a hundred examples you copy.

How to apply it

The minimum viable project

Just like Adafruit designs kits you can finish in an afternoon, your learning projects should be small and completable:

```
Learning React: Build a component catalog (just the list)
Learning Node: Build a REST API for IoT sensors
Learning CSS: Replicate Adafruit's homepage
```

The 20/80 rule

```
20% reading → Understand basic concepts
80% building → Build, break, fix, iterate
```

Share what you build

Limor turned Adafruit into a community where people share their projects, however imperfect. Do the same: push your code to GitHub, write a post, record a short video. Teaching is the most powerful way to consolidate knowledge.

Final reflection

The people who know the most aren't the ones who read the most. They're the ones who built the most. Limor Fried started selling kits from her dorm room. Today Adafruit has over 100 employees. She didn't get there by reading about entrepreneurship. She got there by building, piece by piece, project by project.

Your next step isn't to read more. It's to open your editor and start building. The first 15 minutes are crucial: make them count.

Write code that's easy to delete, not easy to extend

Por Tania Rascia - Architecture

Tania Rascia, developer and technical writer, promotes a liberating idea: **the best code isn't the most extensible, it's the code you can delete without fear**.

The extensibility obsession

```
// "Just in case we need more notification types"
class NotificationFactory {
  static create(type, options) {
    switch(type) {
      case 'email': return new EmailNotification(options);
      case 'sms': return new SMSNotification(options);
      case 'push': return new PushNotification(options);
      // Ready for 10 more types we'll never use
    }
  }
}

// 6 months later: we only use email
// But we have 500 lines of "infrastructure"
```

Code you can delete

```
// Simple, direct, deletable
async function sendEmailNotification(user, message) {
  await emailService.send({
    to: user.email,
    subject: 'Notification',
    body: message
  });
}

// If tomorrow you need SMS:
// 1. Create sendSMSNotification
// 2. Use it where you need it
// 3. You didn't touch existing code
```

Why being able to delete matters

1. Code is temporary

```
Real codebase statistics:
- 50% of code is rewritten in 2 years
- 20% is never used (dead code)
- 30% stays the same

If 70% will change or die, why optimize for extension?
```

Signs of hard-to-delete code

1. **Many circular dependencies**
2. **"Core" module that everything uses**
3. **Hidden side effects**
4. **Global state everywhere**

How to write deletable code

Independent modules

```
// Each feature is self-contained
/features
  /auth
    - login.js
    - signup.js
  /payments
    - checkout.js
    - paymentService.js

// Deleting /payments doesn't affect /auth
```

Final reflection

Tania reminds us that code isn't a monument. It's a temporary tool that solves today's problems. When you write code thinking that someday you'll delete it, you write simpler, cleaner, and paradoxically, more lasting code.

Git is your time machine, use it well

Por Julia Evans - Tools

Julia Evans has explained Git better than anyone with her visual zines. Her advice: **Git isn't just for saving code, it's for understanding your project's history**.

Commits that tell stories

```
# ❌ Useless commits
git commit -m "fix"
git commit -m "update"
git commit -m "changes"

# ✅ Commits that document
git commit -m "fix: prevent double-submit on payment form"
git commit -m "feat: add email validation to signup"
git commit -m "refactor: extract auth logic to separate module"
```

In 6 months, `git log` will be your best documentation.

Commands that change everything

Understanding what happened

```
# Who wrote this line and why?
git blame src/components/Auth.js

# When was this bug introduced?
git bisect start
git bisect bad HEAD
git bisect good v1.0.0
# Git automatically finds the problematic commit
```

Undoing with confidence

```
# "I want to see how this file was yesterday"
git show HEAD~1:src/app.js

# "I need to revert the last commit"
```

```
git revert HEAD

# "I want to discard local changes in a file"
git checkout -- src/file.js
```

Work in progress

```
# Save your work without committing
git stash push -m "WIP: payments feature"

# List what you have saved
git stash list

# Recover your work
git stash pop
```

The workflow that works

```
# 1. Create branch for your feature
git checkout -b feature/new-functionality

# 2. Small, frequent commits
git add -p  # Add changes interactively
git commit -m "add: button component structure"

# 3. Before merging, clean up
git rebase -i main

# 4. Merge with clean history
git checkout main
git merge feature/new-functionality
```

Final reflection

Julia shows us that Git is more than `add-commit-push`. It's a time travel system that lets you understand, experiment, and undo without fear. Investing time in learning it well saves you hours of frustration.

Explicit is better than implicit

Por Guido van Rossum - Fundamentals

Guido van Rossum, creator of Python, included this gem in the Zen of Python: **"Explicit is better than implicit"**. And it applies to all code, not just Python.

Implicit is magical (and dangerous)

```
// ❌ Implicit magic
function processUser(user) {
  // Where does LOCALE come from? And API_URL?
  const name = formatName(user.name, LOCALE);
  const response = await fetch(API_URL + '/users');
}

// What does this function need? You don't know without reading all the coc
```

Explicit is clear

```
// ✅ Explicit dependencies
function processUser(user, { locale, apiUrl }) {
  const name = formatName(user.name, locale);
  const response = await fetch(`${apiUrl}/users`);
}

// Usage:
processUser(user, {
  locale: 'en-US',
  apiUrl: 'https://api.example.com'
});

// Now you know exactly what the function needs
```

Examples in different contexts

Imports

```
// ❌ Implicit: what am I importing?
import * as utils from './utils';
utils.something();

// ✅ Explicit: clear what I use
import { formatDate, parseJSON } from './utils';
```

Props in React

```
// ❌ Implicit: spread of unknown props
function Button(props) {
  return <button {...props} />;
}

// ✅ Explicit: documented props
function Button({ onClick, disabled, children, variant = 'primary' }) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn btn-${variant}`}
    >
      {children}
    </button>
  );
}
```

The cost of implicit

1. **Harder debugging**: "Where does this value come from?"
2. **Slower onboarding**: New devs don't understand conventions
3. **Fragile tests**: Hidden dependencies break tests
4. **Dangerous refactoring**: You don't know what depends on what

Final reflection

Guido created Python with readability as priority. Explicit requires more characters but saves hours of debugging and confusion. When in doubt between being clever and implicit or verbose and explicit, choose explicit.

Errors are data, not exceptions

Por Rich Harris - Clean Code

Rich Harris, besides creating Svelte, has promoted a shift in how we think about errors: **errors aren't unexpected exceptions, they're possible results you should handle**.

The problem with try/catch

```
// ❌ Try/catch hides the error flow
async function getUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);
    const user = await response.json();
    return user;
  } catch (error) {
    // What failed? The fetch? The JSON? Something else?
    console.error(error);
    return null;
  }
}
```

Errors as data

```
// ✅ The error is part of the result
async function getUser(id) {
  const response = await fetch(`/api/users/${id}`);

  if (!response.ok) {
    return {
      ok: false,
      error: {
        type: 'http',
        status: response.status,
        message: `Failed to fetch user: ${response.statusText}`
      }
    };
  }

  try {
    const user = await response.json();
    return { ok: true, data: user };
```

```
  } catch {
    return {
      ok: false,
      error: { type: 'parse', message: 'Invalid JSON response' }
    };
  }
}
```

The Result pattern

```
// Define a generic Result type
type Result<T, E = Error> =
  | { ok: true; data: T }
  | { ok: false; error: E };

// Use it in your functions
async function divide(a: number, b: number): Result<number, string> {
  if (b === 0) {
    return { ok: false, error: 'Cannot divide by zero' };
  }
  return { ok: true, data: a / b };
}
```

Why it's better

1. Visible errors

With try/catch, errors are invisible in the signature. With Result, errors are explicit.

2. Forced handling

Try/catch allows ignoring errors silently. Result forces you to handle them.

Final reflection

Rich and others have brought this idea from languages like Rust and Go. Treating errors as data, not exceptions, makes your code more predictable, easier to test, and harder to accidentally ignore.

The README is your first impression

Por Tania Rascia - Documentation

Tania Rascia, known for her clear and accessible documentation, has a rule for her projects: **if someone can't understand and run your project in 5 minutes reading the README, you've failed**.

The minimum viable README

```
# Project Name

One line explaining what it does.

## Installation

\`\`\`bash
npm install my-project
\`\`\`

## Quick usage

\`\`\`javascript
import { thing } from 'my-project';
thing.doSomething();
\`\`\`

## License

MIT
```

This is better than nothing. But you can do more.

The README that converts

A good README should have:

1. Immediate hook

```
❌ "This is a project for..."
✅ "Authentication that works. No pain."
```

```
The first line sells the project.
```

2. Copy-pasteable installation

```
❌ "First make sure you have Node 18+, then..."
✅
\`\`\`bash
npm install thing
\`\`\`

One line that works.
```

3. Working example

```
❌ Pseudocode that no one can run
✅ Real code that you copy-paste and it works
```

For personal/portfolio projects

```
# 🌟 My Project

> A Twitter clone built to learn React and Node.

## Live Demo

👉 [See demo](https://my-project.vercel.app)

## Technologies

- React 18 + TypeScript
- Node.js + Express
- PostgreSQL + Prisma
```

Final reflection

Tania treats every README as if it were a product landing page. The README is the first thing recruiters, collaborators, and users see. Invest time in it: it's your cover letter.

The terminal is your superpower

Por Jessie Frazelle - Tools

Jessie Frazelle was a core maintainer of Docker, contributed to Kubernetes and Go, and added the default Seccomp profile to Docker. She's famous for running **everything** in containers from the terminal: Chrome, Spotify, LibreOffice, even desktop apps. Her philosophy: **the terminal is your complete operating environment, and containers are the layer that makes it secure and reproducible**.

Everything is a container

Why run a browser inside Docker? Because every application should run isolated, with minimal permissions, without polluting your system.

```
# Run Chrome in a container
docker run -d \
  --memory 512mb \
  --net host \
  --security-opt seccomp=chrome.json \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -e DISPLAY=unix$DISPLAY \
  -v $HOME/Downloads:/home/user/Downloads \
  --name chrome \
  jess/chrome
```

If Chrome gets compromised, it has no access to your SSH keys, API tokens, or work files. **Security by default, not by effort.**

Essential commands

```
# Fast text search
rg "TODO" --type js

# Follow a log in real time
tail -f /var/log/app.log

# What's using port 3000?
lsof -i :3000

# Kill process by port
kill $(lsof -t -i:3000)
```

```
# Disk usage sorted by size
du -sh * | sort -rh | head -20
```

Docker-first development

```
# Disposable Node.js environment
docker run --rm -it -v $(pwd):/app -w /app node:20 bash

# Run with multi-layer security
docker run --rm \
  --security-opt seccomp=custom-profile.json \
  --read-only \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  my-app

# Inspect container capabilities
docker inspect --format='{{.HostConfig.CapAdd}}' my-container
```

Aliases that change everything

```
# Git
alias gs="git status"
alias gc="git commit -m"
alias gl="git log --oneline -15"

# Docker (Jessie-style)
alias dps="docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'"
alias dprune="docker system prune -af --volumes"
alias dlog="docker logs -f"
alias dex="docker exec -it"

# Quick cleanup
alias rmnode="find . -name 'node_modules' -type d -prune -exec rm -rf {} +"
```

Modern terminal tools

```
# fzf: fuzzy search for files, history, branches
# bat: cat with syntax highlighting
# eza: modern ls with git info
# jq: manipulate JSON like a pro
```

```
docker inspect my-container | jq '.[0].NetworkSettings.IPAddress'
# lazydocker: Docker dashboard in your terminal
```

Jessie's lesson

Jessie Frazelle proved you never need to leave the terminal. Her approach goes beyond productivity: every process isolated in a container is an extra security layer, every alias is a second saved that multiplies thousands of times, and every script is a task you'll never have to repeat manually.

The terminal isn't "old school". It's your operating environment, your security system, and your productivity multiplier, all in one.

The best bug is the one that never reaches production

Por Guillermo Rauch - Testing

Guillermo Rauch, with his philosophy of continuous CI/CD at Vercel, promotes a simple principle: **every layer of prevention you add before production multiplies your peace of mind**.

The layers of defense

```
                PRODUCTION
                     ↑
          ┌──────────────────┐
          │    Monitoring    │  ← Detects after the fact
          └──────────────────┘
                     ↑
          ┌──────────────────┐
          │ Preview Deploys  │  ← Detects before merge
          └──────────────────┘
                     ↑
          ┌──────────────────┐
          │    E2E Tests     │  ← Detects broken flows
          └──────────────────┘
                     ↑
          ┌──────────────────┐
          │    Unit Tests    │  ← Detects broken logic
          └──────────────────┘
                     ↑
          ┌──────────────────┐
          │    Type Check    │  ← Detects type errors
          └──────────────────┘
                     ↑
          ┌──────────────────┐
          │     Linting      │  ← Detects obvious errors
          └──────────────────┘
                     ↑
                YOUR CODE
```

Each layer catches different types of errors.

The cost of each layer

```
| Layer           | Time      | Cost of bug found       |
|-----------------|-----------|-------------------------|
| Linting         | Seconds   | $0 - just fix           |
| TypeScript      | Seconds   | $0 - doesn't compile    |
| Unit Tests      | Minutes   | $10 - local fix         |
| E2E Tests       | Minutes   | $100 - debugging        |
| Preview         | Hours     | $1,000 - PR blocked     |
| Production      | N/A       | $10,000+ - users affected |
```

Final reflection

Guillermo designed Vercel so the path to production has multiple checkpoints. Every bug you catch before production is an incident you don't have to manage at 3am. Invest in prevention, not reaction.

Don't optimize what you didn't measure

Por Donald Knuth - Performance

Donald Knuth, legend of computer science, gave us the famous quote about premature optimization. But the complete context is even more valuable: **measure before optimizing, because your intuition deceives you**.

Your intuition lies

```
// "This map is slow, I should use a classic for"
const result = items.map(x => x * 2);

// After measuring: map takes 0.3ms
// The API fetch takes 800ms

// You optimized what didn't matter
```

How to measure correctly

```
// Basic measurement
console.time('operation');
await yourOperation();
console.timeEnd('operation');
```

What to measure

1. **API response time**
2. **Render time**
3. **Memory usage**
4. **Bundle size**

Final reflection

Knuth didn't say "never optimize". He said "don't optimize without data". The difference between a novice optimizer and an expert is that the expert measures first.

Names reveal intention

Por Robert C. Martin - Readability

Uncle Bob insists: **a good name eliminates the need for comments**. If you need a comment to explain what a variable does, the name is wrong.

Names that hide

```
// ❌ What is this?
const d = 7;
const list = getData();

// ✅ Now it's clear
const daysUntilExpiration = 7;
const activeUsers = getActiveUsers();
```

Practical rules

1. **Use verbs for functions**: `getUser`, `calculateTotal`
2. **Use nouns for variables**: `userName`, `totalAmount`
3. **Booleans with is/has/can**: `isActive`, `hasPermission`

Final reflection

Naming is one of the hardest things in programming. But it's where investing time pays off the most.

Mutable state is the root of all evil

Por Rich Harris - Architecture

Rich Harris, although he created Svelte with mutable state for simplicity, recognizes a fundamental principle: **the less mutable state you have, the fewer bugs you'll have**.

The shared state problem

```
// ❌ Shared mutable state
let cart = [];

function addItem(item) {
  cart.push(item); // Mutation
}

// Somewhere else in the code...
cart = []; // Someone resets the cart unexpectedly
```

Immutable state

```
// ✅ Each operation returns new state
function addItem(cart, item) {
  return [...cart, item];
}

// State is never modified, it's replaced
let cart = [];
cart = addItem(cart, { id: 1, price: 10 });
```

Benefits

1. **Predictability**: Same input = same output
2. **Easy debugging**: You can "time travel"
3. **Safe concurrency**: No race conditions

Final reflection

You don't need to eliminate all mutable state. But each piece of mutable state is a place where bugs can hide. Minimize it where you can.

The user doesn't wait: every millisecond counts

Por Addy Osmani - Performance

Addy Osmani has extensively documented how speed affects user experience: **100ms is instant, 1 second is noticeable, 10 seconds is abandonment**.

Perception thresholds

```
| Time       | User perception           |
|------------|---------------------------|
| 0-100ms    | Instant                   |
| 100-300ms  | Slightly noticeable       |
| 300-1000ms | App feels slow            |
| 1-10s      | Loses focus               |
| 10s+       | Abandons                  |
```

High-impact optimizations

```html
<!-- Prioritize LCP image -->
<img src="hero.jpg" fetchpriority="high" />

<!-- Lazy load what's not visible -->
<img src="footer.jpg" loading="lazy" />
```

Final reflection

Perceived speed is as important as actual speed. Sometimes optimizing perceived experience is more effective than optimizing code.

Don't reinvent the wheel (unless you're learning about wheels)

Por Kent C. Dodds - Productivity

Kent C. Dodds is pragmatic: **use existing solutions for production, reinvent for learning**. The distinction is crucial.

When to use existing

```
// ❌ Reinventing email validation
const emailRegex = /^[a-zA-Z0-9... // 200 chars of regex

// ✅ Use proven library
import { isEmail } from 'validator';
```

When to reinvent

```
// To learn how React works:
function useState(initialValue) {
  // Your basic implementation
  // Now you understand hooks
}

// For production:
import { useState } from 'react';
```

Final reflection

The best solutions to common problems already exist. Your time is better invested in solving your business's unique problems, not rewriting what others have solved.

Code is communication between humans

Por Martin Fowler - Readability

Martin Fowler sums it up perfectly: **"Any fool can write code that a computer can understand. Good programmers write code that humans can understand"**.

Code for machines vs. humans

```
// ❌ Machine understands, human doesn't
const r = d.filter(i => i.a && i.b > 5).map(i => ({...i, c: i.b * 2}));

// ✅ Both understand
const activeHighValueItems = items
  .filter(item => item.isActive && item.value > 5)
  .map(item => ({
    ...item,
    doubledValue: item.value * 2
  }));
```

Final reflection

Code is read 10 times more than it's written. Optimize for reading, not writing.

Simplicity is not simplistic

Por Edsger W. Dijkstra - Architecture

Dijkstra distinguished between **simple** (easy to understand) and **simplistic** (inadequately simple). Real simplicity is hard to achieve.

Simple vs. simplistic

```
// Simplistic: ignores important cases
function divide(a, b) {
  return a / b; // What if b is 0?
}

// Simple: handles what's necessary, no more
function divide(a, b) {
  if (b === 0) return { ok: false, error: 'Division by zero' };
  return { ok: true, result: a / b };
}
```

Final reflection

"Make things as simple as possible, but not simpler" - Einstein. Simplicity is the result of eliminating the unnecessary, not ignoring the important.

TypeScript makes you faster, not slower

Por Anders Hejlsberg - Tools

Anders Hejlsberg, creator of TypeScript (and C# and Turbo Pascal), designed TypeScript with one goal: **make developers more productive, not more bureaucratic**.

What TypeScript gives you

1. **Intelligent autocomplete**: `user.` shows you all properties
2. **Compile-time errors**: Don't wait for users to report bugs
3. **Safe refactoring**: Renaming a function updates all code

Final reflection

The time you "lose" writing types you recover multiplied in less debugging, better documentation, and fearless refactoring.

Tests are the best documentation

Por Kent Beck - Testing

Kent Beck sees tests as more than verification: **they're executable examples of how to use your code**.

Tests as documentation

```
describe('Cart', () => {
  it('adds items and calculates total', () => {
    const cart = new Cart();
    cart.add({ name: 'Laptop', price: 1000 });
    cart.add({ name: 'Mouse', price: 50 });

    expect(cart.total).toBe(1050);
  });
});

// These tests tell you exactly how to use Cart
// And unlike comments, they're always up to date
```

Final reflection

Next time you write tests, think: "Would this help a new developer understand how to use this module?"

Code reviews are for learning, not criticizing

Por Sarah Drasner - Team

Sarah Drasner, VP of Developer Experience at Netlify, promotes a positive code review culture: **the goal is to improve code and the team, not prove who knows more**.

Reviews that help

```
❌ "This code is bad"
✅ "What do you think about extracting this logic to a function? It would b

❌ "Don't use var"
✅ "I prefer const here because the value doesn't change. What do you think
```

Final reflection

The best code reviews are conversations, not judgments. The author learns, the reviewer learns, and the code improves.

Start from the end: write first how you want to use the code

Por Kent Beck - Process

Kent Beck has a trick for designing APIs: **write first the code that will use your function, then implement the function**.

Design from usage

```
// 1. First I write how I WANT to use it
const user = await User.findByEmail('user@example.com');
const orders = await user.getRecentOrders({ limit: 5 });
```

Why it works

When you start from implementation, you create awkward APIs. When you start from usage, you create natural APIs.

Final reflection

TDD takes this idea to the extreme: you write the test (usage) before the code. Even without strict TDD, thinking about usage first improves your design.

Logs are your best friend in production

Por Kelsey Hightower - DevOps

Kelsey Hightower insists: **in production, logs are your only window into what's happening**.

Useful vs. useless logs

```
// ❌ Useless
console.log('here');

// ✅ Useful
logger.info('Payment processed', {
  userId: user.id,
  amount: payment.amount,
  transactionId: payment.id
});
```

Final reflection

Well-structured logs save you hours of debugging. Invest time in making them useful.

Every dependency is debt you'll have to pay

Por David Heinemeier Hansson - Architecture

DHH is known for minimizing dependencies: **every npm install is a promise of future maintenance**.

Before installing, ask

1. Can I do this with native JavaScript?
2. How much of the package will I actually use?
3. Is it actively maintained?

Final reflection

I'm not saying "don't use dependencies". I'm saying "evaluate the cost". Sometimes 20 lines of your own code is better than a 2000-line dependency.

The first step to fixing a bug is reproducing it

Por Julia Evans - Debugging

Julia Evans has a methodical process for debugging: **if you can't reproduce the bug, you can't fix it with confidence**.

Julia's process

```
1. Reproduce the bug locally
2. Write a test that fails because of the bug
3. Fix the bug
4. The test now passes
5. The bug can't come back
```

Final reflection

A bug you can't reproduce is a bug that will return. Invest time in reproducing it before trying to fix it.

Separate what changes from what stays the same

Por Sandi Metz - Architecture

Sandi Metz identifies a pattern in well-designed code: **code that changes together should live together**.

Final reflection

When something changes, how many files do you have to modify? If it's many, something is coupled that shouldn't be.

Performance degrades gradually: always monitor

Por Addy Osmani - DevOps

Addy Osmani warns: **performance doesn't break suddenly, it degrades little by little until it's a problem**.

Solution: Performance budgets

```
// In your CI
const budget = {
  maxBundleSize: 250 * 1024, // 250KB
  maxLCP: 2500, // 2.5s
};
```

Final reflection

What you don't measure degrades. Automate performance alerts before it becomes a problem.

Divide the problem until each part is trivial

Por Edsger W. Dijkstra - Process

Dijkstra was a master at decomposing problems: **a difficult problem is simply many easy problems together**.

Final reflection

When you're stuck, don't try to solve everything at once. Divide until each part is obvious.

First make it work, then make it beautiful

Por Dan Abramov - Process

Dan Abramov constantly reminds us: **the first version of any code will be ugly, and that's okay**.

Final reflection

Ugly code that works is infinitely more valuable than beautiful code that doesn't exist. Refactor afterwards, not during.

Context is everything: there are no universal best practices

Por Kent C. Dodds - Mindset

Kent C. Dodds has fought against dogmatism: **what works for Google doesn't necessarily work for your 3-person startup**.

The right question

Don't ask "What's the best practice?"

Ask "What's the best practice **for my context**?"

Final reflection

Dogmas are comfortable because they avoid thinking. But good engineering is exactly that: thinking about the specific context.

Software is a process, not a finished product

Por Martin Fowler - Mindset

Martin Fowler says it clearly: **software is never "finished", it's always evolving**.

Final reflection

Don't try to predict the future of software. Create the conditions for change to be cheap and safe.

Don't assume the next person knows: document decisions

Por Sarah Drasner - Documentation

Sarah Drasner promotes documenting decisions, not just code: **the "why" is more valuable than the "what"**.

ADRs: Architecture Decision Records

Document why decisions were made, not just what they were.

Final reflection

In 2 years, no one will remember why PostgreSQL was chosen. But if it's documented, the next team won't repeat the investigation.

Learn to say 'no' or 'not yet'

Por David Heinemeier Hansson - Career

DHH is famous for saying no: **every feature you don't build is maintenance you don't have**.

Final reflection

The most successful software isn't the one with the most features. It's the one that solves a specific problem well.

If you do it three times, automate it

Por Jeff Atwood - Productivity

Jeff Atwood, co-founder of Stack Overflow, applies a simple rule: **if you do something more than three times, automate it**.

Final reflection

Time invested in automation is recovered exponentially. But don't automate too early: first do it 3 times manually.

Take care of your mental health: burnout is real

Por Cassidy Williams - Career

Cassidy Williams speaks openly about burnout in tech: **your career is a marathon, not a sprint**.

Final reflection

No deploy is more important than your health. Companies survive without you; you don't survive without taking care of yourself.

Before how, ask what for

Por Barbara Liskov - Process

Barbara Liskov, Turing Award winner, emphasizes understanding the problem: **the right solution to the wrong problem is still wrong**.

Final reflection

Half of your job is understanding what to build. The other half is building it.

Early feedback is worth gold

Por Guillermo Rauch - Process

Guillermo Rauch built Vercel around an idea: **seeing your change in production in seconds changes how you develop**.

Final reflection

The faster you see the result of your code, the faster you learn and the better your code gets.

Humility makes you a better developer

Por Linus Torvalds - Mindset

Linus Torvalds, despite his reputation for being direct, recognizes something: **the best programmers know they don't know everything**.

Final reflection

The best engineers I know are the most willing to say "I don't know".

Code is temporary, the problems it solves matter

Por Grace Hopper - Mindset

Grace Hopper, programming pioneer, understood something fundamental: **technology changes, human problems remain**.

Final reflection

In 10 years, the framework you use today will be legacy. But the skills of solving problems, communicating, and learning will always accompany you.

Share what you know: the community gives back more

Por Cassidy Williams - Community

Cassidy Williams has built a career by sharing: **teaching is the best way to learn, and the community gives you more than you give**.

Final reflection

Everything you know you learned from someone who shared. Return the favor.

Never stop learning: curiosity is your superpower

Por Ada Lovelace - Career

Ada Lovelace, the first programmer in history, had a quality that defines the best developers: **insatiable curiosity**.

The beginner's mindset

```
The expert who stops learning becomes obsolete.
The curious beginner never stops improving.
```

How to maintain curiosity

```
- Read code from projects you admire
- Try a new technology each month
- Ask "why?" constantly
- Accept that there will always be more to learn
- Surround yourself with people who challenge you
```

What history teaches

Ada saw potential in machines that nobody else saw. Grace Hopper imagined programming languages that seemed impossible. Linus created an operating system "for fun".

They all had one thing in common: **limitless curiosity**.

Final reflection

You've reached tip number 100. But this isn't the end - it's just the beginning. Technology will change. Frameworks will come and go. But if you keep curiosity burning, you'll always be ready for whatever comes next.

Never stop learning. Never stop asking. Never stop building.